

PSY9613 (Automne 2026)

Perception, cognition et intelligence artificielle

Intelligence artificielle:

Perspectives historiques (problématiques et conceptuelles)

Pierre Poirier

Département de philosophie, UQAM

Aujourd'hui

- L'explication de la cognition (un vieux problème)
- La solution computationnelle / symbolique
- La solution connexionniste

Aujourd'hui

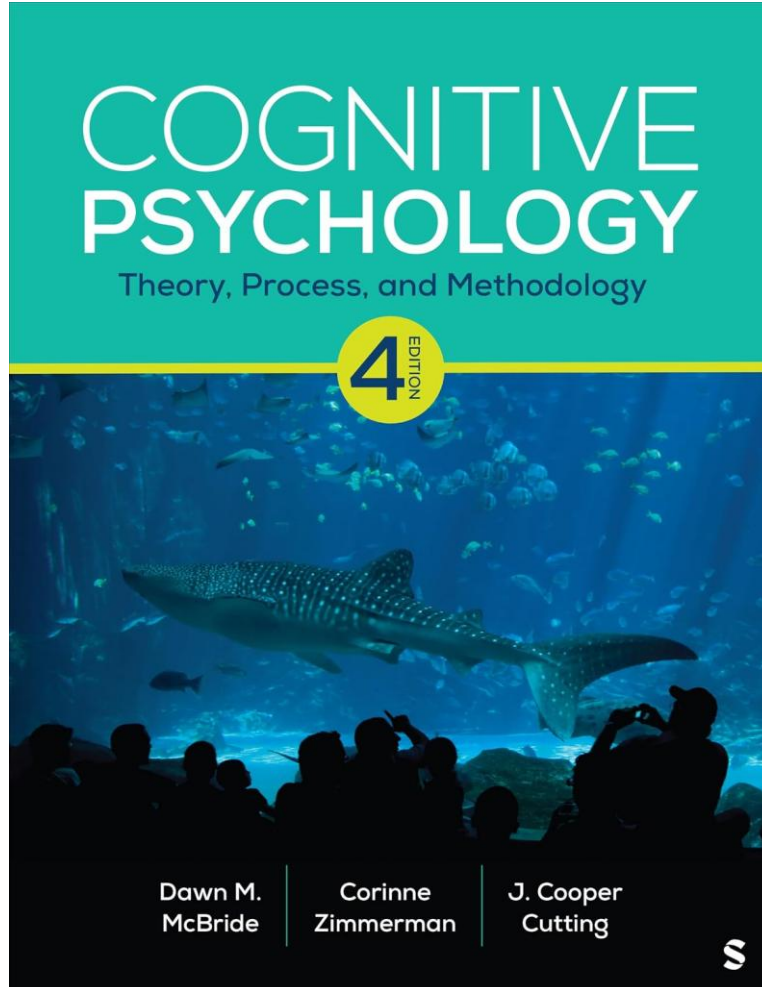
- **L'explication de la cognition (un vieux problème)**
- La solution computationnelle / symbolique
- La solution connexionniste

L'explication de la cognition (un vieux problème)



J'avais expliqué assez particulièrement toutes ces choses dans le traité que j'avais eu ci-devant dessein de publier. Et ensuite j'y avais montré quelle doit être la fabrique des nerfs et des muscles du corps humain, pour faire que les esprits animaux, étant dedans, aient la force de mouvoir ses membres: ainsi, qu'on voit que les têtes, un peu après être coupées, se remuent encore et mordent la terre, nonobstant qu'elles ne soient plus animées: quels changements se doivent faire dans cerveau pour causer la veille, et le sommeil et les songes; comment la lumière, les sons, les odeurs, les goûts, la chaleur, et toutes les autres qualités des objets extérieurs y peuvent imprimer diverses idées par l'entremise des sens; ... comment la faim, la soif, et les autres passions intérieures, y peuvent aussi envoyer les leurs; ce qui doit y être pris pour le sens commun, où ces idées sont reçues; pour la mémoire, qui les conserve; et pour la fantaisie, qui les peut diversement changer et en composer de nouvelles, et par même moyen, distribuant les esprits animaux dans les muscles, faire mouvoir les membres de ce corps en autant de diverses façons, et autant à propos des objets qui se présentent à ses sens, et des passions intérieures qui sont en lui, que les nôtres se puissent mouvoir, sans que la volonté les conduise. (pp. 163-164)

L'explication de la cognition (un vieux problème)



CONTENTS

[Preface](#)

[Acknowledgments](#)

[About the Authors](#)

[Chapter 1 Introduction to Cognitive Psychology](#)

[Chapter 2 Cognitive Neuroscience](#)

[Chapter 3 Perception](#)

[Chapter 4 Attention](#)

[Chapter 5 Memory Structures and Processes](#)

[Chapter 6 Long-Term Memory Retrieval](#)

[Chapter 7 Memory Errors](#)

[Chapter 8 Language](#)

[Chapter 9 Concepts and Knowledge](#)

[Chapter 10 Imagery](#)

[Chapter 11 Problem Solving](#)

[Chapter 12 Reasoning](#)

[Chapter 13 Decision Making](#)

[Chapter 14 Cognitive Development](#)

[Appendix: Stop and Think Answers](#)

[Glossary](#)

[References](#)

[Author Index](#)

L'explication de la cognition (un vieux problème)



Ce qui ne semblera nullement étrange à ceux qui, sachant combien de divers *automates, ou* machines mouvantes, l'industrie des hommes peut faire, sans y employer que fort peu de pièces, à comparaison de la grande multitude des os, des muscles, des nerfs, des artères, des veines, et de toutes les autres parties qui sont dans le corps de chaque animal, considéreront ce corps comme une machine, qui, ayant été faite des mains de Dieu est incomparablement mieux ordonnée et, a en soi des mouvements plus admirables, qu'aucune de celles qui peuvent être inventées par les hommes. Et je m'étais ici particulièrement arrêté à faire voir que, s'il y avait de telles machines, qui eussent les organes et la figure extérieure d'un singe, ou de quelque autre animal sans raison nous n'aurions aucun moyen pour reconnaître qu'elles ne seraient pas en tout de même nature que ces animaux, au lieu que, s'il y en avait qui eussent la ressemblance de nos corps et imitassent autant nos actions que moralement il serait possible, nous aurions toujours deux moyens très certains pour reconnaître qu'elles ne seraient point pour cela de vrais hommes. (pp. 163-164)

L'explication de la cognition (un vieux problème)



Dont le premier est que jamais elles ne pourraient user de paroles, ni d'autres signes en les composant, comme nous faisons pour déclarer aux autres nos pensées. Car on peut bien concevoir qu'une machine soit tellement faite qu'elle profère des paroles, et même qu'elle en profère quelques-unes à propos des actions corporelles qui causeront quelque changement en ses organes: comme, si on la touche en quelque endroit, qu'elle demande ce qu'on lui veut dire; si en un autre, qu'elle crie qu'on lui fait mal, et choses semblables; mais non pas qu'elle les arrange diversement, pour répondre au sens de tout ce qui se dira en sa présence, ainsi que les hommes les plus hébétés peuvent faire. Et le second est que, bien qu'elles fissent plusieurs choses aussi bien, ou peut-être mieux qu'aucun de nous, elles manqueraient infailliblement en quelques autres, par lesquelles on découvrirait qu'elles n'agiraient pas par connaissance, mais seulement par la disposition de leurs organes. Car, au lieu que la raison est un instrument universel, qui peut servir en toutes sortes de rencontres, ces organes ont besoin de quelque particulière disposition pour chaque action particulière; d'où vient qu'il est moralement impossible qu'il y en ait assez de divers en une machine pour la faire agir en toutes les occurrences de la vie, de même façon que notre raison nous fait agir. (pp. 164-165)

L'explication de la cognition (un vieux problème)



La production d'une infinité de pensées ou de phrases (productivité)

- Comment un **mécanisme** peut nous faire agir **en toutes les occurrences de la vie (intelligence générale)** de même façon que notre **raison (cognition)** nous fait agir ?

L'explication de la cognition (le même vieux problème)

VOL. LIX. No. 236.]

[October, 1950

MIND

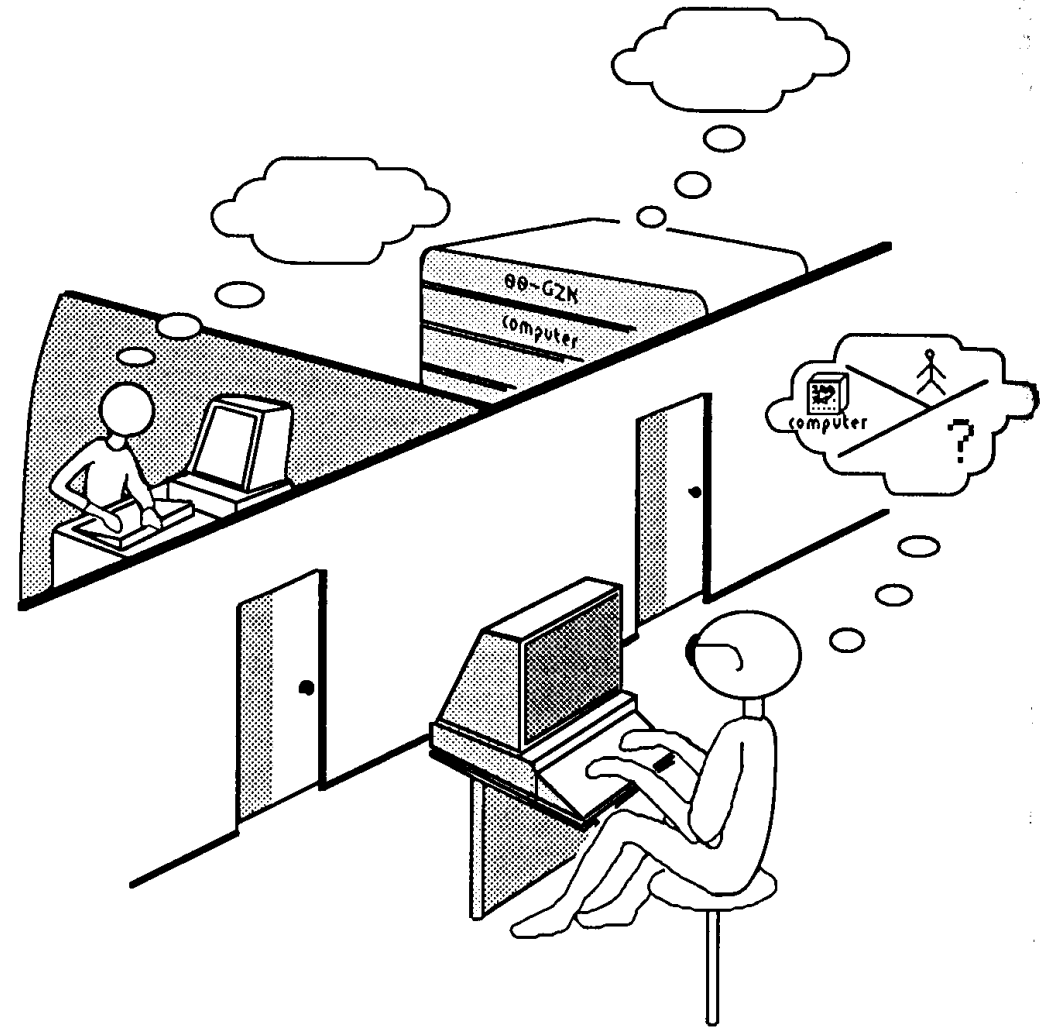
A QUARTERLY REVIEW
OF
PSYCHOLOGY AND PHILOSOPHY

I.—COMPUTING MACHINERY AND INTELLIGENCE

BY A. M. TURING

1. *The Imitation Game.*

I PROPOSE to consider the question, 'Can machines think?' This should begin with definitions of the meaning of the terms 'machine' and 'think'. The definitions might be framed so as to reflect so far as possible the normal use of the words, but this attitude is dangerous. If the meaning of the words 'machine' and 'think' are to be found by examining how they are commonly used it is difficult to escape the conclusion that the meaning



L'explication de la cognition (le même vieux problème)

VOL. LIX. No. 236.]

[October, 1950]

MIND A QUARTERLY REVIEW OF PSYCHOLOGY AND PHILOSOPHY

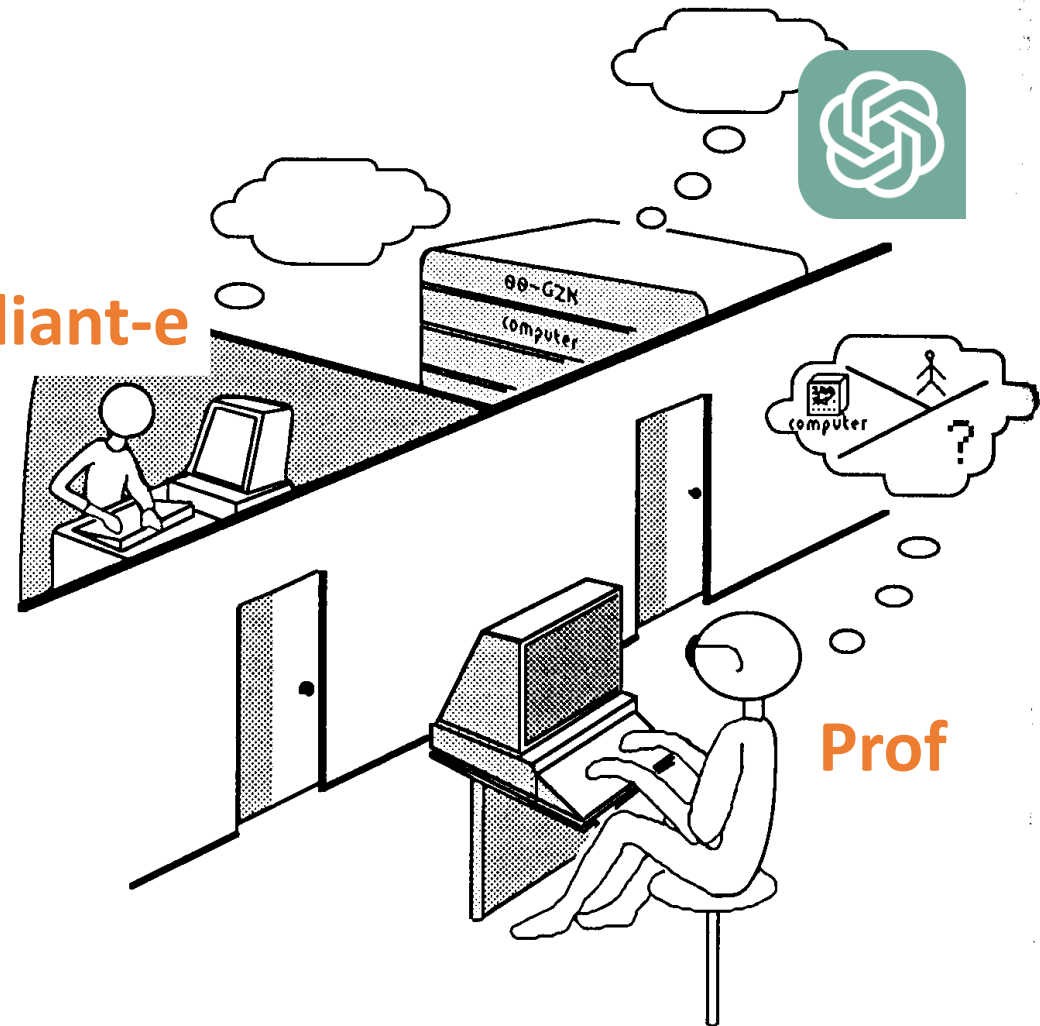
I.—COMPUTING MACHINERY AND INTELLIGENCE

BY A. M. TURING

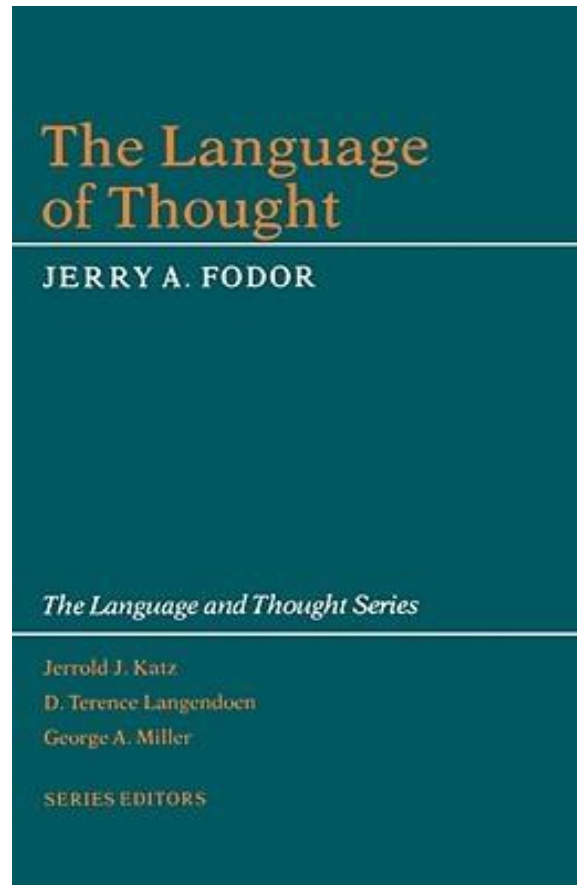
1. *The Imitation Game.*

I PROPOSE to consider the question, 'Can machines think?' This should begin with definitions of the meaning of the terms 'machine' and 'think'. The definitions might be framed so as to reflect so far as possible the normal use of the words, but this attitude is dangerous. If the meaning of the words 'machine' and 'think' are to be found by examining how they are commonly used it is difficult to escape the conclusion that the meaning

Étudiant-e



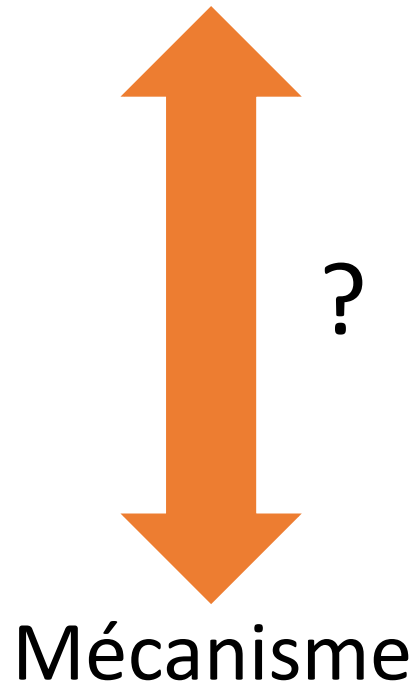
L'explication de la cognition (le même vieux problème)



(...) le système doit posséder une infinité de représentations distinctes. L'argument ici est précisément analogue à l'argument pour la non-finitude des langues naturelles: de même que, dans ce dernier cas, il n'y a pas de limite supérieure à la complexité d'une phrase qui peut être utilisée pour faire un énoncé, de même dans le premier cas, il n'y a pas de limite supérieure à la complexité de la représentation qui peut être requise pour spécifier les options comportementales disponibles pour l'agent, ou la situation dans laquelle il se trouve, ou les conséquences d'agir d'une manière ou d'une autre. (Fodor, 1975, p. 31)

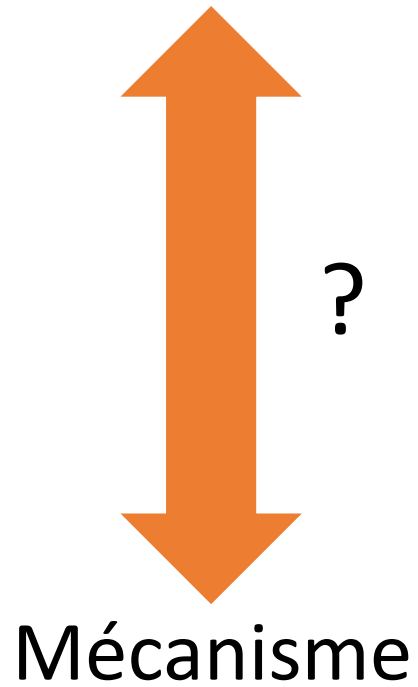
L'explication de la cognition (un vieux problème)

(Infinité de) représentation(s)



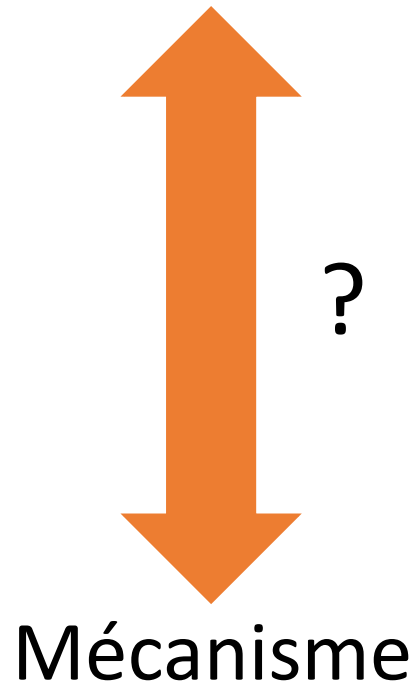
L'explication de la cognition (un vieux problème)

(Infinité de) représentation(s)



Un problème des études de la cognition

(Infinité de) représentation(s)



Aujourd'hui

- L'explication de la cognition (un vieux problème)
- **La solution computationnelle / symbolique**
- La solution connexionniste

La solution computationnelle / symbolique

230

A. M. TURING

[Nov. 12,

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM

By A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]

The “computable” numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means. Although the subject of this paper is ostensibly the computable *numbers*, it is almost equally easy to define and investigate computable functions of an integral variable or a real or computable variable, computable predicates, and so forth. The fundamental problems involved are, however, the same in each case, and I have chosen the computable numbers for explicit treatment as involving the least cumbersome technique. I hope shortly to give an account of the relations of the computable numbers, functions, and so forth to one another. This will include a development of the theory of functions of a real variable expressed in terms of computable numbers. According to my definition, a number is computable if its decimal can be written down by a machine.

In §§ 9, 10 I give some arguments with the intention of showing that the computable numbers include all numbers which could naturally be regarded as computable. In particular, I show that certain large classes of numbers are computable. They include, for instance, the real parts of all algebraic numbers, the real parts of the zeros of the Bessel functions, the numbers π , e , etc. The computable numbers do not, however, include all definable numbers, and an example is given of a definable number which is not computable.

Although the class of computable numbers is so great, and in many ways similar to the class of real numbers, it is nevertheless enumerable. In § 8 I examine certain arguments which would seem to prove the contrary. By the correct application of one of these arguments, conclusions are reached which are superficially similar to those of Gödel†. These results

† Gödel, “Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme, I”, *Monatshefte Math. Phys.*, 38 (1931), 173–198.



La solution computationnelle / symbolique

230

A. M. TURING

[Nov. 12,

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM

By A. M. TURING.

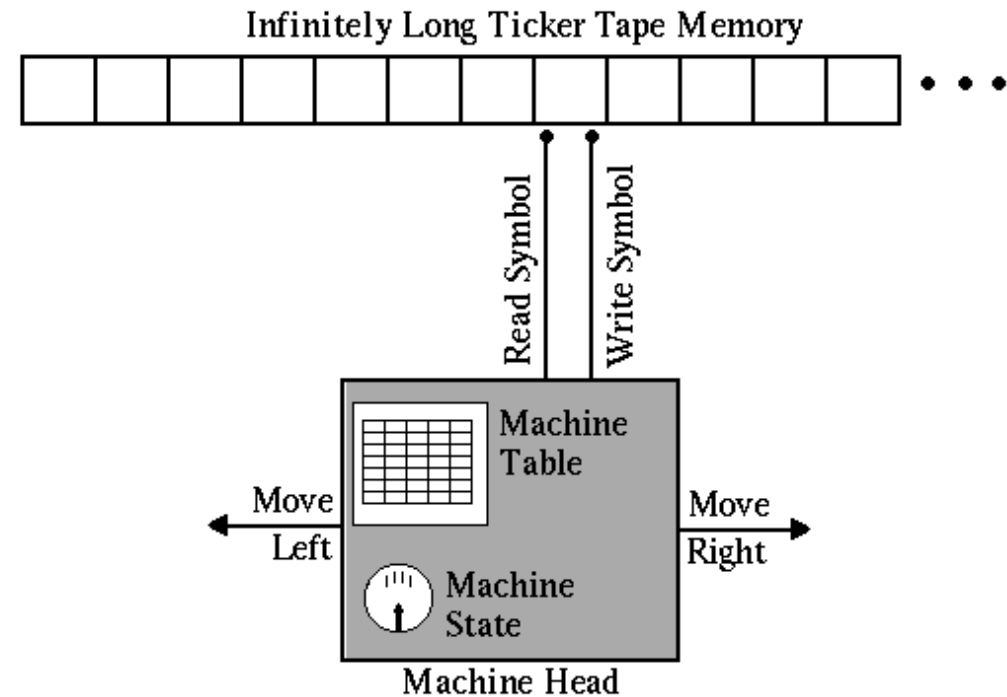
[Received 28 May, 1936.—Read 12 November, 1936.]

The “computable” numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means. Although the subject of this paper is ostensibly the computable *numbers*, it is almost equally easy to define and investigate computable functions of an integral variable or a real or computable variable, computable predicates, and so forth. The fundamental problems involved are, however, the same in each case, and I have chosen the computable numbers for explicit treatment as involving the least cumbersome technique. I hope shortly to give an account of the relations of the computable numbers, functions, and so forth to one another. This will include a development of the theory of functions of a real variable expressed in terms of computable numbers. According to my definition, a number is computable if its decimal can be written down by a machine.

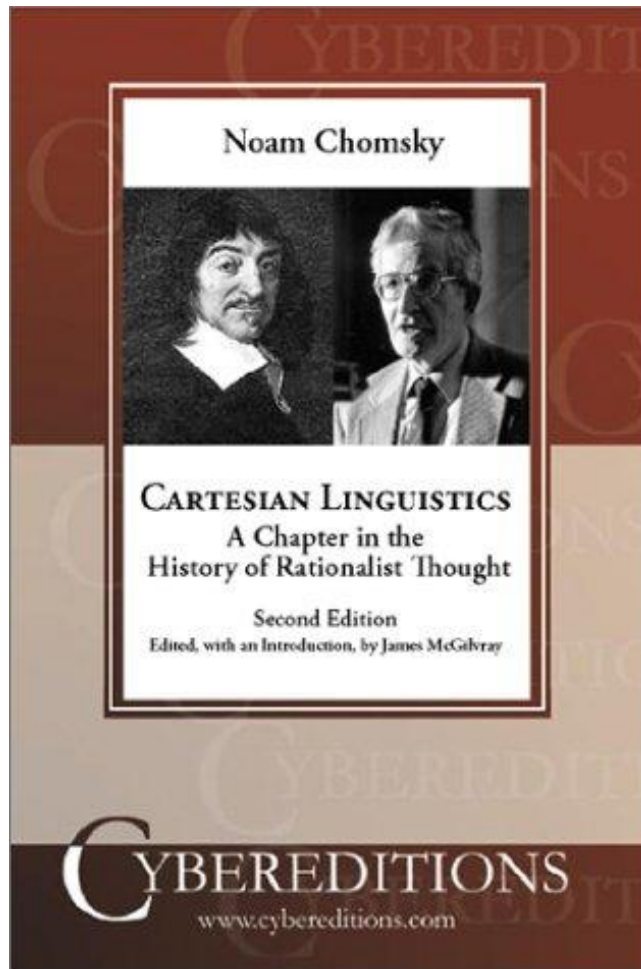
In §§ 9, 10 I give some arguments with the intention of showing that the computable numbers include all numbers which could naturally be regarded as computable. In particular, I show that certain large classes of numbers are computable. They include, for instance, the real parts of all algebraic numbers, the real parts of the zeros of the Bessel functions, the numbers π , e , etc. The computable numbers do not, however, include all definable numbers, and an example is given of a definable number which is not computable.

Although the class of computable numbers is so great, and in many ways similar to the class of real numbers, it is nevertheless enumerable. In § 8 I examine certain arguments which would seem to prove the contrary. By the correct application of one of these arguments, conclusions are reached which are superficially similar to those of Gödel†. These results

† Gödel, “Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme, I”, *Monatshefte Math. Phys.*, 38 (1931), 173–198.

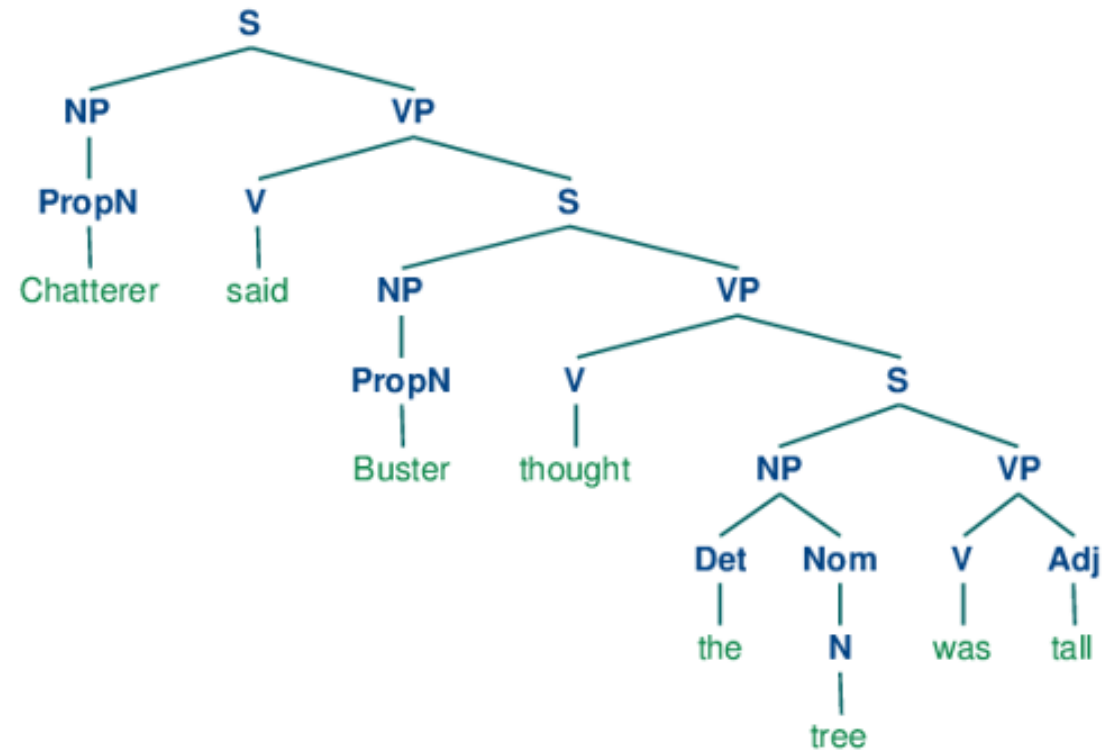
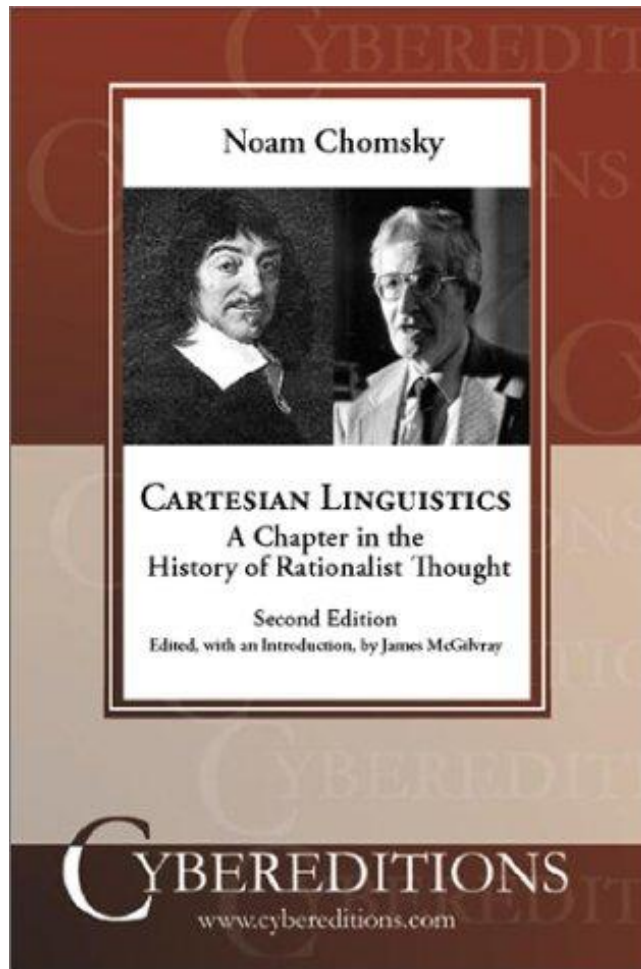


La solution computationnelle / symbolique



Mais la raison fondamentale de l'inadéquation des grammaires traditionnelles est plus technique. Sans doute avait-on bien compris que les processus linguistiques sont en un certain sens « créateurs »; mais jusqu'à une date très récente, on ne disposait tout simplement pas de l'appareil technique propre à formuler un système de processus récursifs. En fait, ce n'est que dans les trente dernières années, au cours d'études touchant les fondements des mathématiques, que s'est développé une compréhension réelle de la façon dont une langue peut, selon l'expression de Humboldt, « faire un usage infini de moyens finis ». (1965/71 : 18-19)

La solution computationnelle / symbolique



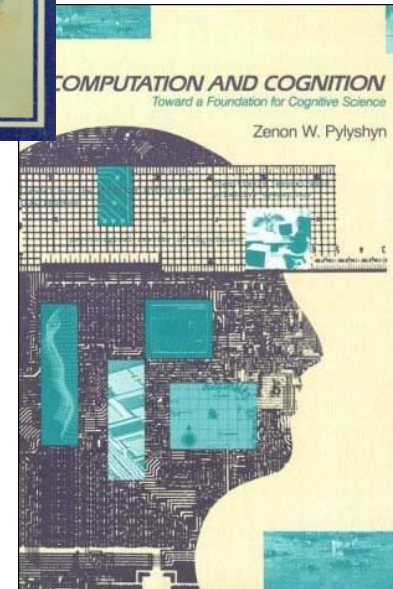
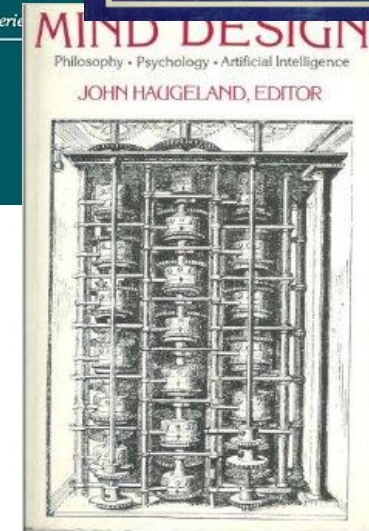
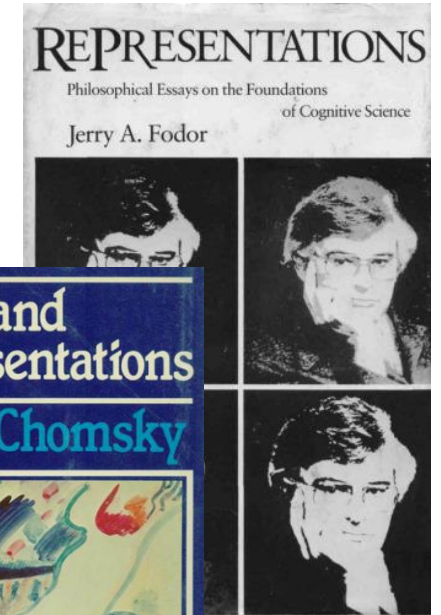
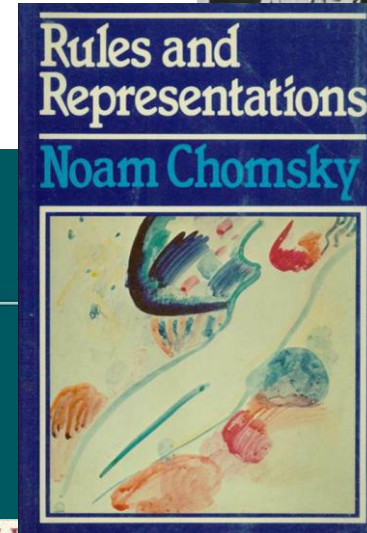
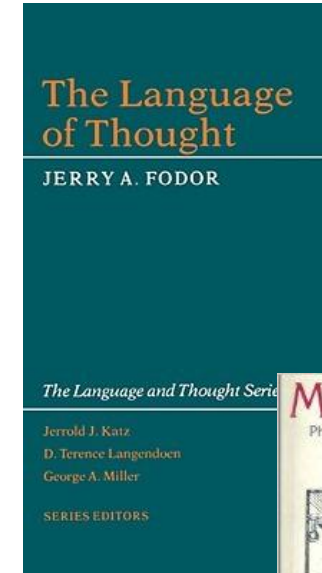
La solution computationnelle / symbolique

(Infinité de) représentation(s)



Computation
(systèmes de processus
récursifs)

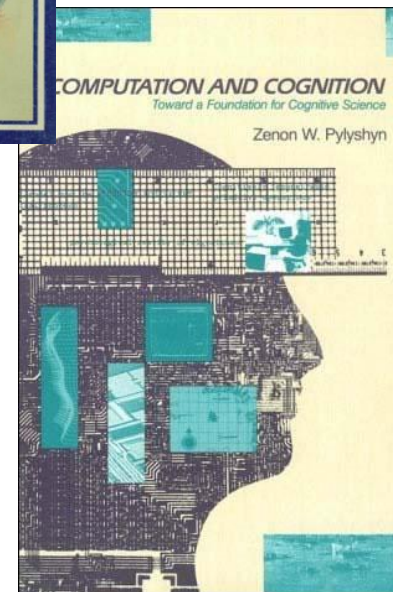
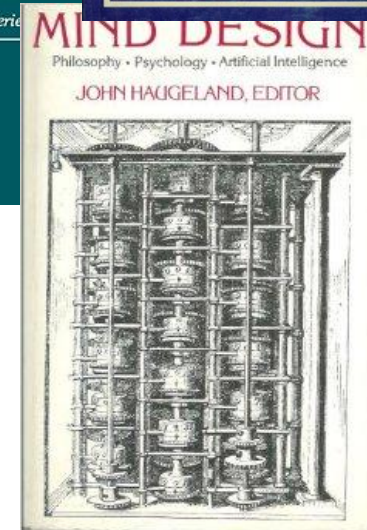
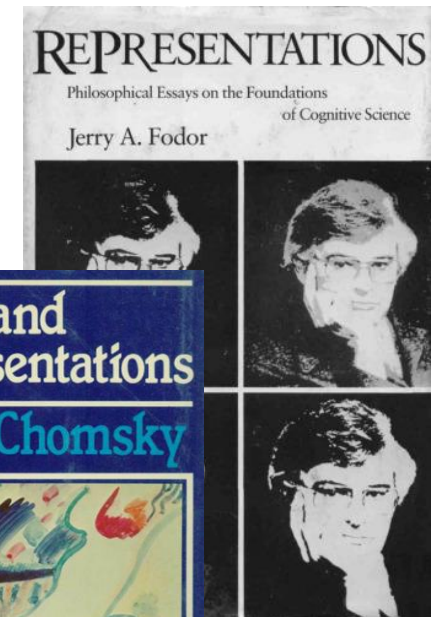
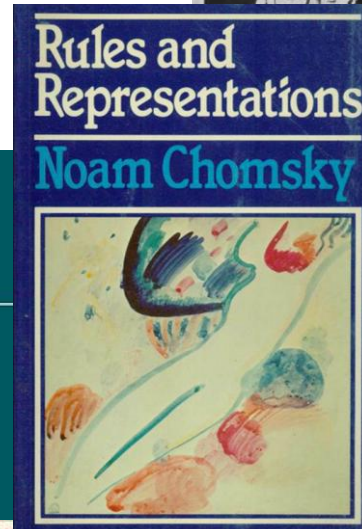
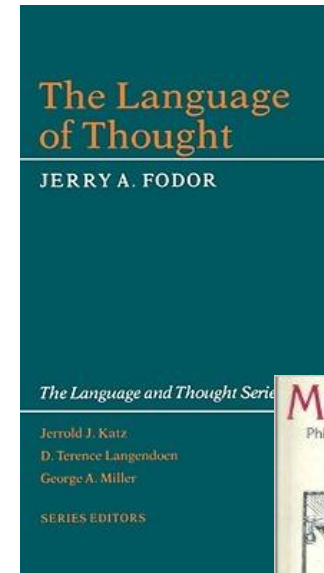
Mécanisme

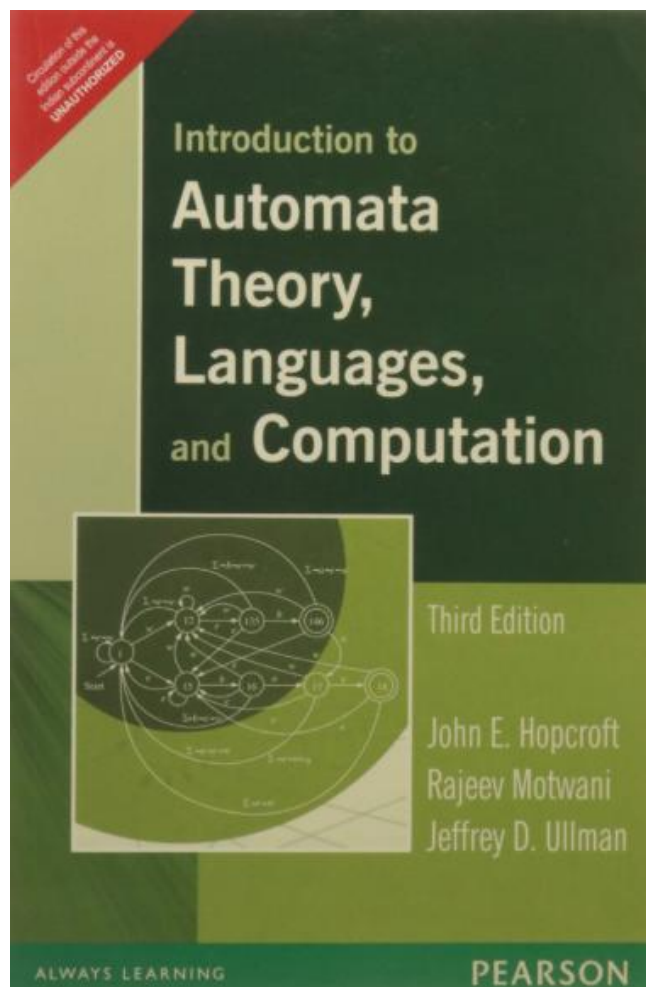


La solution computationnelle / symbolique

Computationnalisme

La thèse selon laquelle l'esprit (en entier) ou des systèmes cognitifs particuliers (ex.: le système visuel ou un système du langage) sont des **systèmes computationnels**.





COURS // MAT7050 - Indécidabilité, langages et automates

Description

Horaire - Été 2026

Horaire - Automne 2026

Horaire - Hiver 2027

■ Cycle : 2

■ Discipline : Mathématiques

■ Nombre de crédits : 3

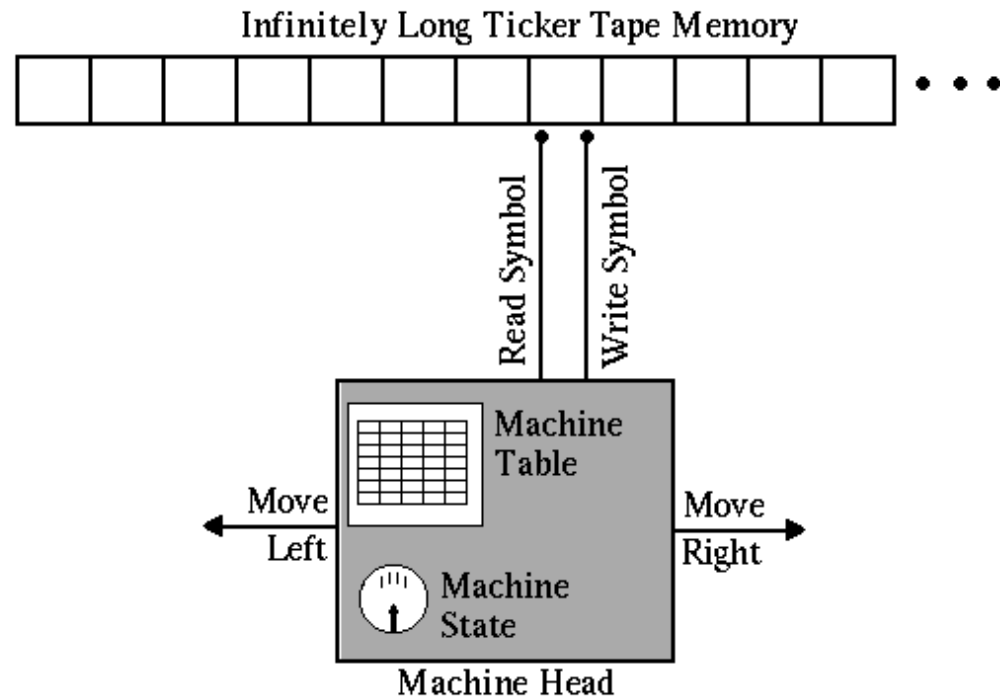
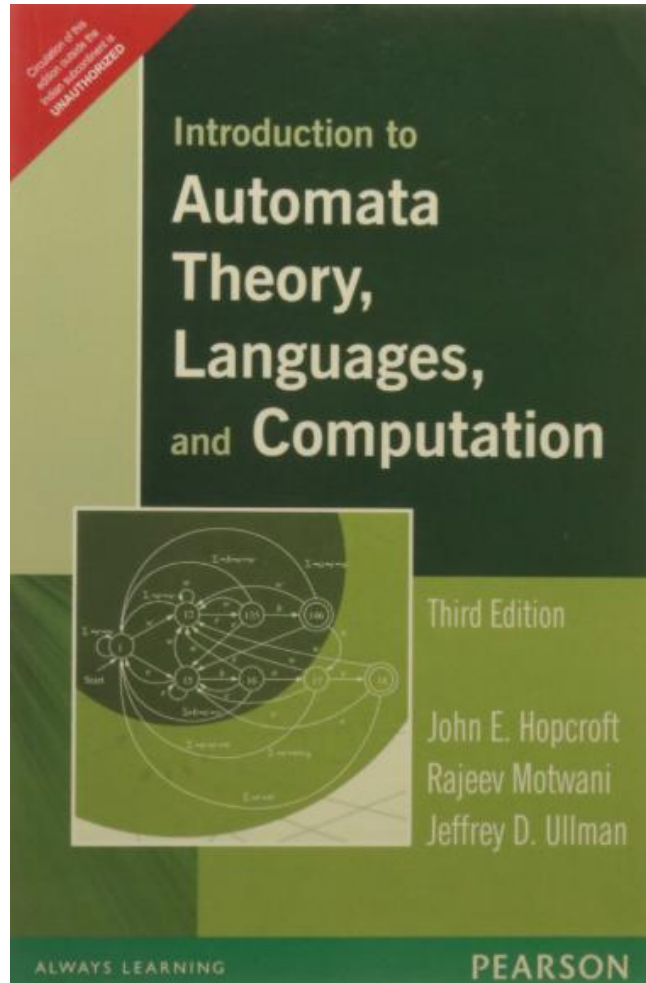
Description

Notions générales sur les fonctions récursives. Méthode générale de démonstration de l'indécidabilité des théories. Automates, machines séquentielles.

Programmes associés à ce cours

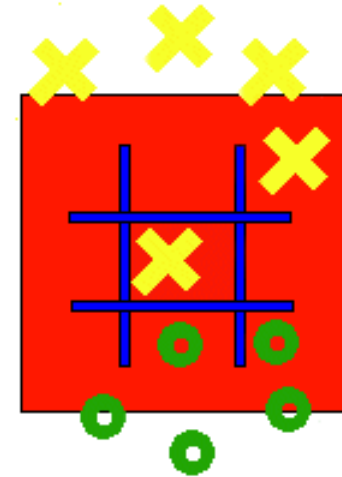
■ Doctorat en mathématiques

■ Maîtrise en mathématiques



Langages et systèmes formels

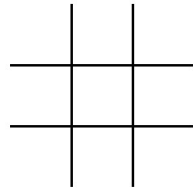
- Système formel
 - Ensemble de pièces,
 - Position de départ,
 - Ensemble de règles de transition



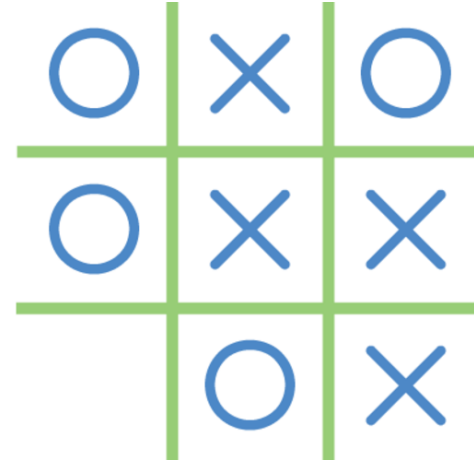
Langages et systèmes formels

- Ensemble fini de pièces: $\Sigma = \{O, O, O, O, O, X, X, X, X, X\}$,

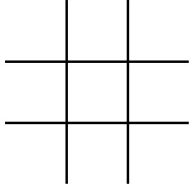
- Position de départ: $q =$

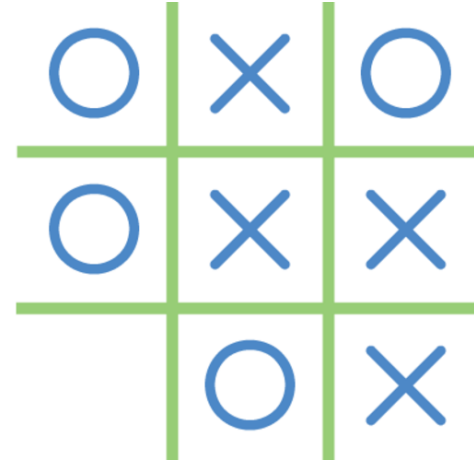


- Mais aussi: $\langle 1, 2, 3, 4, 5, 6, 7, 8, 9 \rangle$
- Ensemble de règles de transition:
 - Placer alternativement un X et un O dans une case vide, ou
 - Si une position dans le vecteur contient un chiffre, alors on peut le remplacer alternativement par un X ou un O.



Langages et systèmes formels

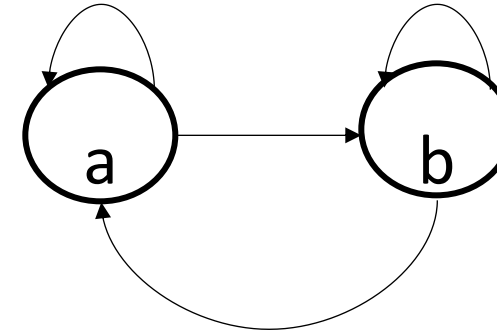
- Ensemble fini de pièces: $\Sigma = \{O, O, O, O, O, X, X, X, X, X\}$,
- Position de départ: $q =$ 
- Ensemble de règles de transition:
 - Placer alternativement un X et un O dans une case vide, ou
 - Si une position dans le vecteur contient un chiffre, alors on peut le remplacer alternativement par un X ou un O.



Indépendance par
rapport au médium

Langages et systèmes formels

- Ensemble fini de pièces: $\Sigma = \{a,b\}$,
- Position de départ: $q = a$
- Ensemble de règles de transition:
 - Placer alternativement un a ou un b à la suite de la séquence actuelle



Language	Grammar	Machine	Example
Regular language	Regular grammar —Right-linear grammar —Left-linear grammar	Deterministic or Nondeterministic finite-state acceptor	a^*
Context-free language	Context-free grammar	Nondeterministic pushdown automaton	$a^n b^n$
Context-sensitive language	Context sensitive grammar	Linear-bounded automaton	$a^n b^n c^n$
Recursively enumerable language	Unrestricted grammar	Turing machine	Any computable function

Langages et systèmes formels

- **Pièces:**

- $p, q, r, \dots$
- $), (, \supset, \equiv, \neg, \wedge, \vee$

- **Position de départ**

- $(p \vee p) \supset p$
- $q \supset (p \vee q)$
- $(p \vee q) \supset (q \vee p)$
- $(q \vee r) \supset ((p \vee q) \supset (p \vee r))$

- **Règles de construction pièces complexes**

- FR1. Une lettre seule est une pièce
- FR2. Si α est une pièce, alors $\neg\alpha$ l'est aussi
- FR3. Si α et β sont des pièces, alors $\alpha \wedge \beta$, $\alpha \vee \beta$, $\alpha \supset \beta$ et $\alpha \equiv \beta$ le sont aussi

- **Règles de transitions**

- TR1. On peut substituer uniformément une pièce α par une autre β
- TR2. On peut écrire β si on déjà écrit α et $\alpha \supset \beta$

Langages et systèmes formels

- **Pièces:**

- $p, q, r, \dots$
- $), (, \supset, \equiv, \neg, \wedge, \vee$

- **Position de départ**

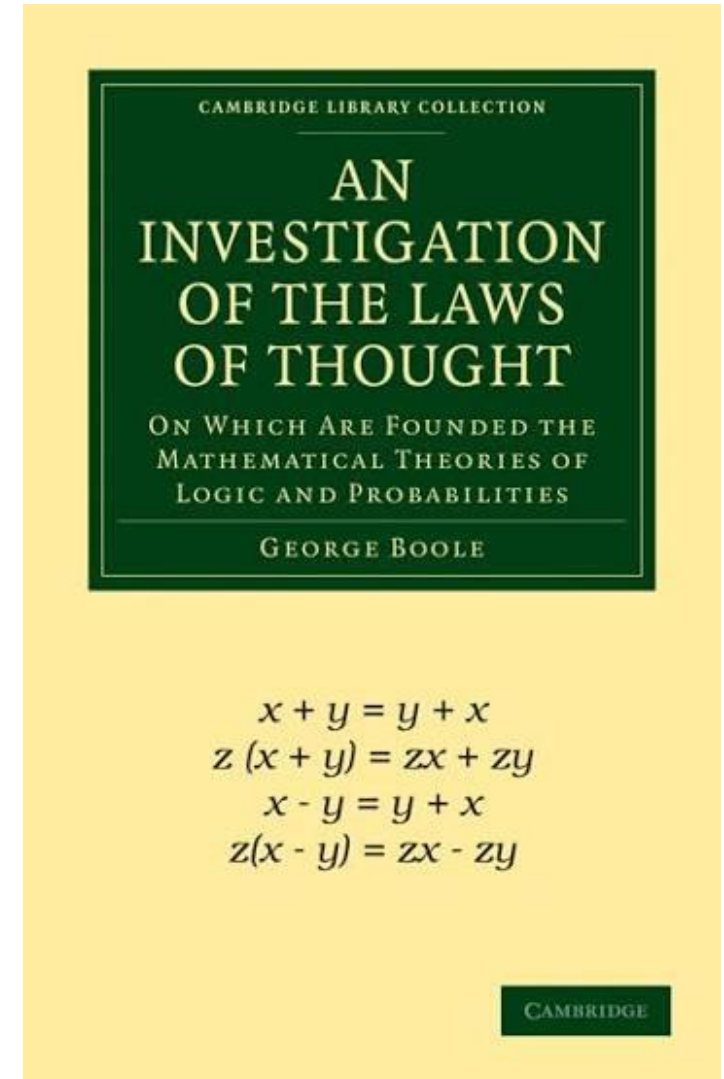
- $(p \vee p) \supset p$
- $q \supset (p \vee q)$
- $(p \vee q) \supset (q \vee p)$
- $(q \vee r) \supset ((p \vee q) \supset (p \vee r))$

- **Règles de construction pièces complexes**

- FR1. Une lettre seule est une pièce
- FR2. Si α est une pièce, alors $\neg\alpha$ l'est aussi
- FR3. Si α et β sont des pièces, alors $\alpha \wedge \beta$, $\alpha \vee \beta$, $\alpha \supset \beta$ et $\alpha \equiv \beta$ le sont aussi

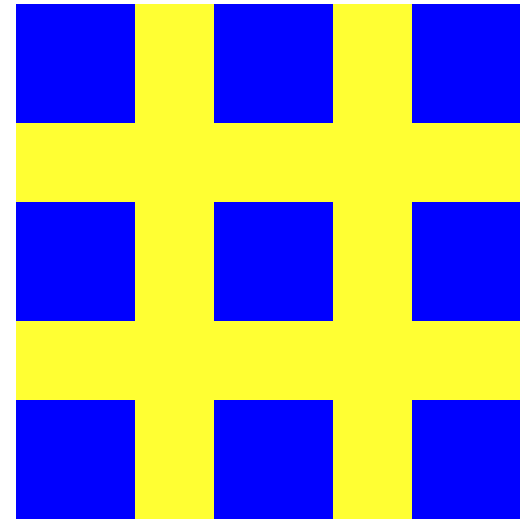
- **Règles de transitions**

- TR1. On peut substituer uniformément une pièce α par une autre β
- TR2. On peut écrire β si on déjà écrit α et $\alpha \supset \beta$



Systèmes formels automatiques (automates)

- Un SFA est un système formel capable de manipuler lui-même les pièces selon les règles définies pour le SF.
 - En théorie de la computation, on nomme les SFA des « automates ».
- Pour ce faire, il s'agit d'inclure
 - Parmi les pièces, des états du système formel
 - Parmi les règles, une fonction de transition définie sur les états.



Systèmes formels automatiques (automates)

- Exemple d'un SFA: automates à états finis (FSA)
 - Pièces:
 - Σ : Ensemble fini d'inputs
 - Q : Ensemble fini d'états du système
 - Position de départ.
 - $q \in Q$
 - Position(s) finale(s) (pas nécessaire pour une SFA mais si pour un FSA)
 - $F \subseteq Q$
 - Une règle (fonction) de transition appliquant un état à un état et un symbole
 - $\delta = f: Q \times \Sigma \rightarrow Q$

Systèmes formels automatiques (automates)

- Exemple d'un SFA: les automates à états finis
 - Si (état = fermé et input = (arrière, les deux, aucun)) → État = fermé
 - Si (état = fermé et input = avant) → État = ouvert
 - Si (état = ouvert et input = (avant, arrière, les deux)) → État = ouvert
 - Si (état = ouvert et input = aucun) → État = fermé



Systèmes formels automatiques (automates)

- Exemple d'un SFA: les automates à états finis
 - Si (état = fermé et input = (arrière, les deux, aucun)) → État = fermé
 - Si (état = fermé et input = avant) → État = ouvert
 - Si (état = ouvert et input = (avant, arrière, les deux)) → État = ouvert
 - Si (état = ouvert et input = aucun) → État = fermé



Machines de Turing (?)

Following Hopcroft and Ullman (1979, p. 148), a (one-tape) Turing machine can be formally defined as a 7-tuple $M = \langle Q, \Gamma, b, \Sigma, \delta, q_0, F \rangle$ where

- Q is a finite, non-empty set of *states*
- Γ is a finite, non-empty set of *tape alphabet symbols*
- $b \in \Gamma$ is the *blank symbol* (the only symbol allowed to occur on the tape infinitely often at any step during the computation)
- $\Sigma \subseteq \Gamma \setminus \{b\}$ is the set of *input symbols*
- $\delta : (Q \setminus F) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is a *partial function* called the *transition function*, where L is left shift, R is right shift. (A relatively uncommon variant allows "no shift", say N, as a third element of the latter set.) If δ is not defined on the current state and the current tape symbol, then the machine halts.^[21]
- $q_0 \in Q$ is the *initial state*
- $F \subseteq Q$ is the set of *final* or *accepting states*. The initial tape contents is said to be *accepted* by M if it eventually halts in a state from F .

Machines de Turing (?)



Following Hopcroft and Ullman (1979, p. 148), a (one-tape) Turing machine can be formally defined as a 7-tuple $M = \langle Q, \Gamma, b, \Sigma, \delta, q_0, F \rangle$ where

- Q is a finite, non-empty set of *states*
- Γ is a finite, non-empty set of *tape alphabet symbols*
- $b \in \Gamma$ is the *blank symbol* (the only symbol allowed to occur on the tape infinitely often at any step during the computation)
- $\Sigma \subseteq \Gamma \setminus \{b\}$ is the set of *input symbols*
- $\delta : (Q \setminus F) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is a *partial function* called the *transition function*, where L is left shift, R is right shift. (A relatively uncommon variant allows "no shift", say N, as a third element of the latter set.) If δ is not defined on the current state and the current tape symbol, then the machine halts.^[21]
- $q_0 \in Q$ is the *initial state*
- $F \subseteq Q$ is the set of *final* or *accepting states*. The initial tape contents is said to be *accepted* by M if it eventually halts in a state from F .

Un système formel: pièces

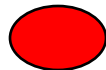
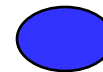
Pièces:



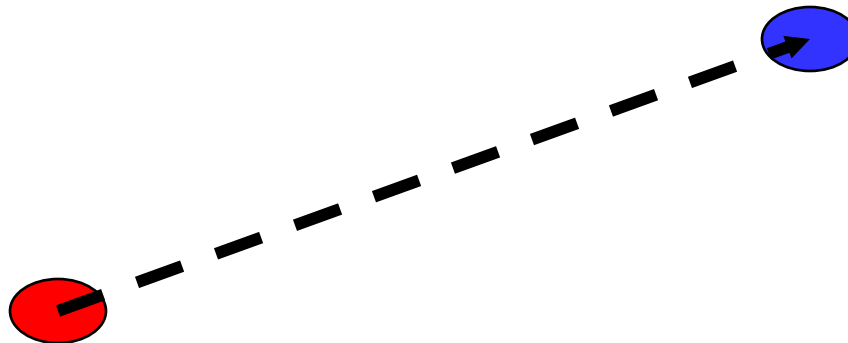
Un système formel: position de départ



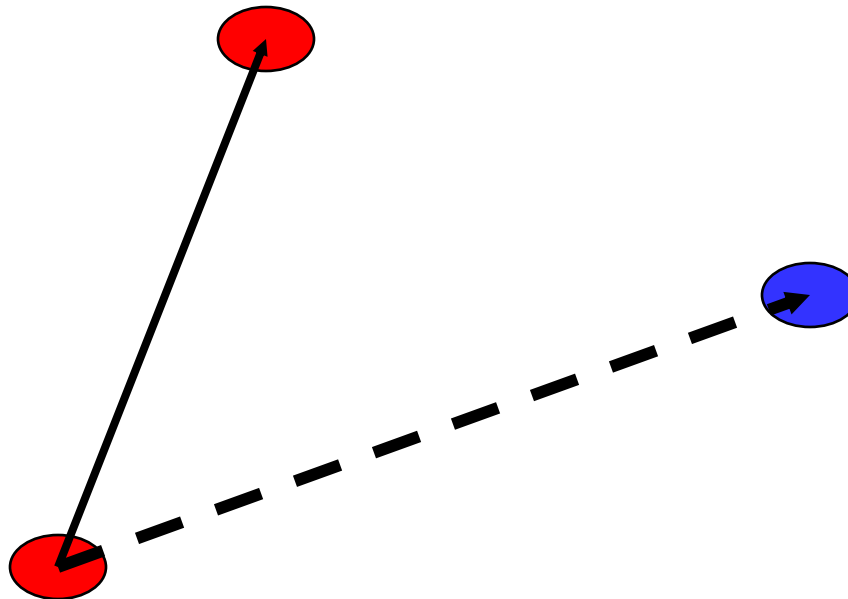
Un système formel: transitions



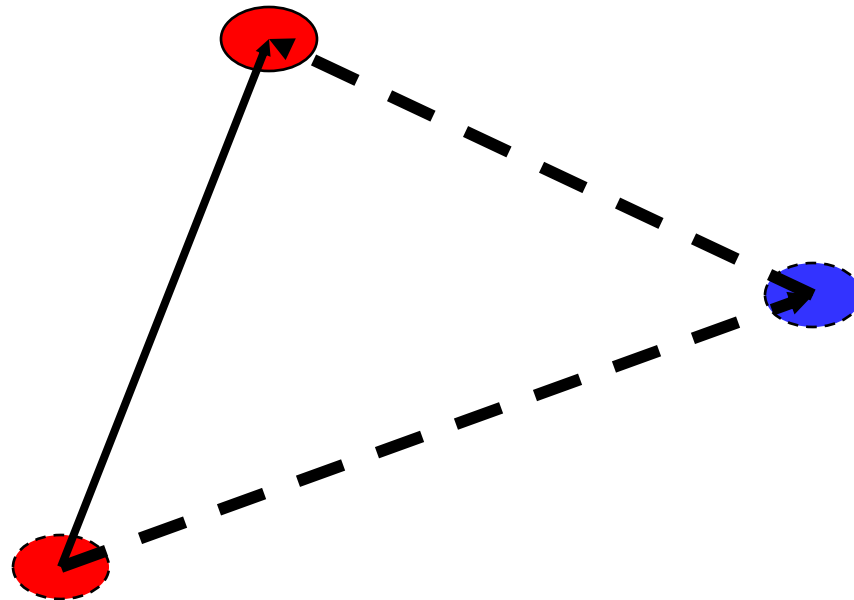
Un système formel: transitions



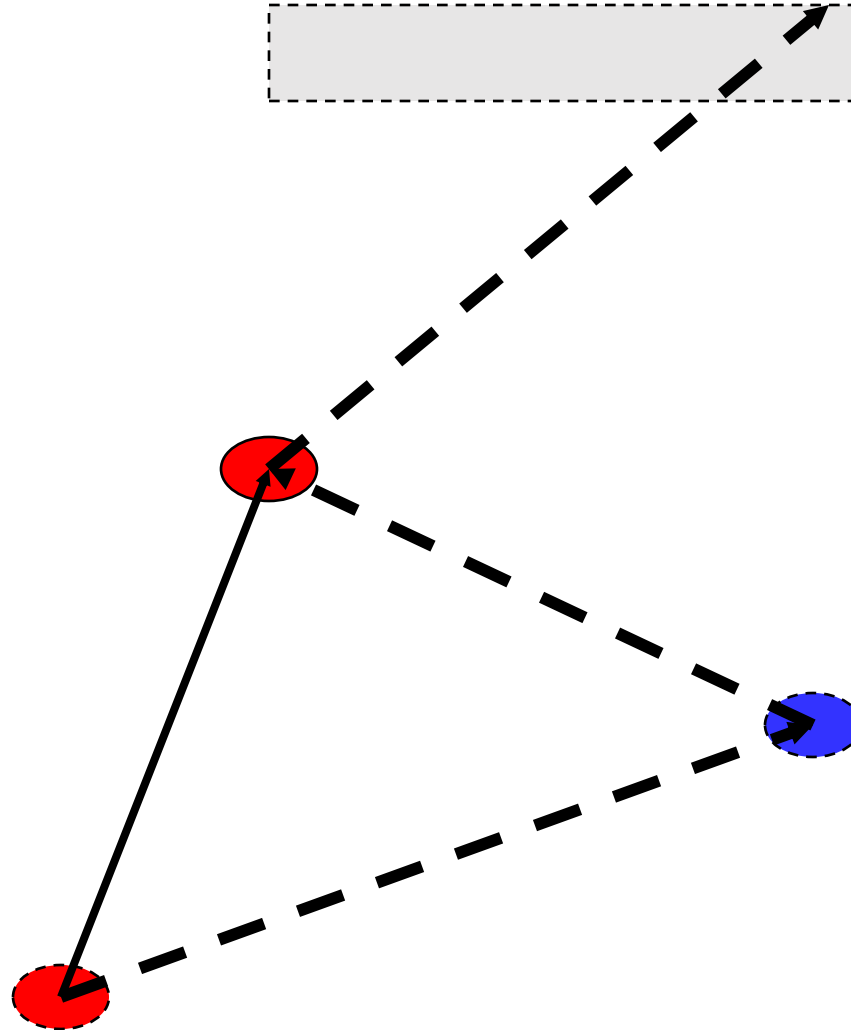
Un système formel: transitions



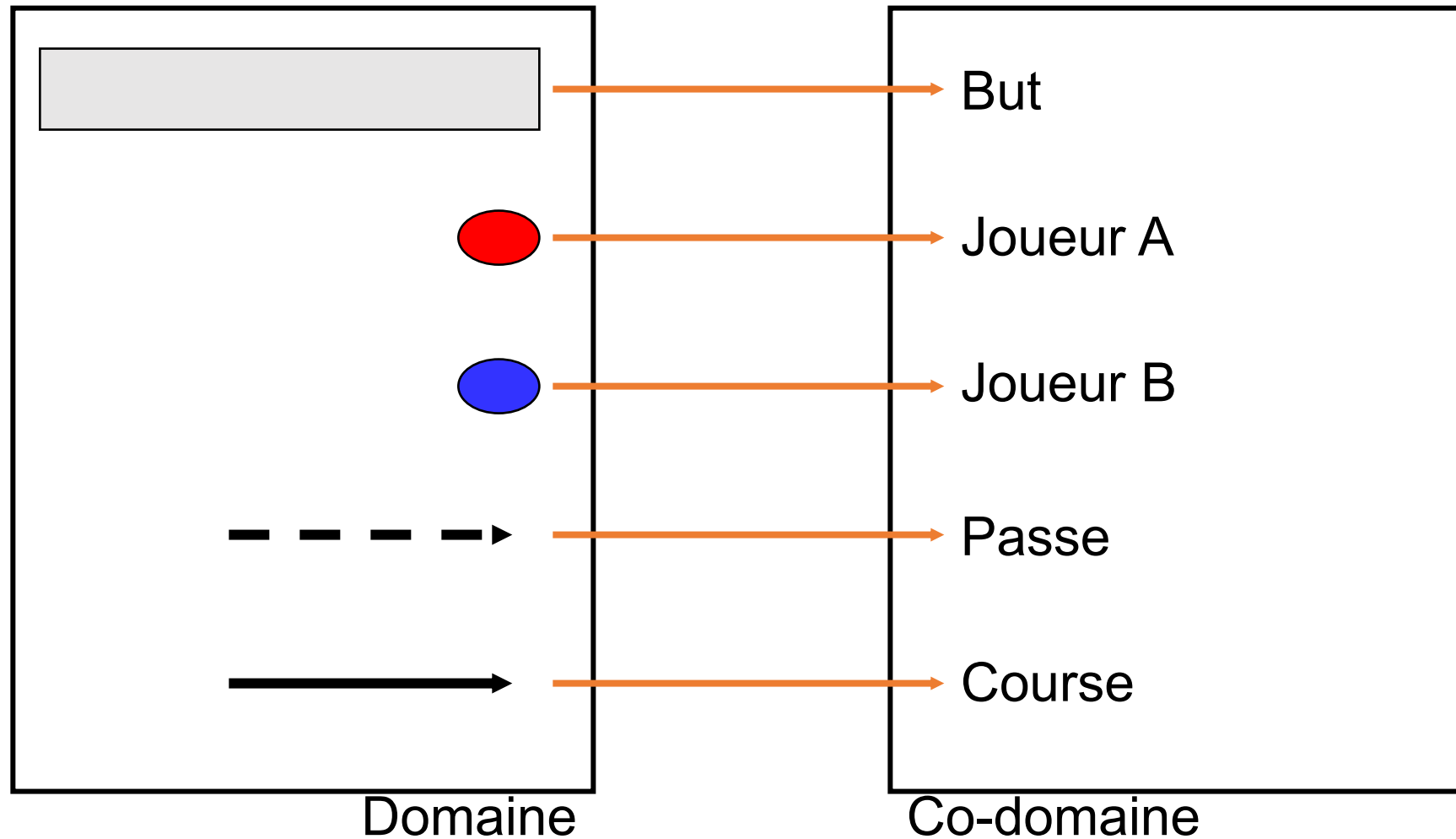
Un système formel: transitions



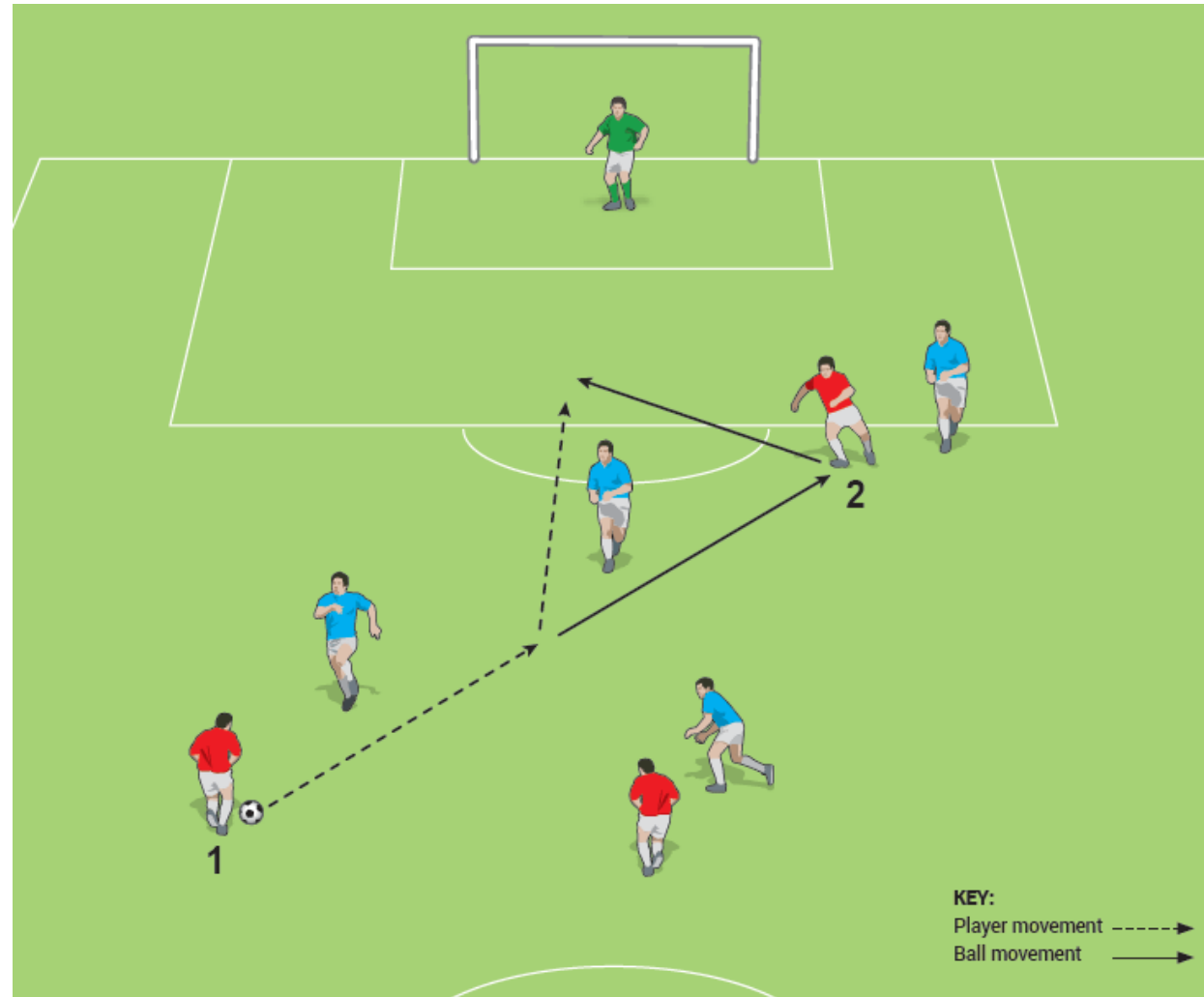
Un système formel: transitions



Fonction d'interprétation (des pièces du système formel)

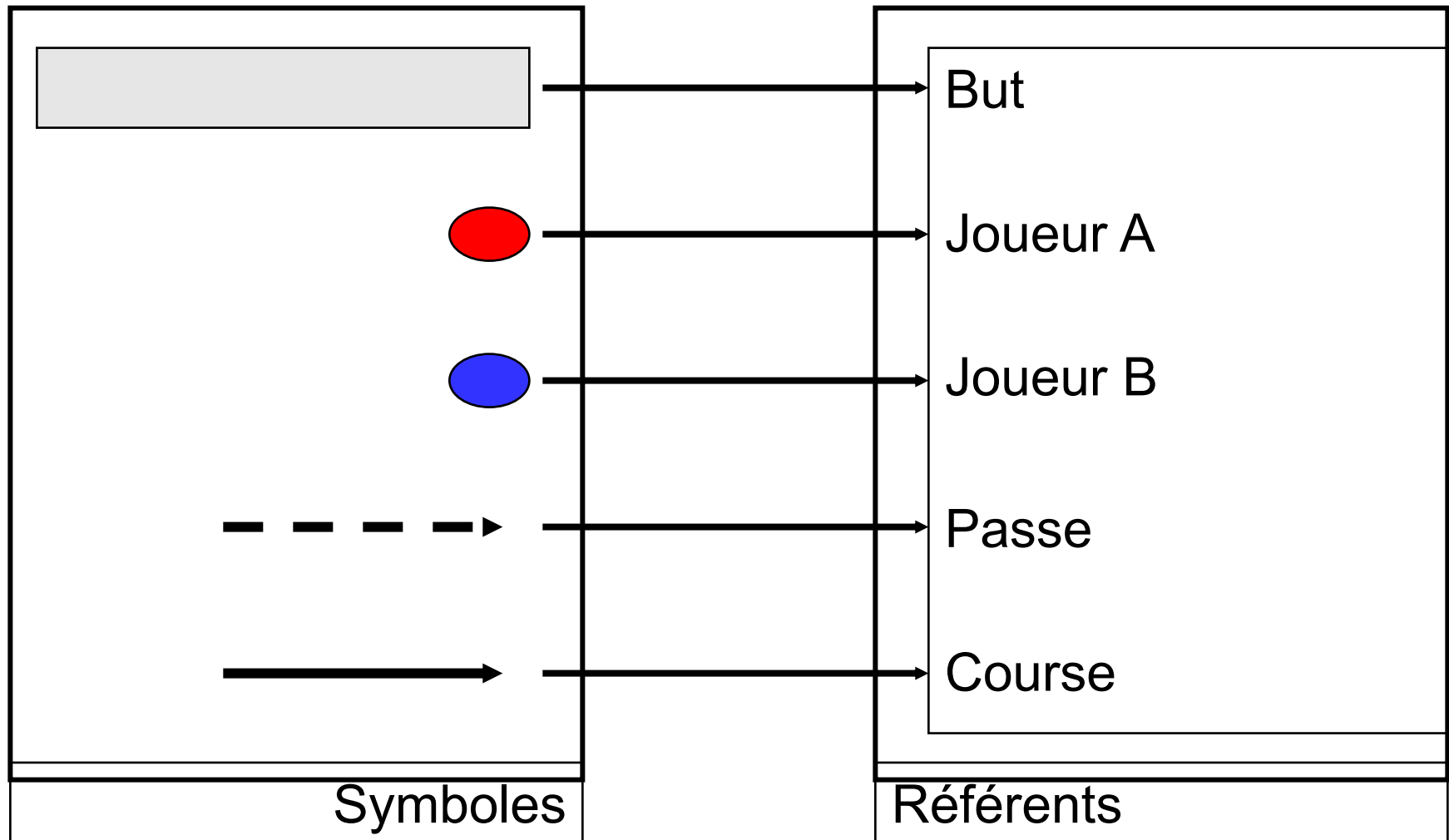


Fonction d'interprétation



A one-two is the classic pass, run and receive back move that bypasses the defender

Fonction d'interprétation (des pièces du système formel)



Système PM: système formel ou système formel interprété?

- **Éléments (Pièces):**

- $p, q, r, \dots$
- $), (, \supset, \equiv, \neg, \wedge, \vee$

- **Axiomes (Position de départ)**

- $(p \vee p) \supset p$
- $q \supset (p \vee q)$
- $(p \vee q) \supset (q \vee p)$
- $(q \vee r) \supset ((p \vee q) \supset (p \vee r))$

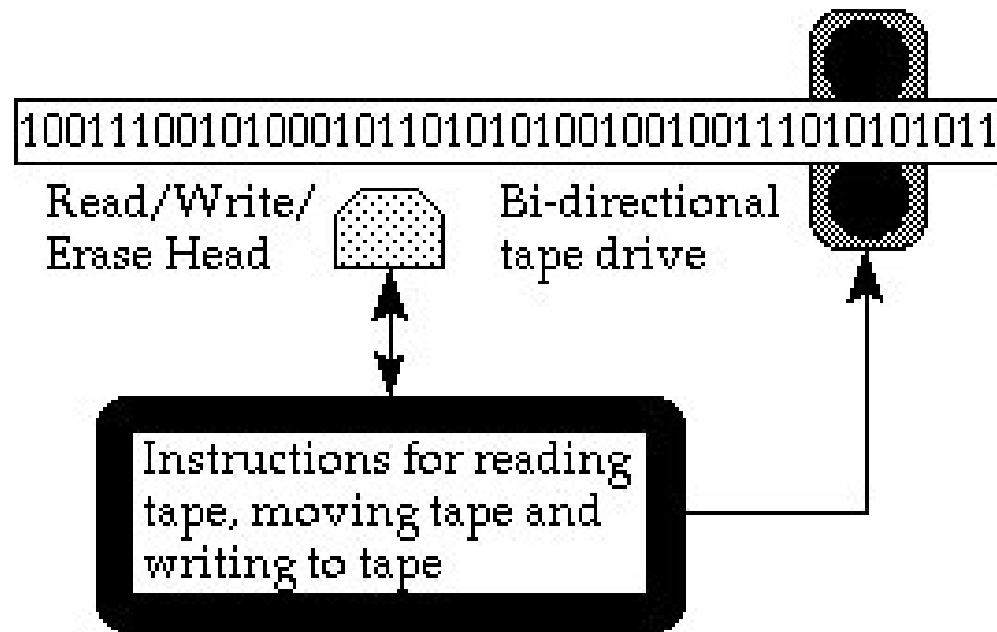
- **Règles de construction d'expressions bien formées (ebf) (Pièces complexes)**

- FR1. Une lettre seule est une ebf (pièce)
- FR2. Si α est une ebf (pièce), alors $\neg\alpha$ l'est aussi
- FR3. Si α et β sont des ebfs (pièces), alors $\alpha \wedge \beta$, $\alpha \vee \beta$, $\alpha \supset \beta$ et $\alpha \equiv \beta$ le sont aussi

- **Règles d'inférences (Règles de transition)**

- TR1. La règle de substitution uniforme: Le résultat d'une substitution uniforme d'une variable dans une thèse par une ebf est aussi une thèse
- TR2. La règle du *Modus Ponens*: Si α et $\alpha \supset \beta$ sont des thèses, alors β l'est aussi.

La machine de Turing « mécanique »



- Une MT est composée de deux éléments:
 - Une tête
 - Lecture, écriture, déplacement, suivre les instructions d'une table d'instructions
 - Un ruban
 - Espace pour emmagasiner des symboles

La machine de Turing « mécanique »

- La réponse de Turing
 - Analyse du comportement des « human computers »
 - Calculer c'est:
 - Lire, Écrire, Effacer des symboles
 - Suivre des instructions indiquant où lire le prochain symbole et quoi écrire.
 - (1936, *On Computable numbers, with an application to the entscheidungsproblem*)



La machine de Turing « mécanique »

- La réponse de Turing
 - Analyse du comportement des « human computers »
 - Calculer c'est:
 - Lire, Écrire, Effacer des symboles
 - Suivre des instructions indiquant où lire le prochain symbole et quoi écrire.
 - (1936, *On Computable numbers, with an application to the entscheidungsproblem*)



Les démonstrations de Turing (1936)

230

A. M. TURING

[Nov. 12,

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO
THE ENTSCHEIDUNGSPROBLEM

By A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]

The “computable” numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means. Although the subject of this paper is ostensibly the computable *numbers*, it is almost equally easy to define and investigate computable functions of an integral variable or a real or computable variable, computable predicates, and so forth. The fundamental problems involved are, however, the same in each case, and I have chosen the computable numbers for explicit treatment as involving the least cumbersome technique. I hope shortly to give an account of the relations of the computable numbers, functions, and so forth to one another. This will include a development of the theory of functions of a real variable expressed in terms of computable numbers. According to my definition, a number is computable if its decimal can be written down by a machine.

In §§ 9, 10 I give some arguments with the intention of showing that the computable numbers include all numbers which could naturally be regarded as computable. In particular, I show that certain large classes of numbers are computable. They include, for instance, the real parts of all algebraic numbers, the real parts of the zeros of the Bessel functions, the numbers π , e , etc. The computable numbers do not, however, include all definable numbers, and an example is given of a definable number which is not computable.

Although the class of computable numbers is so great, and in many ways similar to the class of real numbers, it is nevertheless enumerable. In § 8 I examine certain arguments which would seem to prove the contrary. By the correct application of one of these arguments, conclusions are reached which are superficially similar to those of Gödel†. These results

† Gödel, “Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme, I”, *Monatshefte Math. Phys.*, 38 (1931), 173–198.

- Pour toute fonction calculable, il existe une machine de Turing capable de produire le résultat de la fonction étant donné ses arguments.
- Il existe une machine, la machine de Turing, dite « universelle » (MU), capable de simuler n’importe quelle autre machine de Turing. Ce faisant, cette machine est donc capable de produire le résultat de toute fonction étant donné ses arguments.

L'explication mécaniste (pré-révolution)

1975 ACM Turing Award Lecture

The 1975 ACM Turing Award was presented jointly to Allen Newell and Herbert A. Simon at the ACM Annual Conference in Minneapolis, October 20. In introducing the recipients, Bernard A. Galler, Chairman of the Turing Award Committee, read the following citation:

"It is a privilege to be able to present the ACM Turing Award to two friends of long standing, Professors Allen Newell and Herbert A. Simon, both of Carnegie-Mellon University.

"In joint scientific efforts extending over twenty years, initially in collaboration with J.C. Shaw at the RAND Corporation, and subsequently with numerous faculty and student colleagues at Carnegie-Mellon University, they have made basic contributions to artificial intelligence, the psychology of human cognition, and list processing.

"In artificial intelligence, they contributed to the establishment of the field as an area of scientific endeavor, to the development of heuristic programming generally, and of heuristic search, means-ends analysis, and methods of induction, in particular; providing

demonstrations of the sufficiency of these mechanisms to solve interesting problems.

"In psychology, they were principal instigators of the idea that human cognition can be described in terms of a symbol system, and they have developed detailed theories for human problem solving, verbal learning and inductive behavior in a number of task domains, using computer programs embodying these theories to simulate the human behavior.

"They were apparently the inventors of list processing, and have been major contributors to both software technology and the development of the concept of the computer as a system of manipulating symbolic structures and not just as a processor of numerical data.

"It is an honor for Professors Newell and Simon to be given this award, but it is also an honor for ACM to be able to add their names to our list of recipients, since by their presence, they will add to the prestige and importance of the ACM Turing Award."

Computer Science as Empirical Inquiry: Symbols and Search

Allen Newell and Herbert A. Simon



Key Words and Phrases: symbols, search, science, computer science, empirical, Turing, artificial intelligence, intelligence, list processing, cognition, heuristics, problem solving.

CR Categories: 1.0, 2.1, 3.3, 3.6, 5.7.

Copyright © 1976, Association for Computing Machinery, Inc. General permission to republish, but not for profit, all or part of this material is granted provided that ACM's copyright notice is given and that reference is made to the publication,

Computer science is the study of the phenomena surrounding computers. The founders of this society understood this very well when they called themselves the Association for Computing Machinery. The machine—not just the hardware, but the programmed, living machine—is the organism we study.

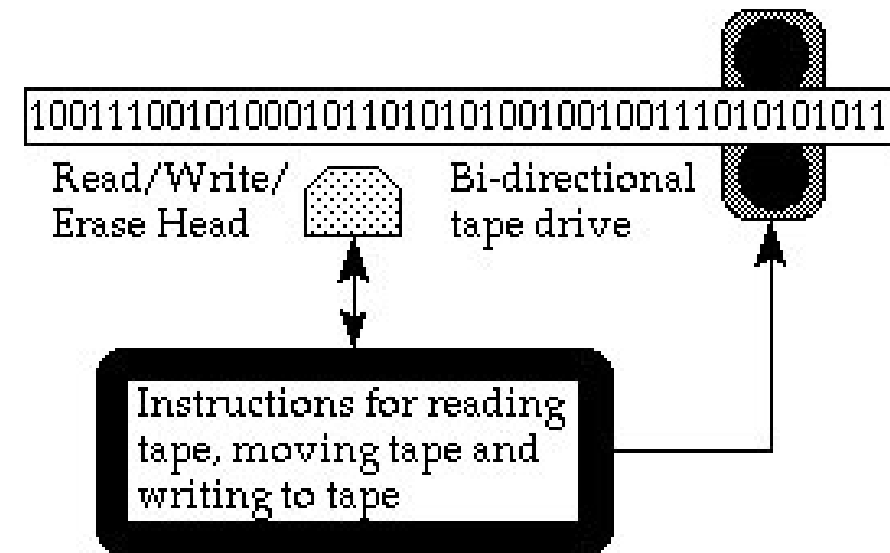
This is the tenth Turing Lecture. The nine persons who preceded us on this platform have presented nine different views of computer science. For our organism, the machine, can be studied at many levels and from many sides. We are deeply honored to appear here today and to present yet another view, the one that has permeated the scientific work for which we have been

to its date of issue, and to the fact that reprinting privileges were granted by permission of the Association for Computing Machinery.

The authors' research over the years has been supported in part by the Advanced Research Projects Agency of the Department of Defense (monitored by the Air Force Office of Scientific Research) and in part by the National Institutes of Mental Health.

Authors' address: Carnegie-Mellon University, Pittsburgh.

The Physical Symbol System Hypothesis. A physical symbol system has the necessary and sufficient means for general intelligent action.



Hypothèse du système symbolique physique

The Physical Symbol System Hypothesis. A physical symbol system has the necessary and sufficient means for general intelligent action.

- Système symbolique physique (SSP)
 - Un système symbolique physique est un système computationnel implémenté dans un mécanisme physique.
- Hypothèse du SSP (Newell et Simon 1976)
 - Un SSP a les moyens nécessaires et suffisants pour un comportement général intelligent
 - Moyens pour un comportement général intelligent → SSP
 - SSP → moyens pour un comportement général intelligent

1975 ACM Turing Award Lecture

The 1975 ACM Turing Award was presented jointly to Allen Newell and Herbert A. Simon at the ACM Annual Conference in Minneapolis, October 20. In introducing the recipients, Bernard A. Galler, Chairman of the Turing Award Committee, read the following citation:

"It is a privilege to be able to present the ACM Turing Award to two friends of long standing, Professors Allen Newell and Herbert A. Simon, both of Carnegie-Mellon University.

"In joint scientific efforts extending over twenty years, initially in collaboration with J.C. Shaw at the RAND Corporation, and subsequently with numerous faculty and student colleagues at Carnegie-Mellon University, they have made basic contributions to artificial intelligence, the psychology of human cognition, and list processing.

"In artificial intelligence, they contributed to the establishment of the field as an area of scientific endeavor, to the development of heuristic programming generally, and of heuristic search, means-ends analysis, and methods of induction, in particular; providing

demonstrations of the sufficiency of these mechanisms to solve interesting problems.

"In psychology, they were principal instigators of the idea that human cognition can be described in terms of a symbol system, and they have developed detailed theories for human problem solving, verbal learning and inductive behavior in a number of task domains, using computer programs embodying these theories to simulate the human behavior.

"They were apparently the inventors of list processing, and have been major contributors to both software technology and the development of the concept of the computer as a system of manipulating symbolic structures and not just as a processor of numerical data.

"It is an honor for Professors Newell and Simon to be given this award, but it is also an honor for ACM to be able to add their names to our list of recipients, since by their presence, they will add to the prestige and importance of the ACM Turing Award."

Computer Science as Empirical Inquiry: Symbols and Search

Allen Newell and Herbert A. Simon



Computer science is the study of the phenomena surrounding computers. The founders of this society understood this very well when they called themselves the Association for Computing Machinery. The machine—not just the hardware, but the programmed, living machine—is the organism we study.

This is the tenth Turing Lecture. The nine persons who preceded us on this platform have presented nine different views of computer science. For our organism, the machine, can be studied at many levels and from many sides. We are deeply honored to appear here today and to present yet another view, the one that has permeated the scientific work for which we have been

Key Words and Phrases: symbols, search, science, computer science, empirical, Turing, artificial intelligence, intelligence, list processing, cognition, heuristics, problem solving.

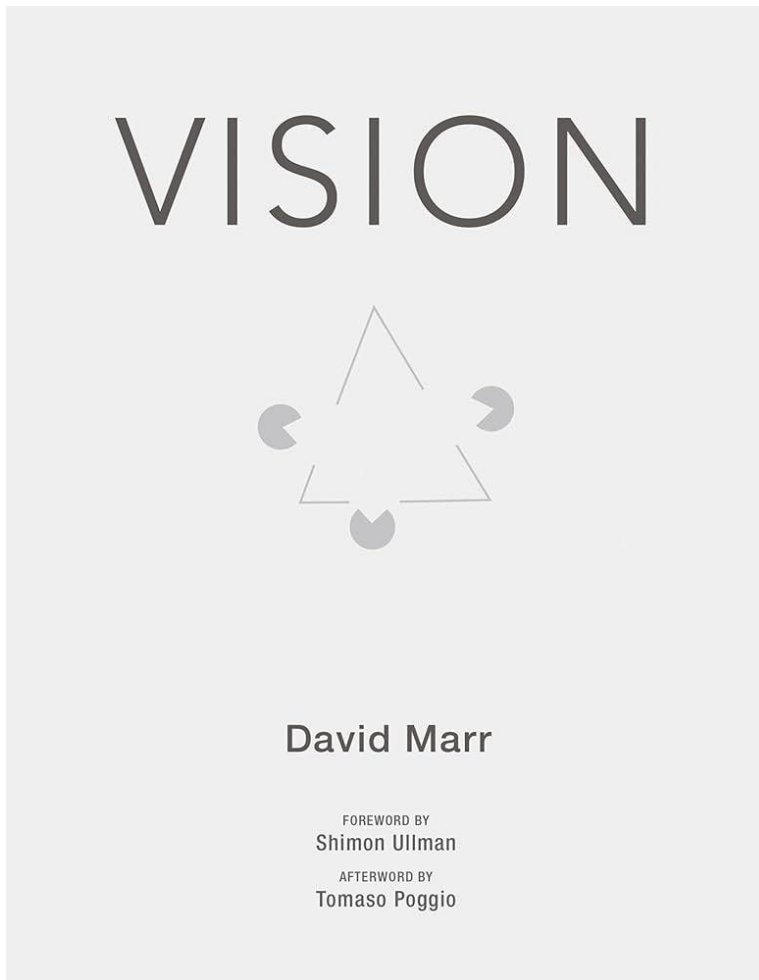
CR Categories: 1.0, 2.1, 3.3, 3.6, 5.7.
Copyright © 1976, Association for Computing Machinery, Inc. General permission to republish, but not for profit, all or part of this material is granted provided that ACM's copyright notice is given and that reference is made to the publication,

to its date of issue, and to the fact that reprinting privileges were granted by permission of the Association for Computing Machinery.

The authors' research over the years has been supported in part by the Advanced Research Projects Agency of the Department of Defense (monitored by the Air Force Office of Scientific Research) and in part by the National Institutes of Mental Health.

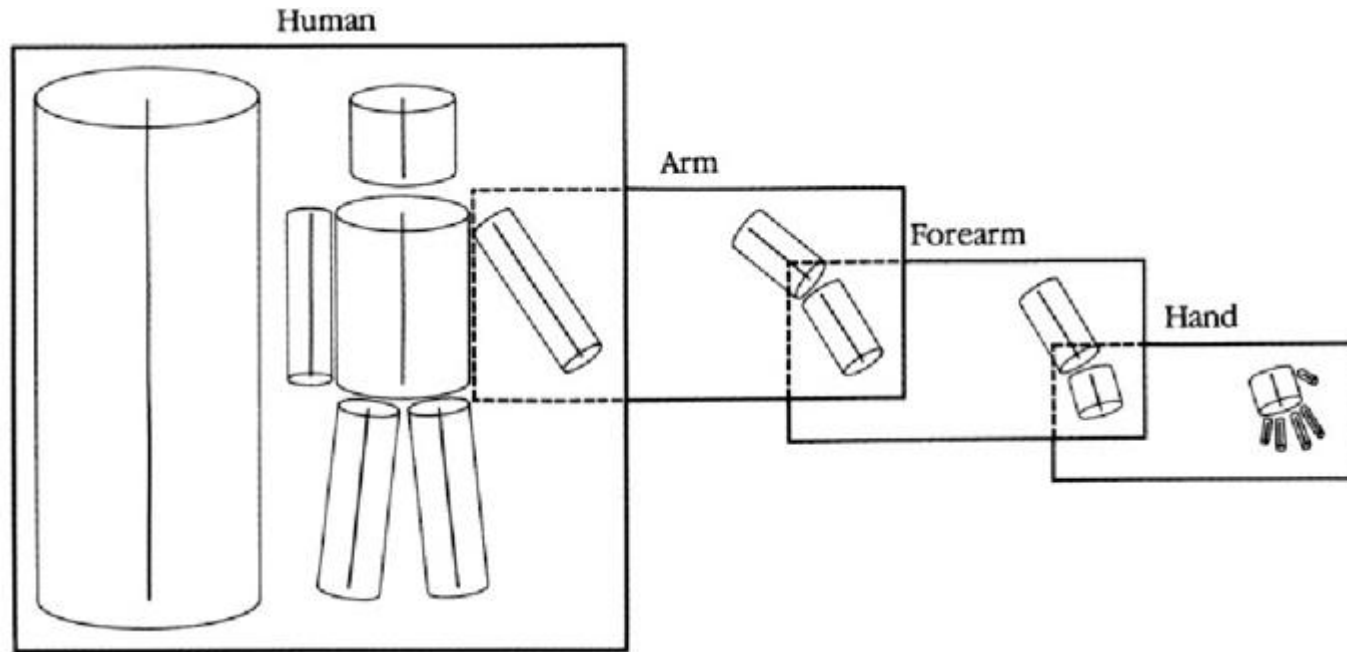
Authors' address: Carnegie-Mellon University, Pittsburgh.

L'analyse fonctionnelle: les « niveaux » de Marr



Computational theory	Representation and algorithm	Hardware implementation
What is the goal of the computation, why is it appropriate, and what is the logic of the strategy by which it can be carried out?	How can this computational theory be implemented? In particular, what is the representation for the input and output, and what is the algorithm for the transformation?	How can the representation and algorithm be realized physically?

Explications computationnelles



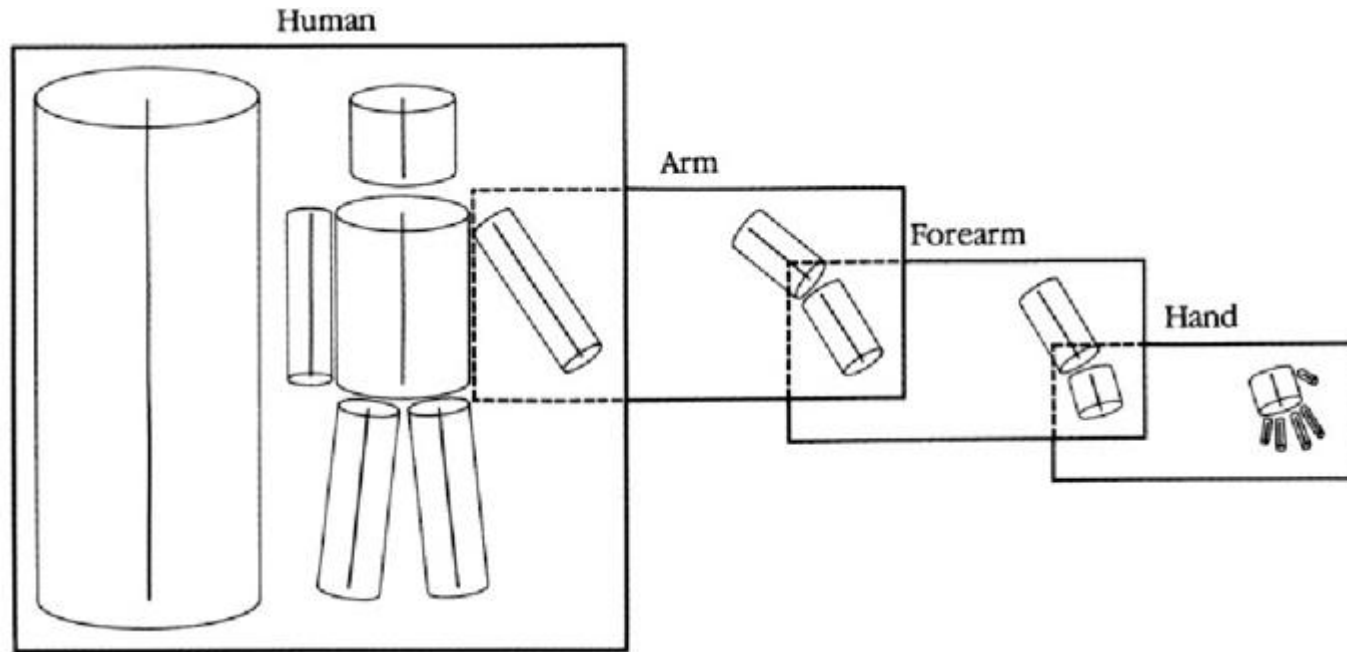
Centré objet

Représentation
par modèle 3D

Hiérarchie de
modèles 3D
organisés en
primitives de
surfaces et
volumes

Explanandum:
Capacité (décrite en termes d'une
fonction calculable):
La capacité de S à ψ -er

Explications computationnelles



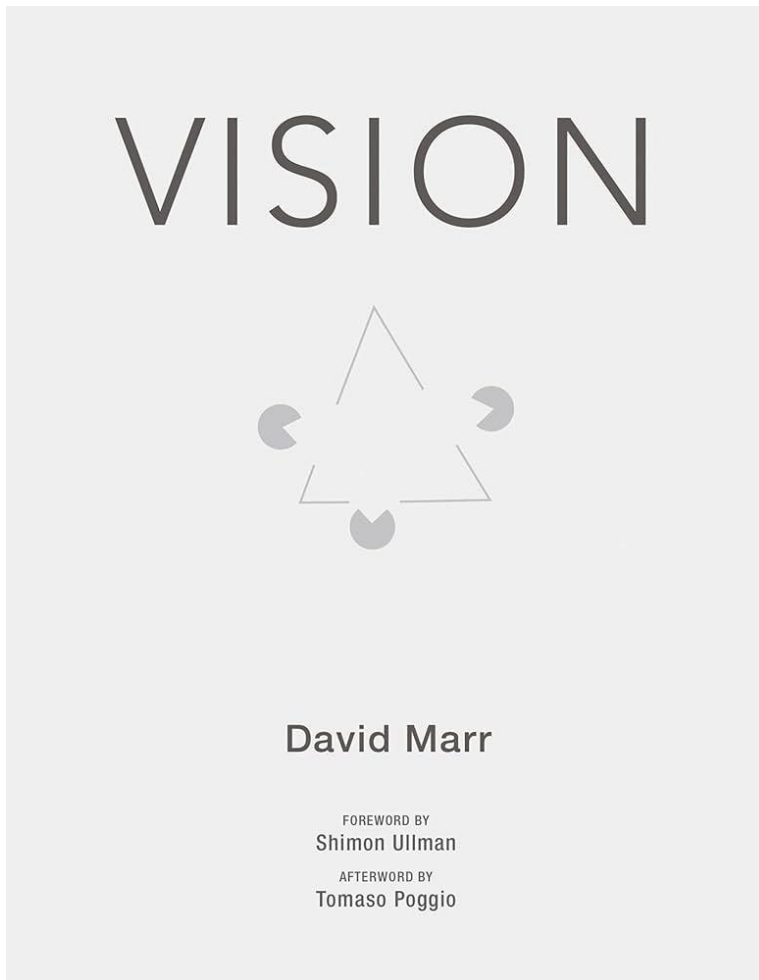
Centré objet

Représentation
par modèle 3D

Hiérarchie de
modèles 3D
organisés en
primitives de
surfaces et
volumes

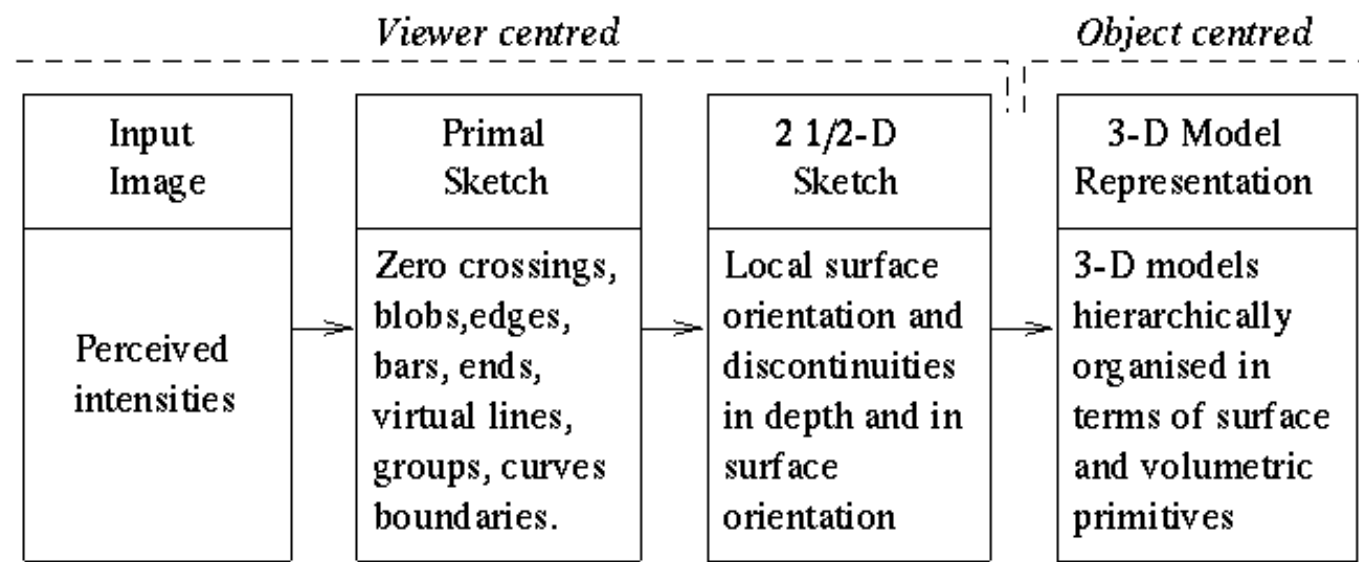
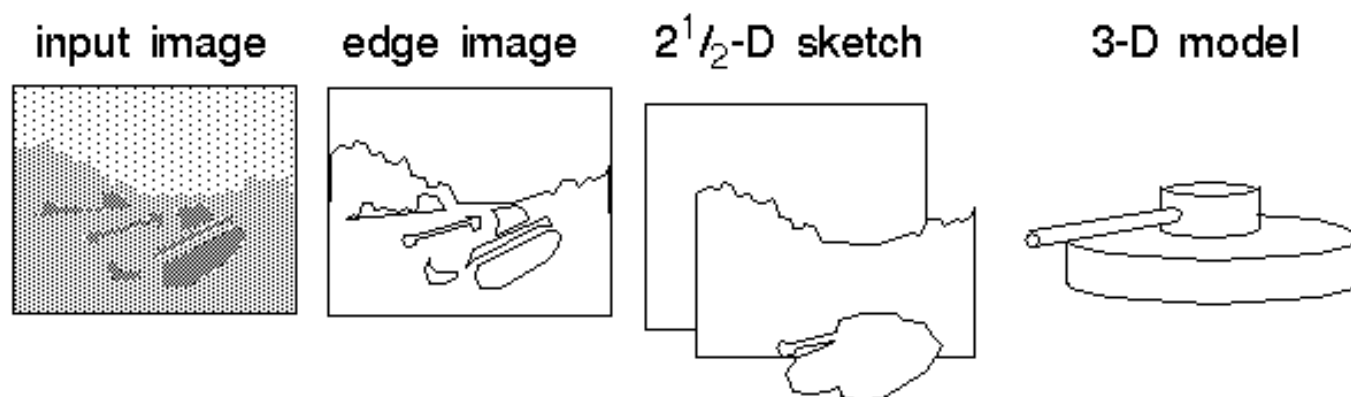
Explanandum:
Capacité (décrite en termes d'une
fonction calculable):
La capacité de S à ψ -er

L'analyse fonctionnelle: les « niveaux » de Marr



Computational theory	Representation and algorithm	Hardware implementation
What is the goal of the computation, why is it appropriate, and what is the logic of the strategy by which it can be carried out?	How can this computational theory be implemented? In particular, what is the representation for the input and output, and what is the algorithm for the transformation?	How can the representation and algorithm be realized physically?

Explications computationnelles

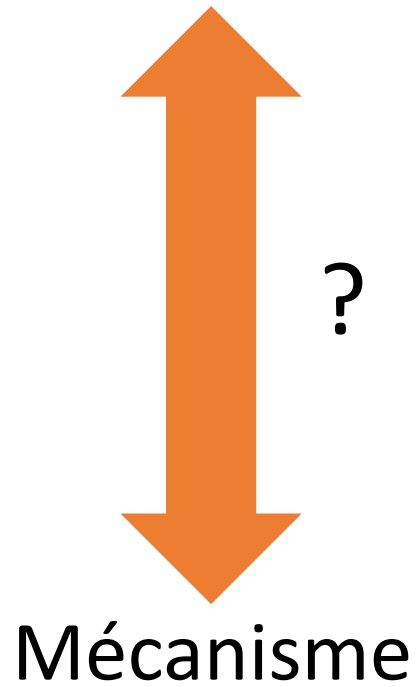


Aujourd'hui

- L'explication de la cognition (un vieux problème)
- La solution computationnelle / symbolique
- **La solution connexionniste**

L'explication de la cognition (un vieux problème)

(Infinité de) représentation(s)



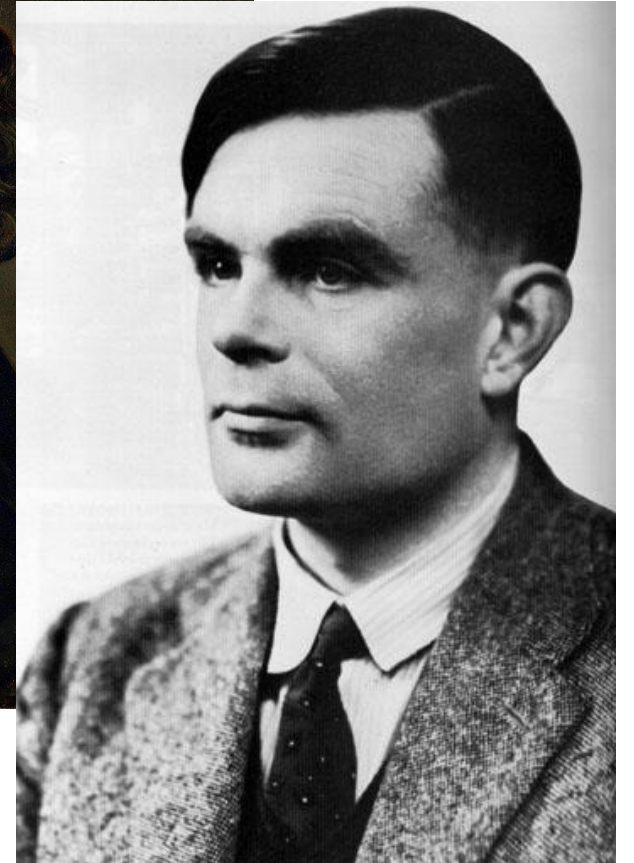
L'explication de la cognition (un vieux problème)

(Infinité de) représentation(s)



?

Mécanisme



L'explication de la cognition (un vieux problème)

(Infinité de) représentation(s)
neuronales

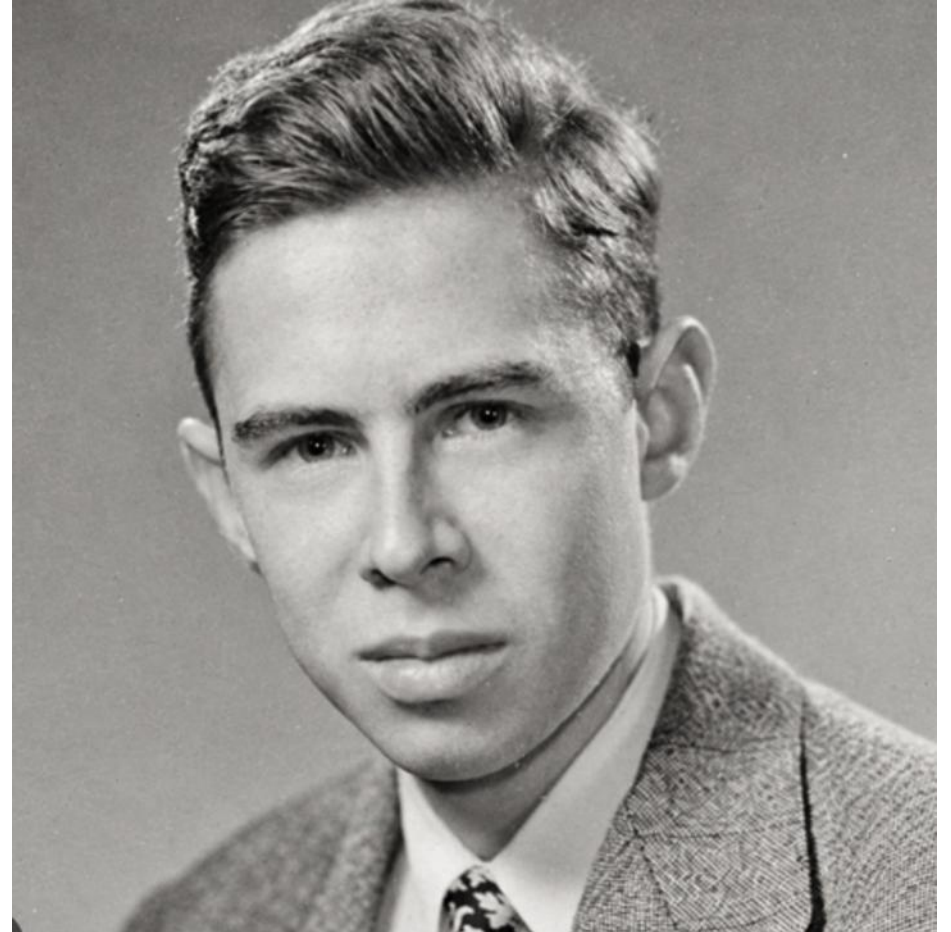
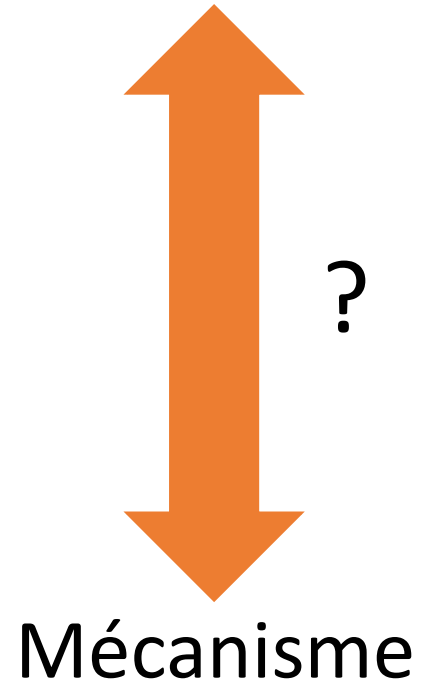


?

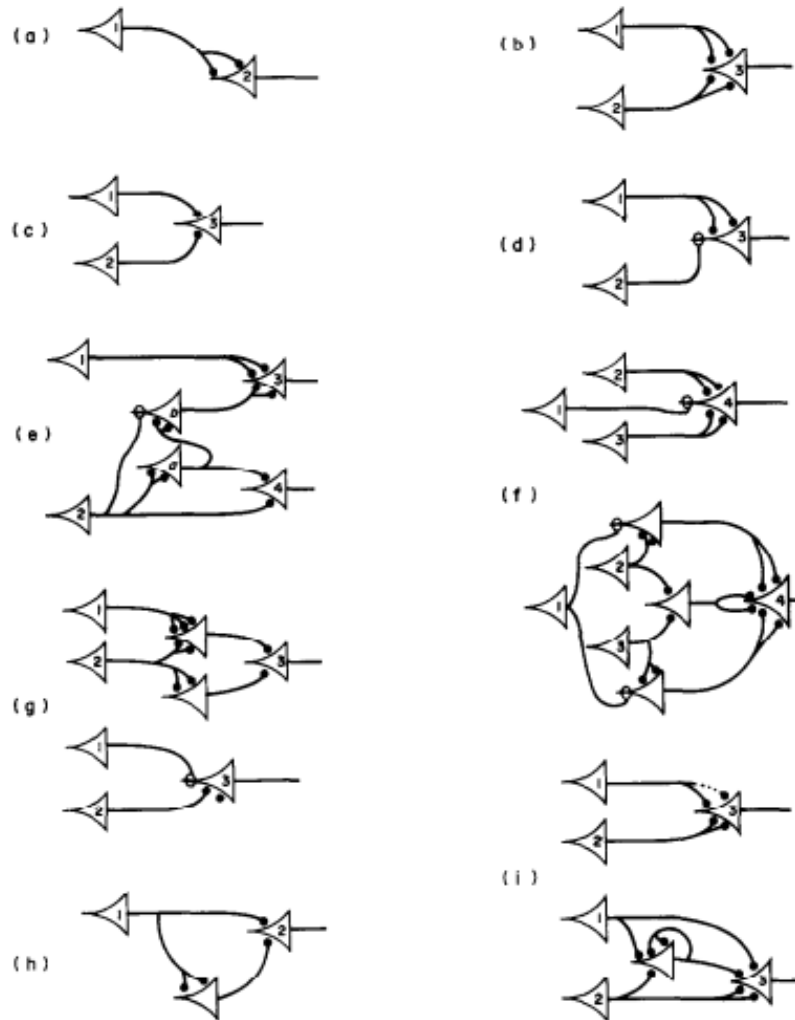
Mécanisme

La solution connexionniste

(Infinité de) représentation(s)
neuronales



Représentations neuronales



A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY*

■ WARREN S. MCCULLOCH AND WALTER PITTS

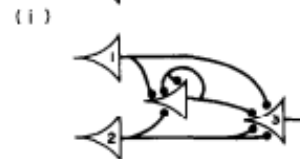
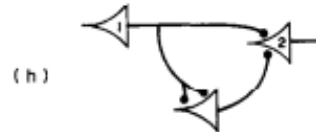
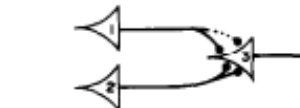
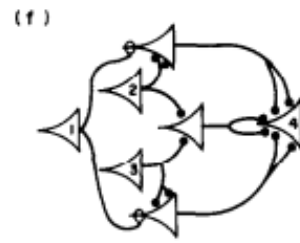
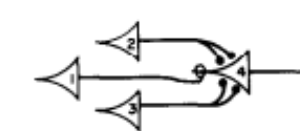
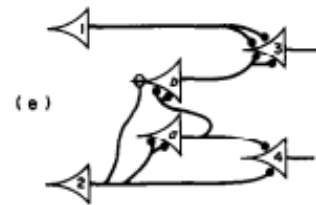
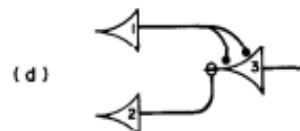
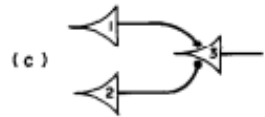
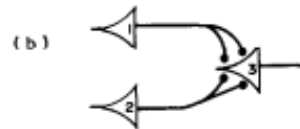
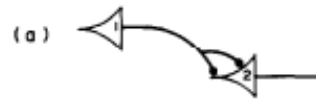
University of Illinois, College of Medicine,
Department of Psychiatry at the Illinois Neuropsychiatric Institute,
University of Chicago, Chicago, U.S.A.

Because of the “all-or-none” character of nervous activity, neural events and the relations among them can be treated by means of propositional logic. It is found that the behavior of every net can be described in these terms, with the addition of more complicated logical means for nets containing circles; and that for any logical expression satisfying certain conditions, one can find a net behaving in the fashion it describes. It is shown that many particular choices among possible neurophysiological assumptions are equivalent, in the sense that for every net behaving under one assumption, there exists another net which behaves under the other and gives the same results, although perhaps not in the same time. Various applications of the calculus are discussed.

1. Introduction. Theoretical neurophysiology rests on certain cardinal assumptions. The nervous system is a net of neurons, each having a soma and an axon. Their adjunctions, or synapses, are always between the axon of one neuron and the soma of another. At any instant a neuron has some threshold, which excitation must exceed to initiate an impulse. This, except for the fact and the time of its occurrence, is determined by the neuron, not by the excitation. From the point of excitation the impulse is propagated to all parts of the neuron. The velocity along the axon varies directly with its diameter, from $< 1 \text{ ms}^{-1}$ in thin axons, which are usually short, to $> 150 \text{ ms}^{-1}$ in thick axons, which are usually long. The time for axonal conduction is consequently of little importance in determining the time of arrival of impulses at points unequally remote from the same source. Excitation across synapses occurs predominantly from axonal terminations to somata. It is still a moot point whether this depends upon irreducibility of individual synapses or merely upon prevalent anatomical configurations. To suppose the latter requires no hypothesis *ad hoc* and explains known exceptions, but any assumption as to cause is compatible with the calculus to come. No case is known in which excitation through a single synapse has elicited a nervous impulse in any neuron, whereas any neuron may be excited by impulses arriving at a sufficient number of neighboring synapses within the period of latent addition, which lasts $< 0.25 \text{ ms}$. Observed temporal summation of impulses at greater intervals

* Reprinted from the *Bulletin of Mathematical Biophysics*, Vol. 5, pp. 115–133 (1943).

Représentations neuronales



A SYMBOLIC ANALYSIS

OF

RELAY AND SWITCHING CIRCUITS

by

Claude Elwood Shannon

B.S., University of Michigan

1936

Submitted in Partial Fulfillment of the

Requirements for the Degree of

MASTER OF SCIENCE

from the

Massachusetts Institute of Technology

1940

Signature of Author Signature redacted

Department of Electrical Engineering, August 10, 1937

Signature of Professor Signature redacted
in Charge of Research

Signature of Chairman of Department Signature redacted
Committee on Graduate Students

La solution connexionniste



FIG. 1 — Organization of a biological brain. (Red areas indicate active cells, responding to the letter X.)

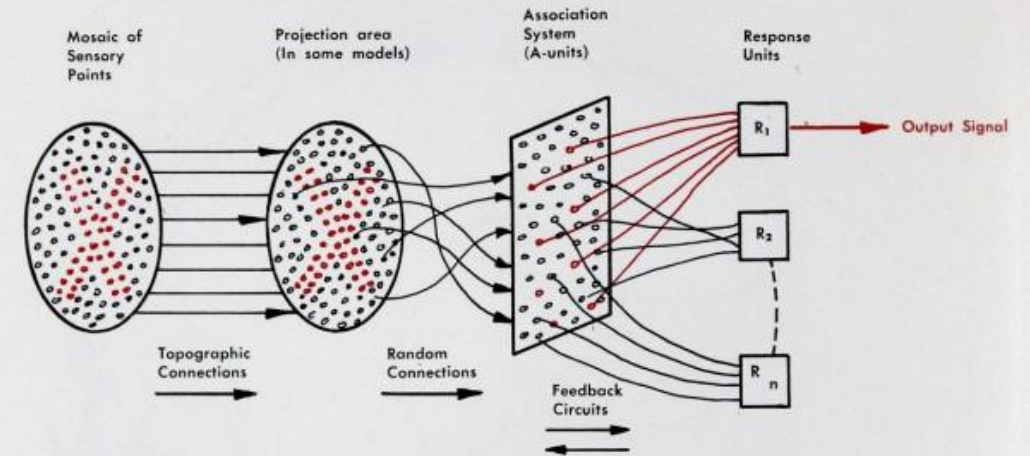
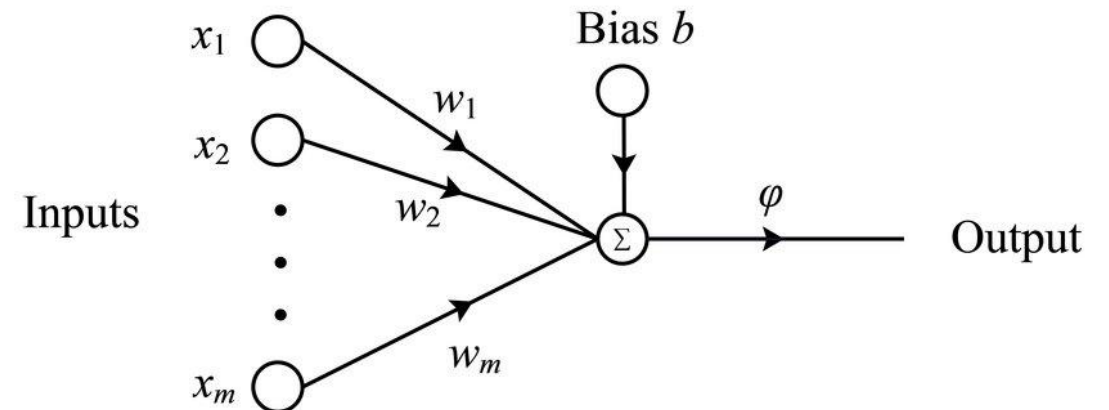
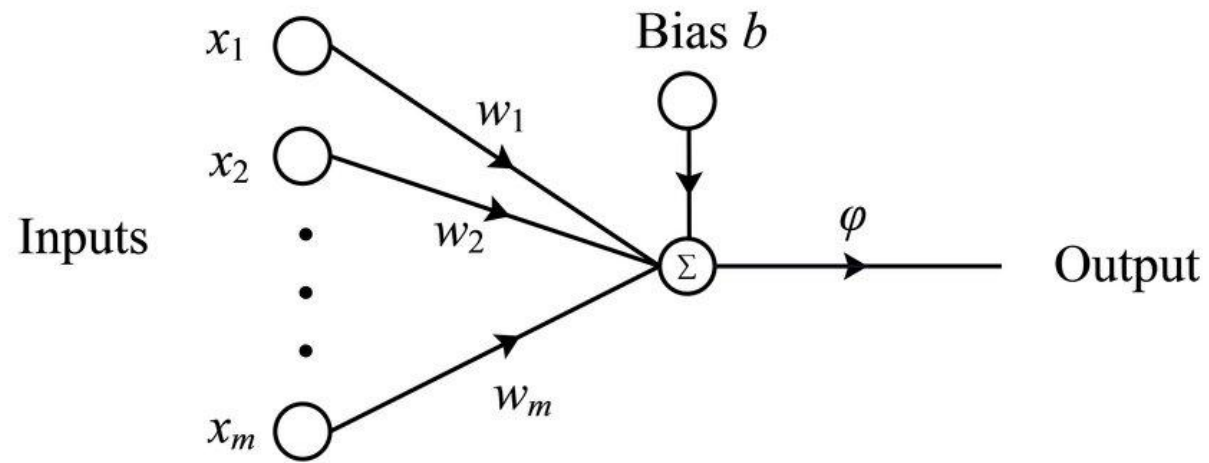


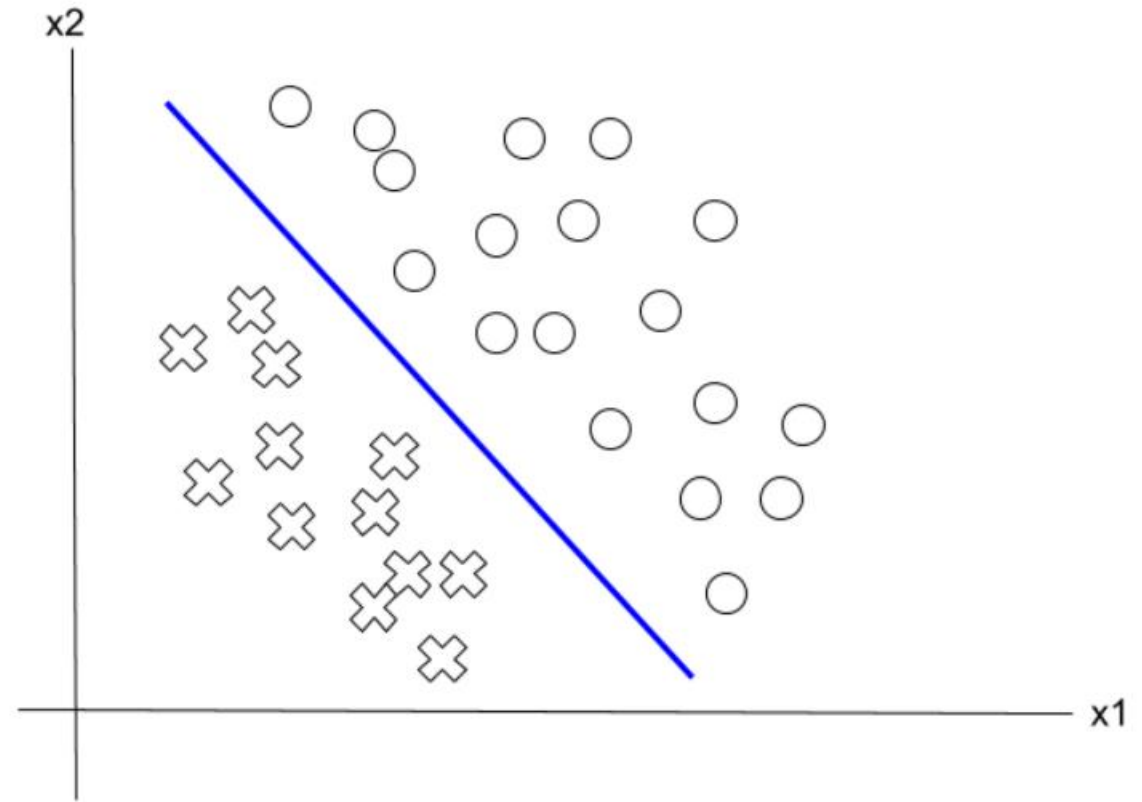
FIG. 2 — Organization of a perceptron.



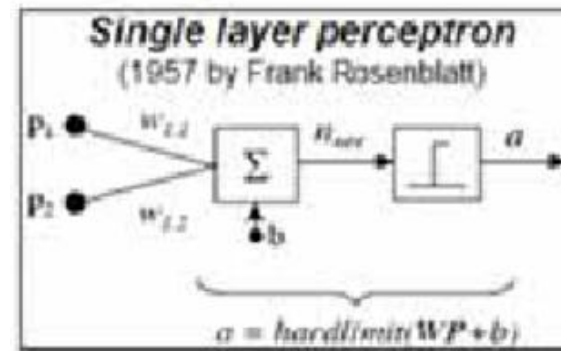
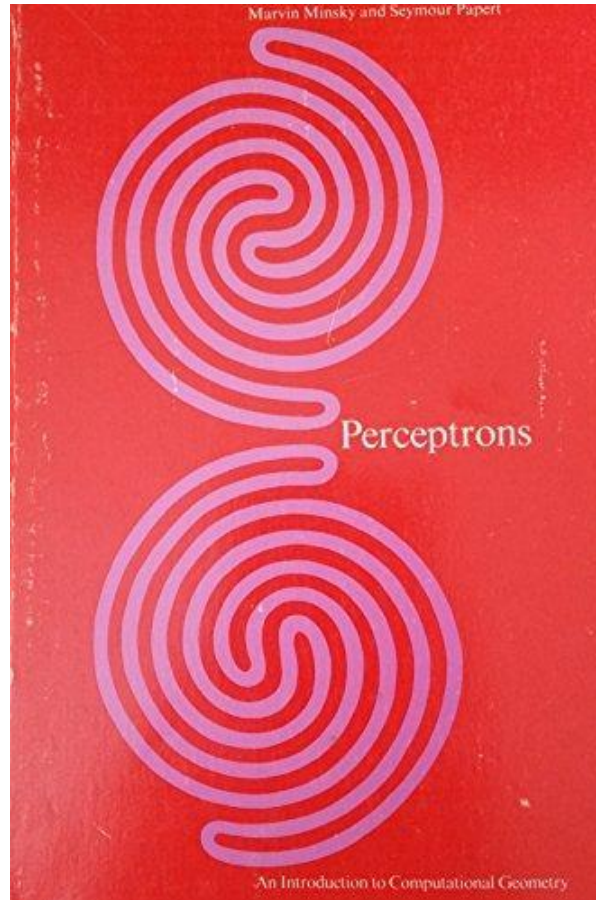
Représentations neuronales



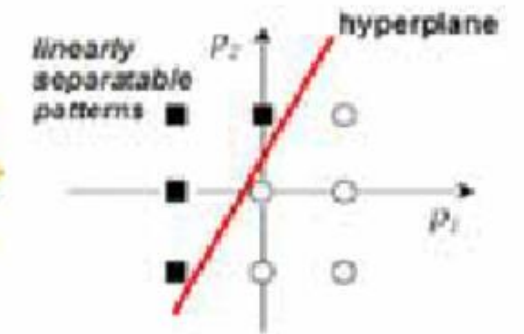
$$y = mx + b$$



Représentations neuronales



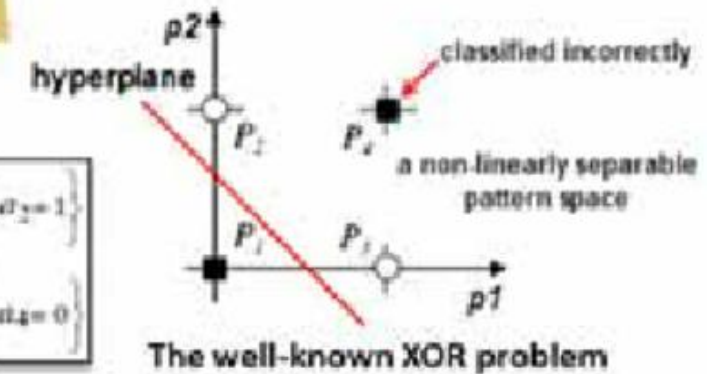
Classify OK



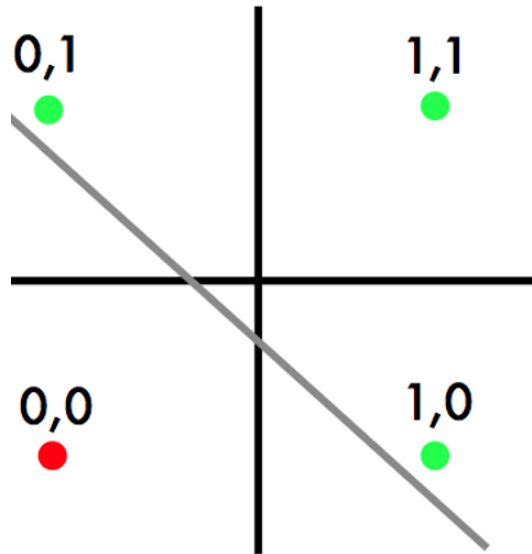
Cannot classify correctly!
(In the book "Perceptrons" in 1969)

XOR function

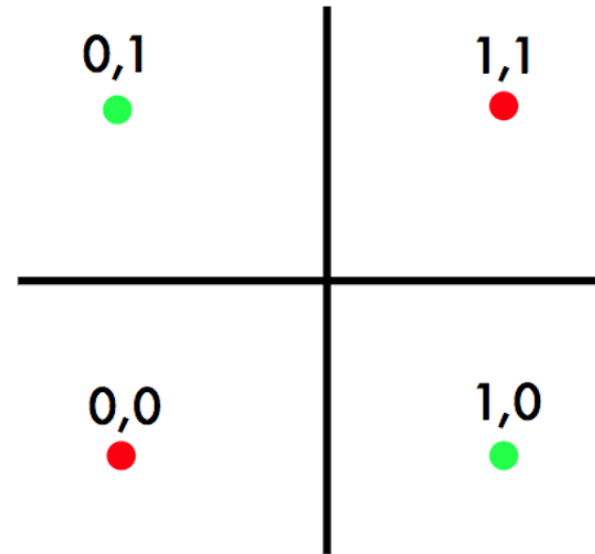
$\left\{ p_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad a_1 = 0 \right\}$	$\left\{ p_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad a_2 = 1 \right\}$
$\left\{ p_3 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad a_3 = 1 \right\}$	$\left\{ p_4 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad a_4 = 0 \right\}$



Représentations neuronales

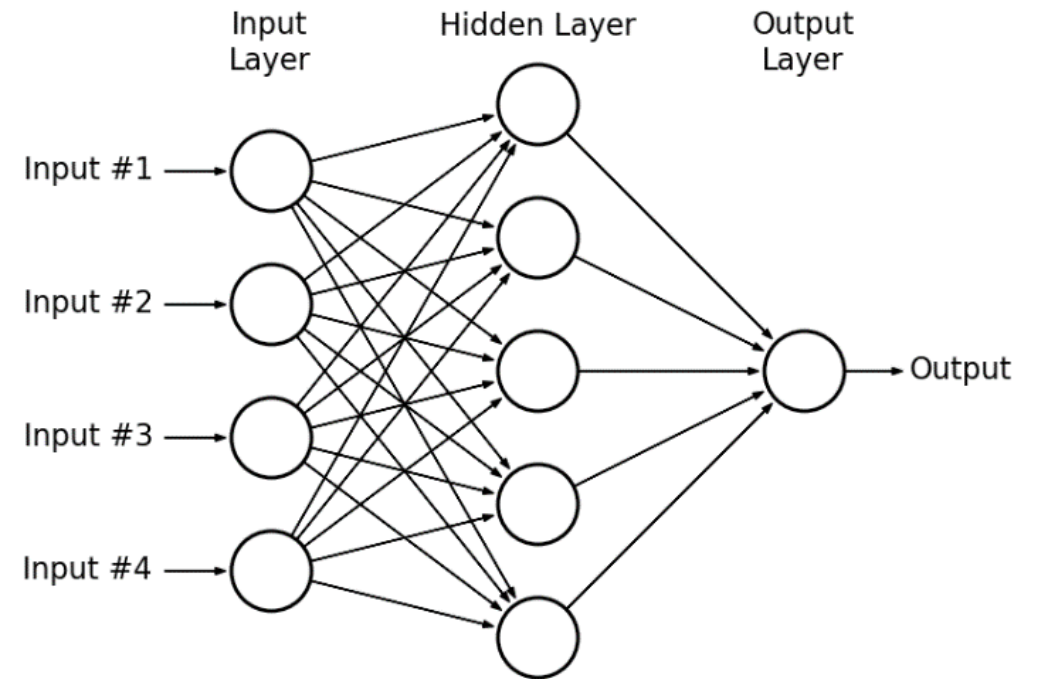
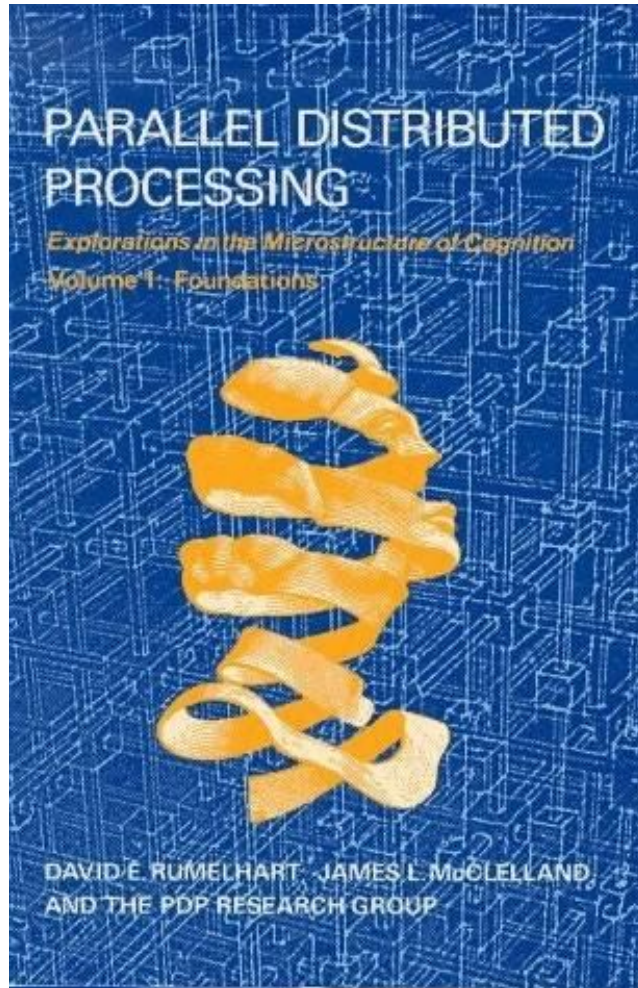


OR

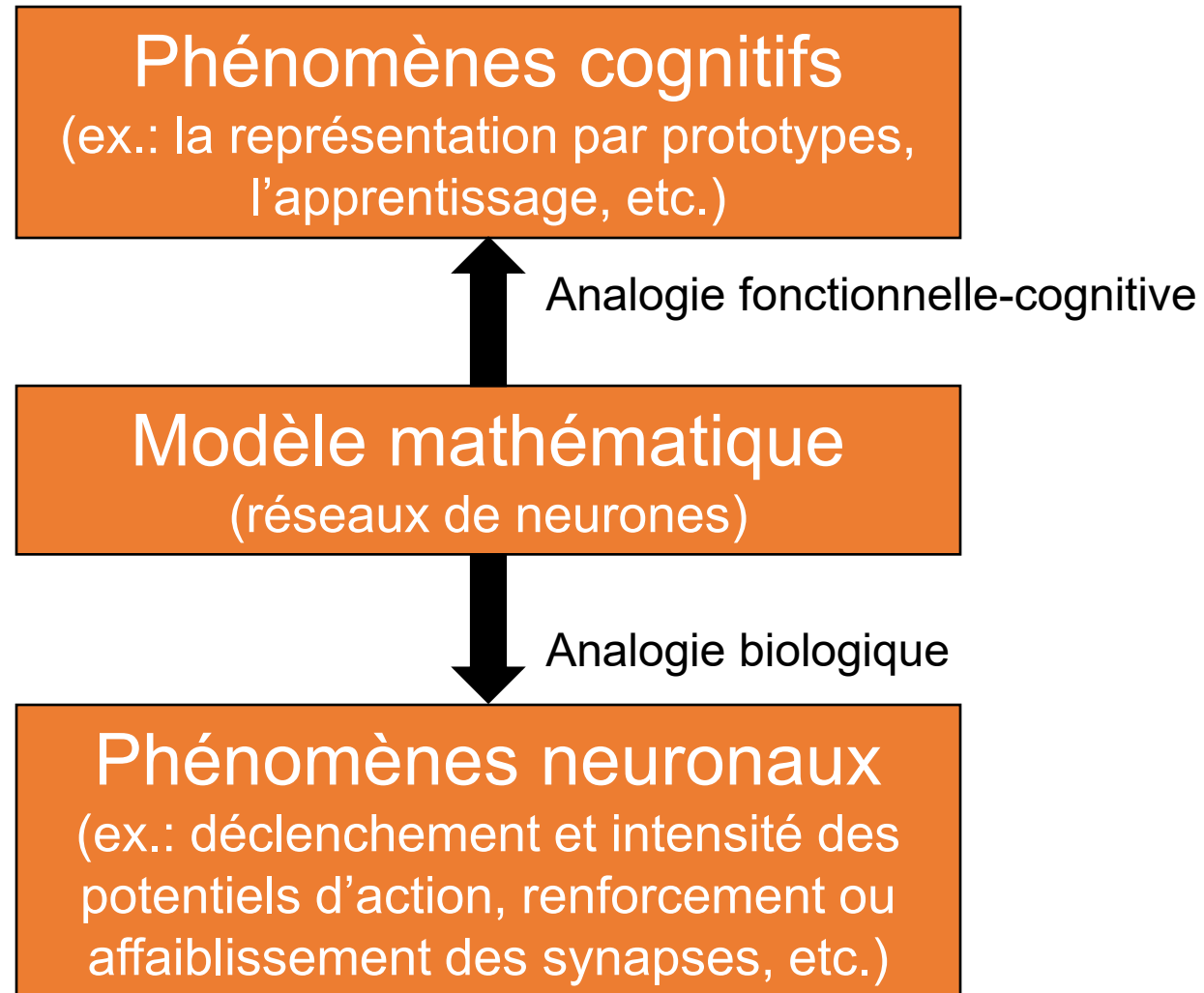


XOR

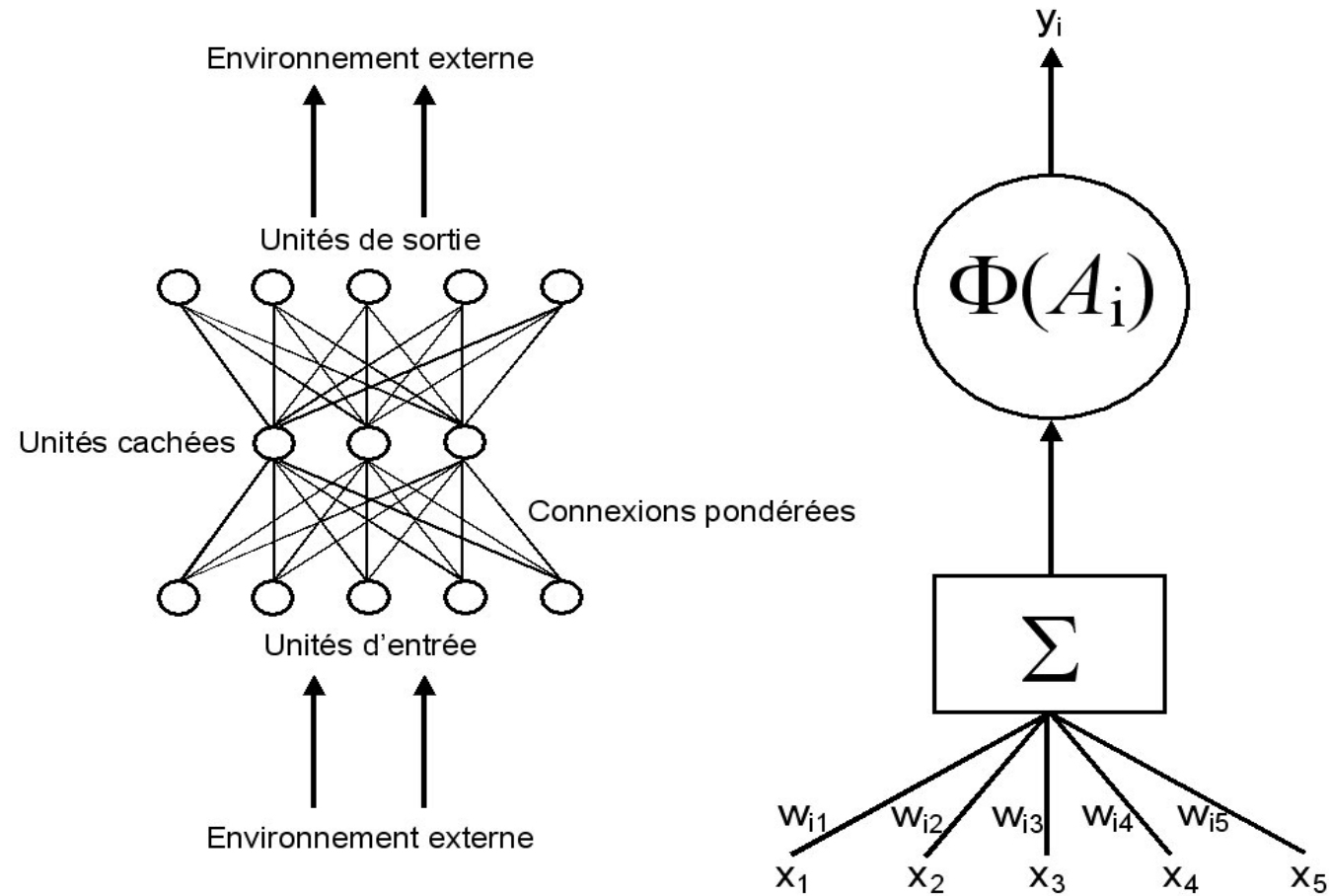
Représentations neuronales



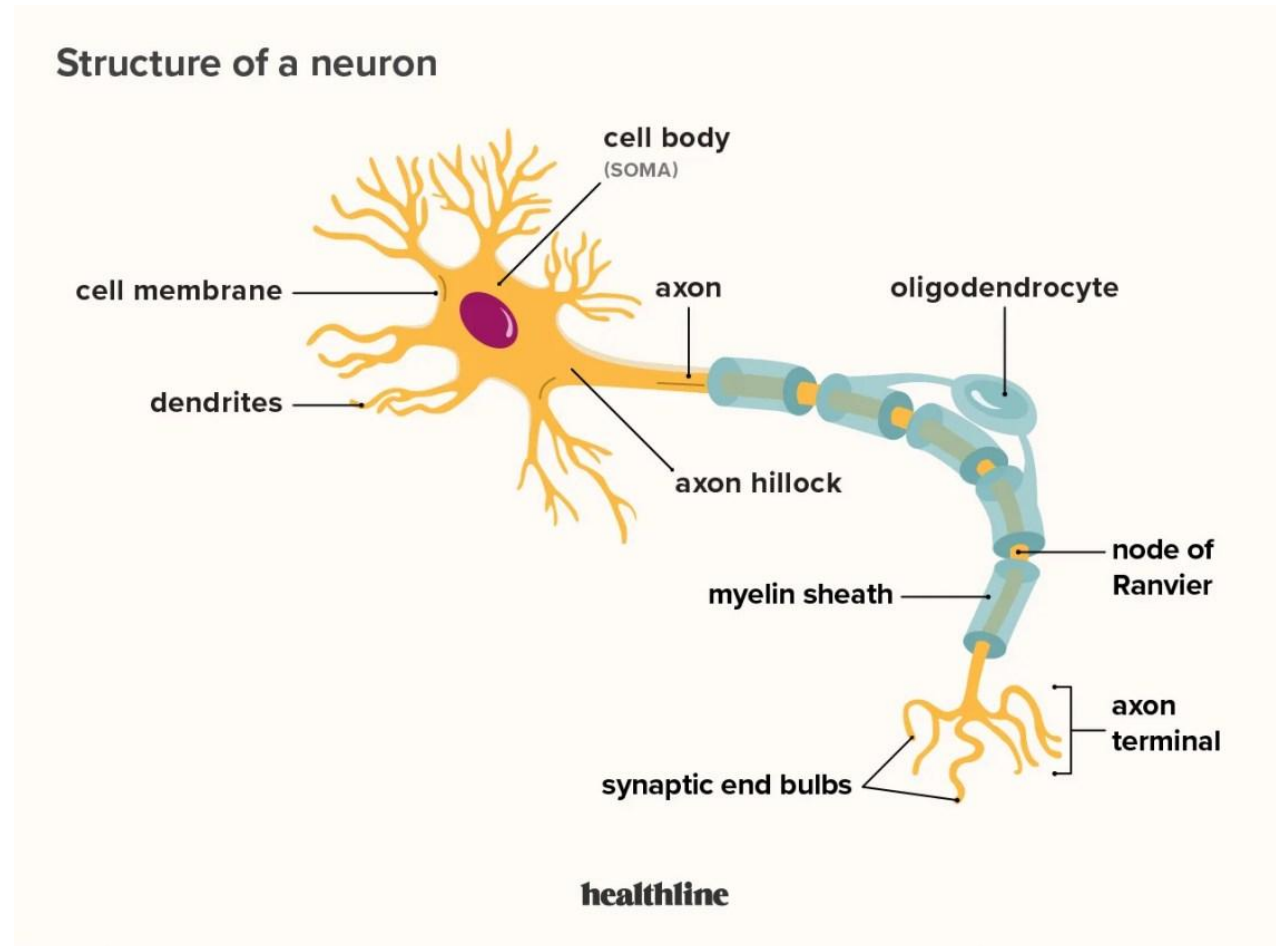
La modélisation connexionniste



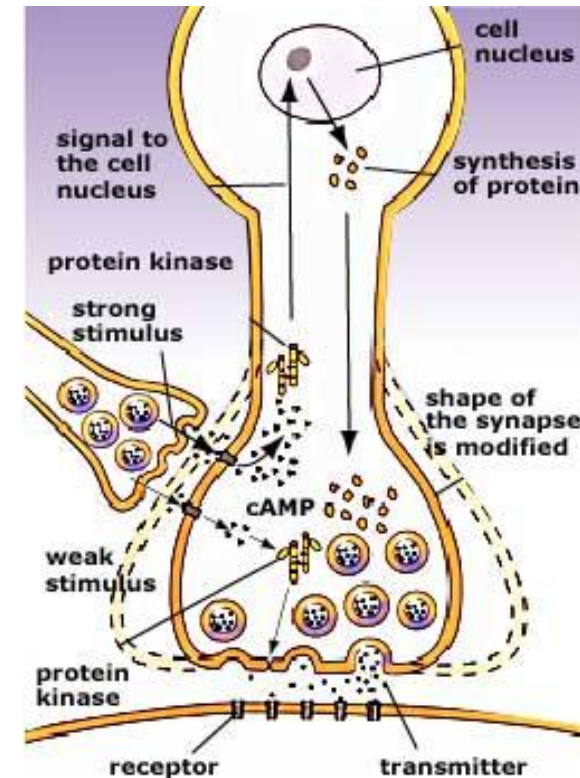
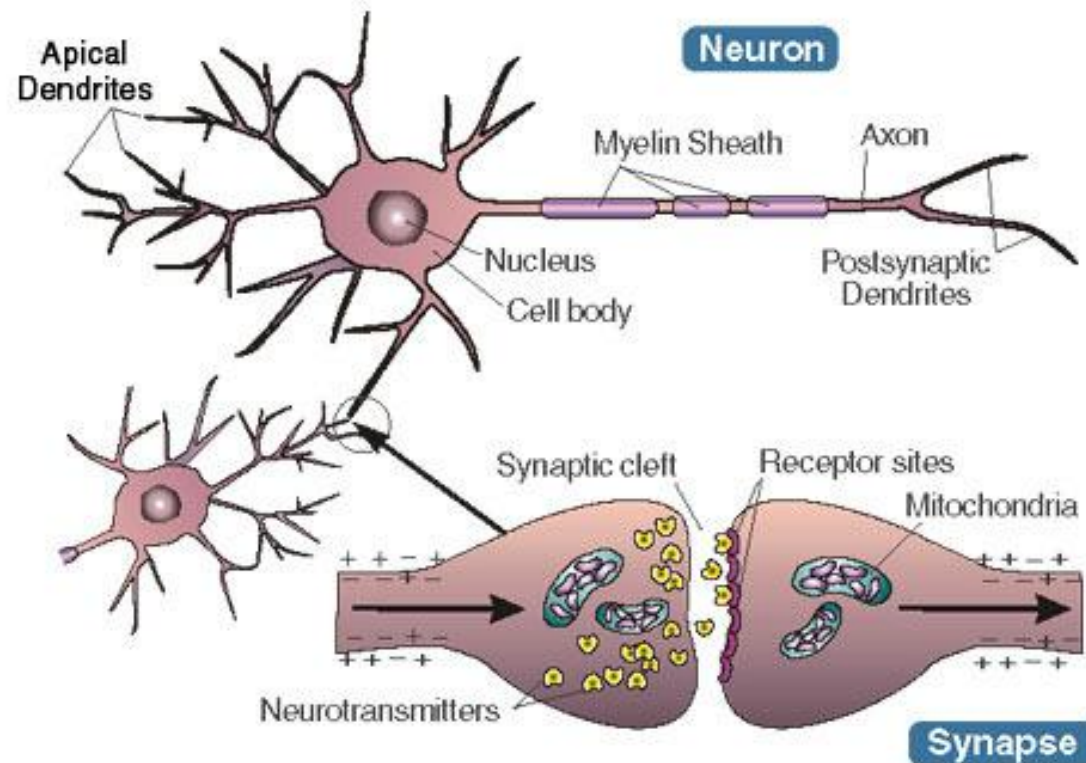
La modélisation connexionniste



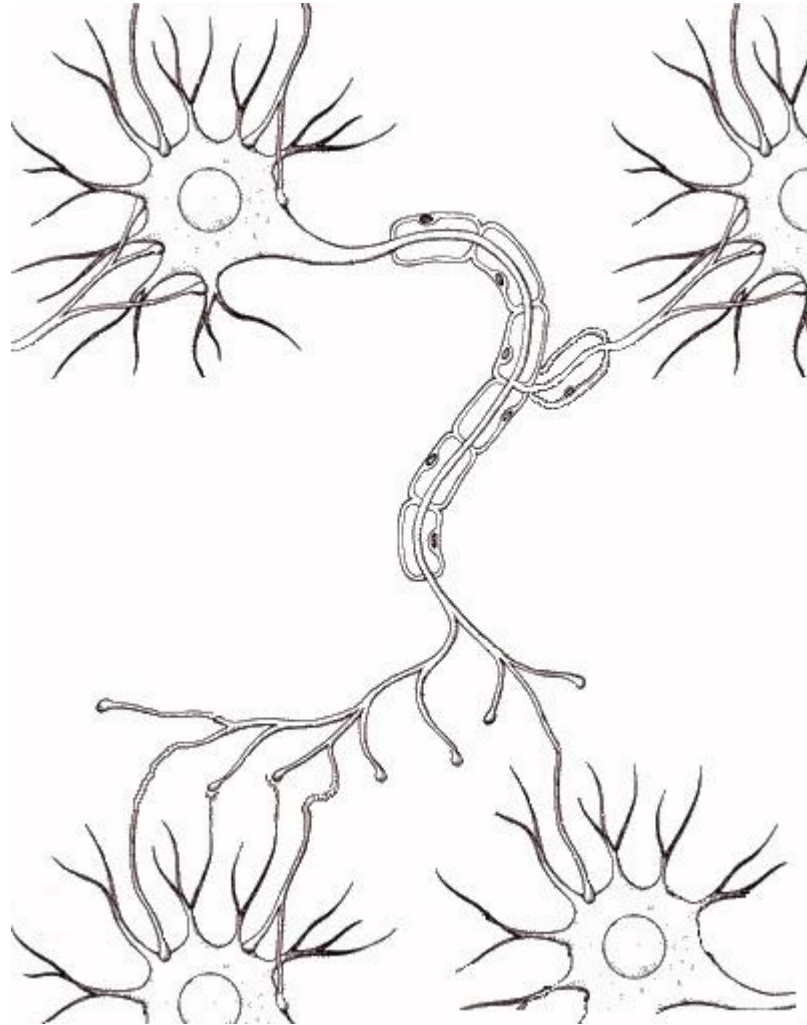
L'analogie biologique: Le neurone



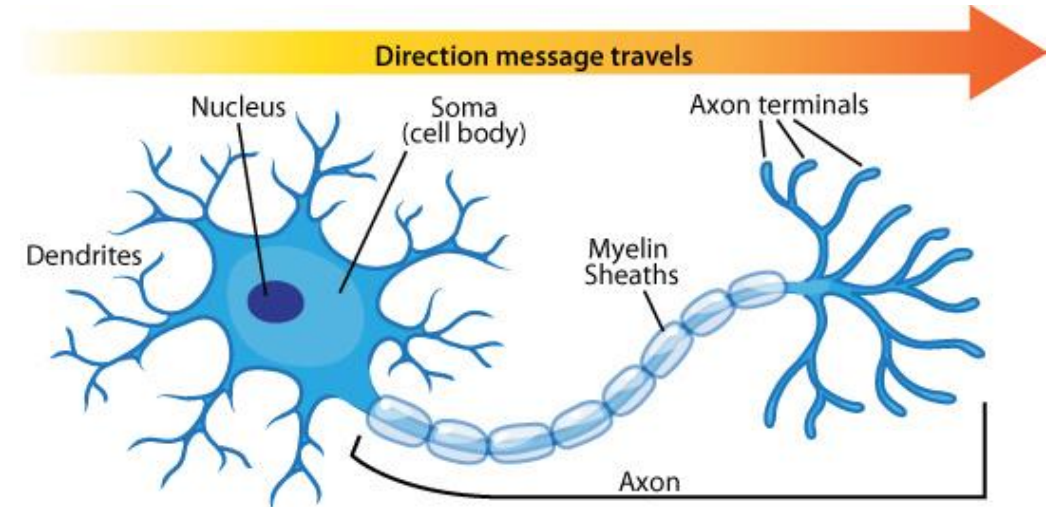
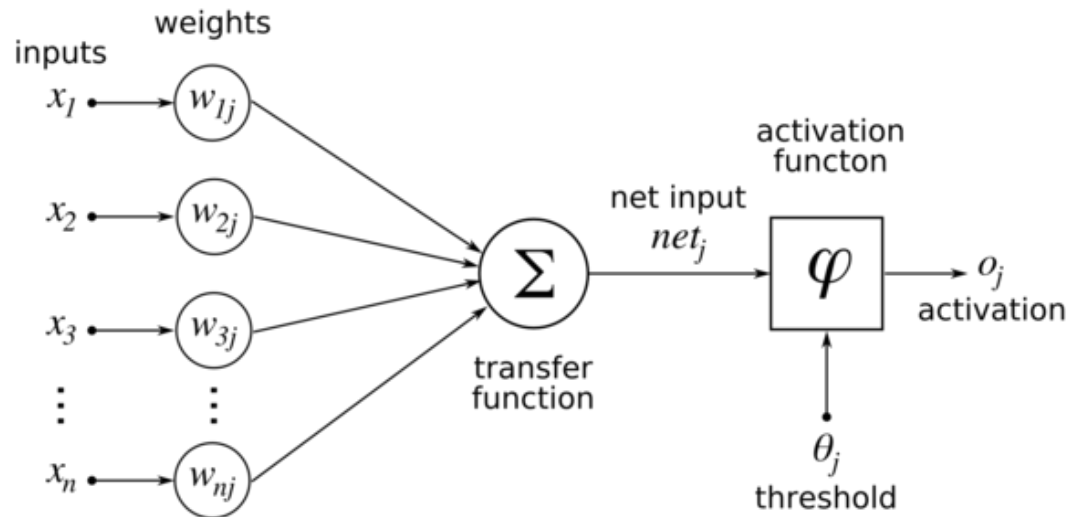
L'analogie biologique: La synapse



L'analogie biologique: Les connexions

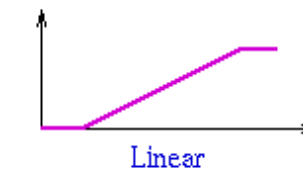
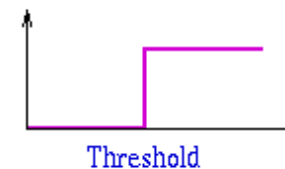
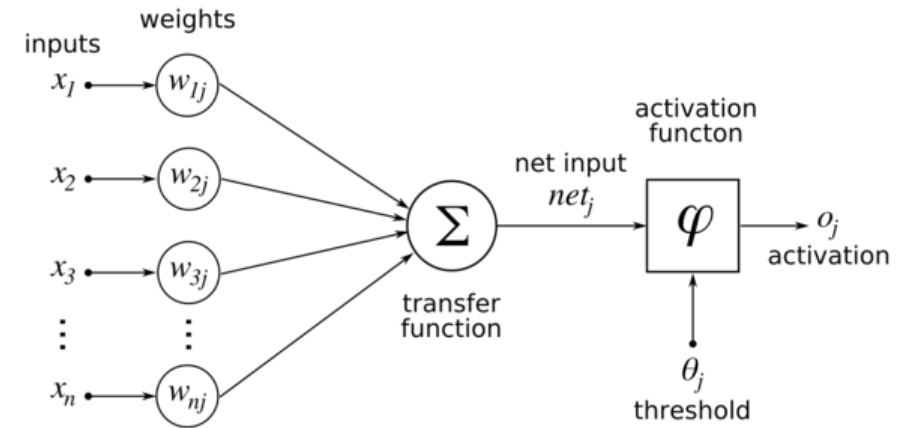


L'analogie biologique: un modèle mathématique du neurone



L'analogie biologique: un modèle mathématique du neurone

- Fonction d'intégration (prise d'information):
 - $net_j = \sum a_j w_{ji}$
- Fonction d'activation (prise de décision):
 - $y_j = F(net_j)$
 - $y_i = net_j$
 - $y_j = 1 / (1 + e^{-net_j})$



L'analogie biologique: un modèle mathématique du neurone

- Fonction d'intégration (prise d'information):
 - $net_j = \sum a_j w_{ji}$
- Fonction d'activation (prise de décision):
 - $y_j = F(net_j)$
 - $y_i = net_j$
 - $y_j = 1 / (1 + e^{-net_j})$



Analogies biologiques: la boîte à outils de modélisation

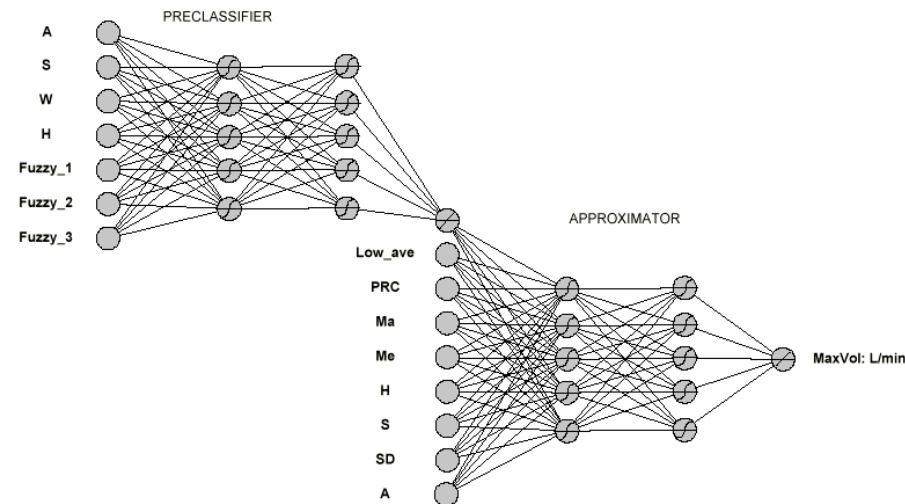
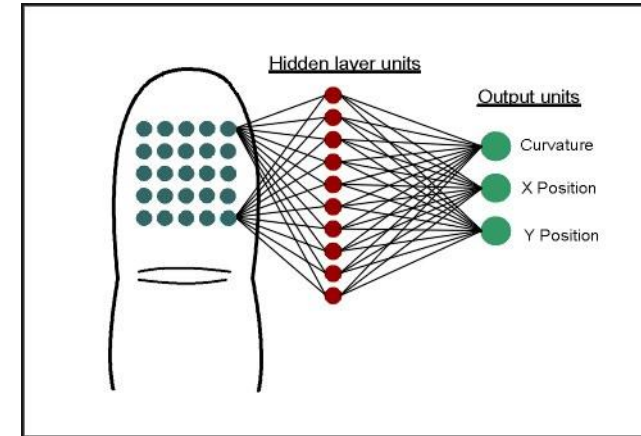
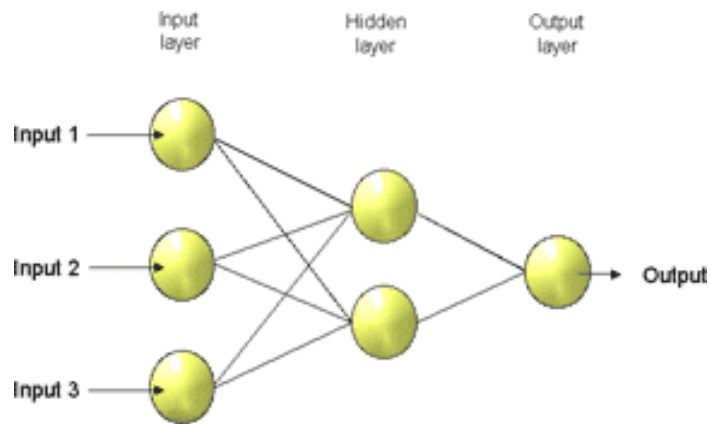
- Architecture du neurone

- Neurone point ou « en compartiment »
- Fonction d'intégration et de transfert (linéaire ou non, etc.)
- Seuil
- Taux d'apprentissage
- *Momentum*
- Etc., etc.

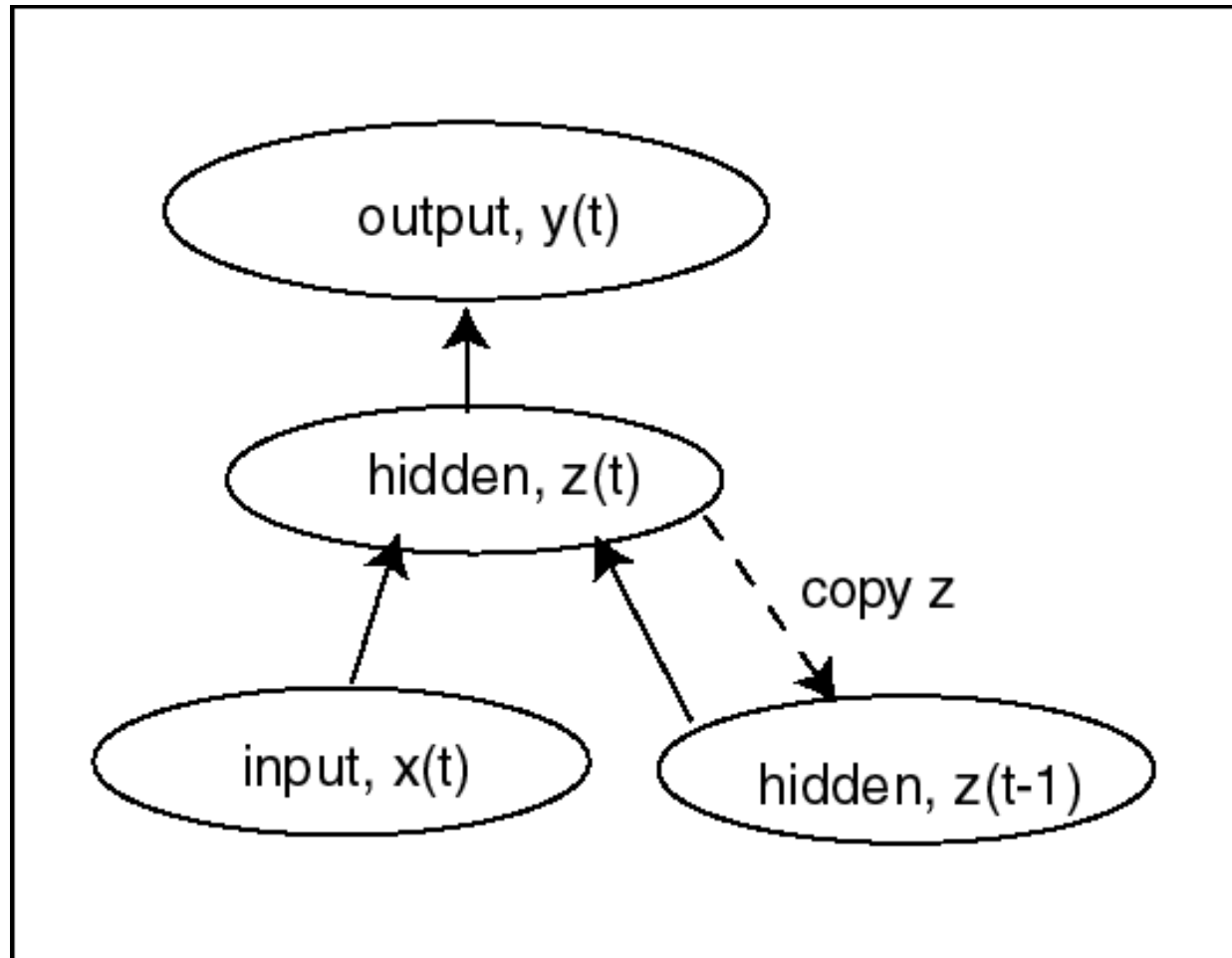
- Architecture du réseau

- Diffus, en bassin ou en couche
- Couches d'entrée, de sortie, cachée
- Nombre de couches cachées
- Connexion feed-forward, multi-directionnelles (feed-forward et feedback), récurrentes
- Excitation vs inhibition
- Modules ou non
- 1D ou 2D
- Pures ou hybrides
- Etc., etc.

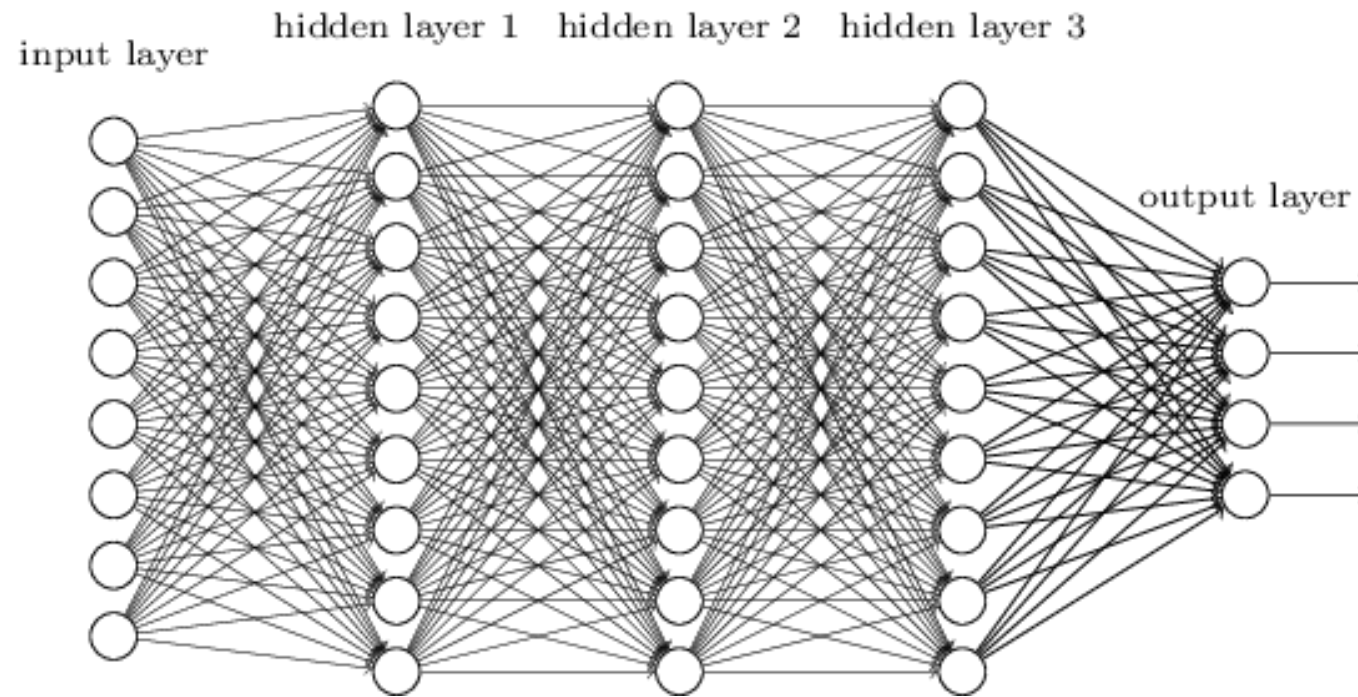
Analogies biologiques: la boîte à outils de modélisation



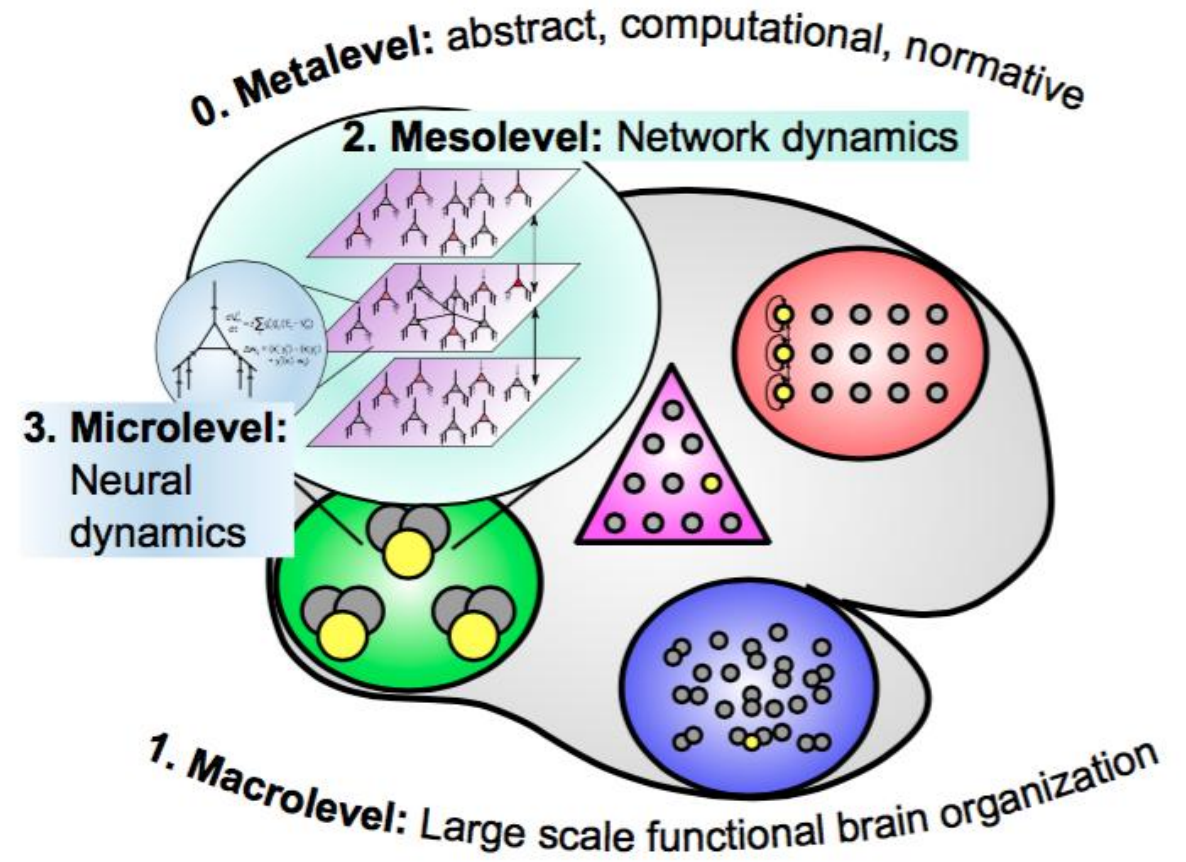
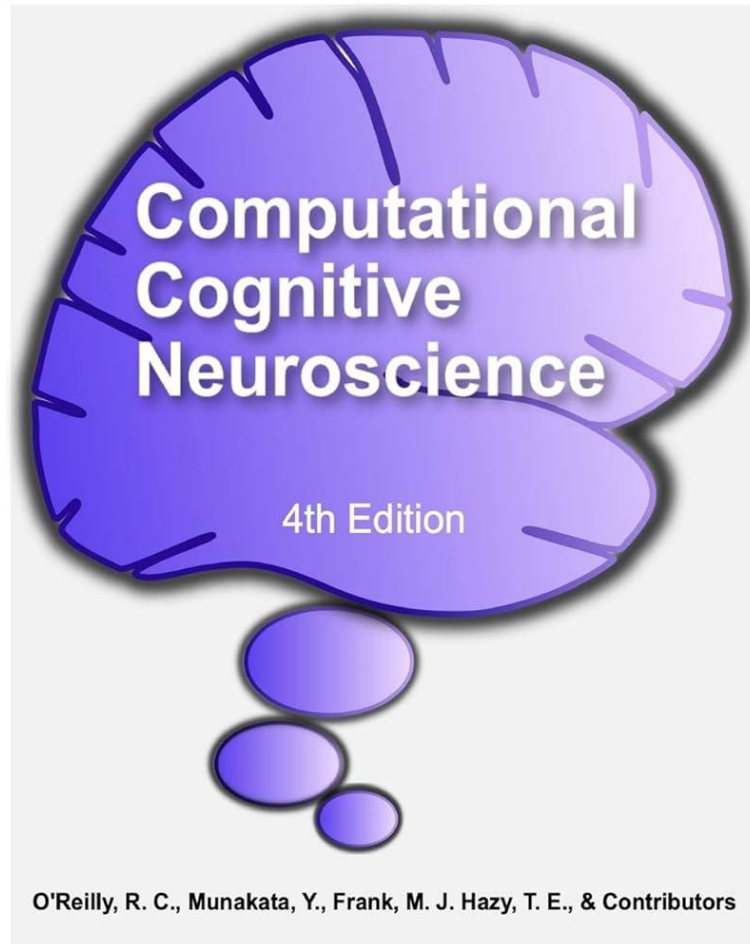
Analogies biologiques: la boîte à outils de modélisation



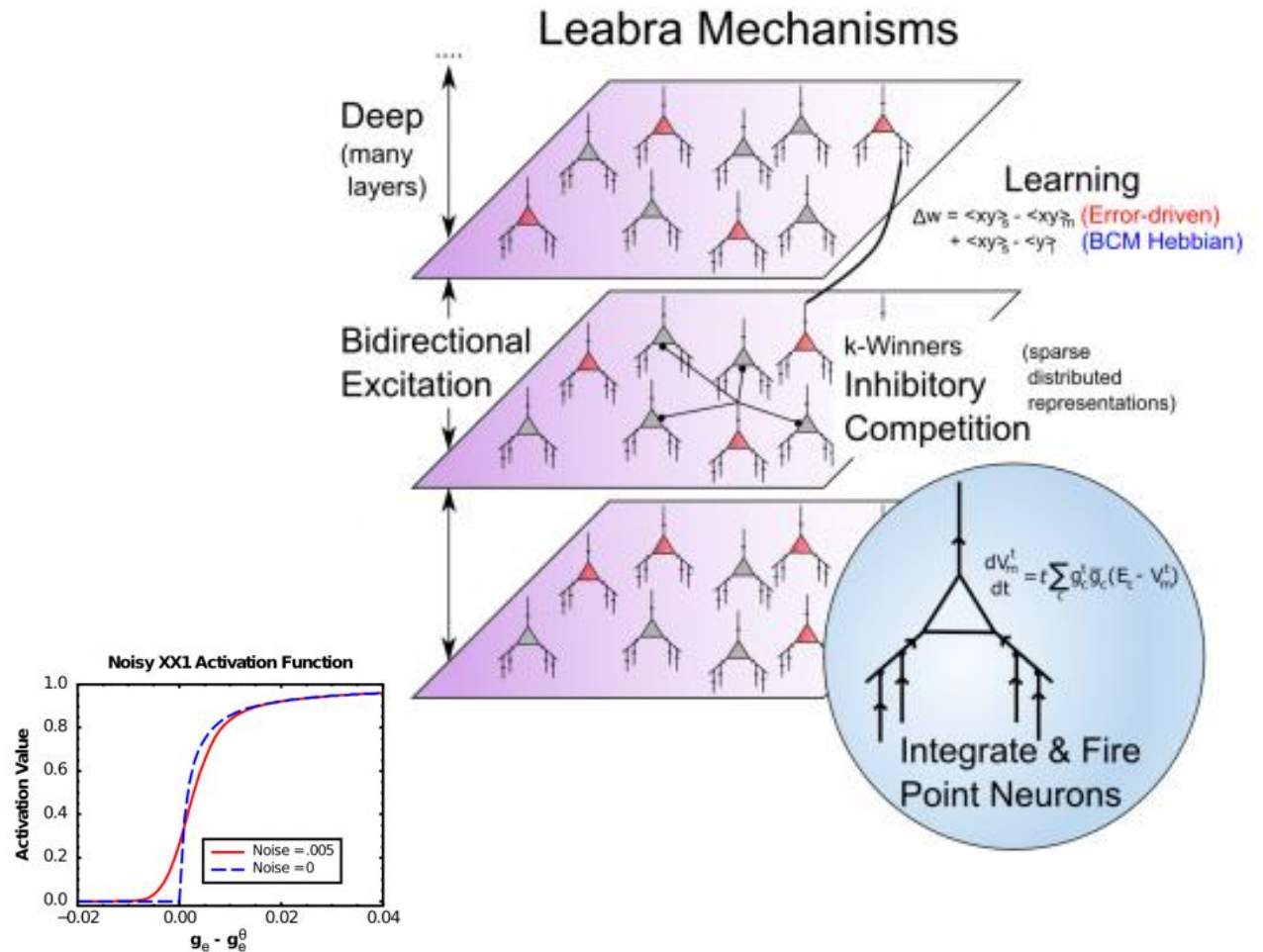
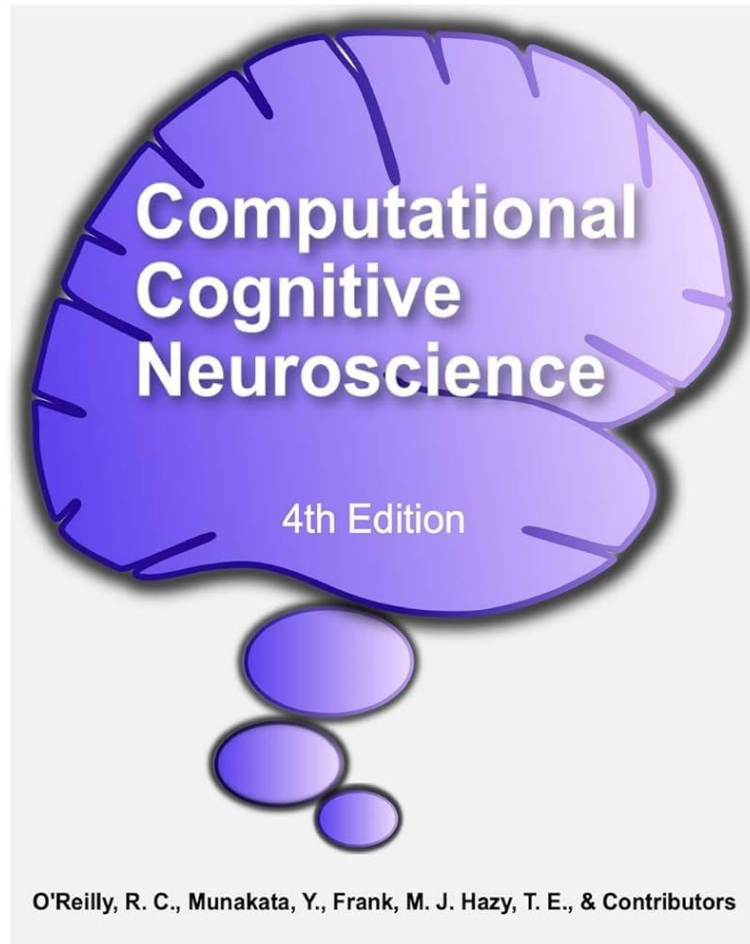
Analogies biologiques: la boîte à outils de modélisation



Analogies biologiques: neurosciences cognitives computationnelles



Analogies biologiques: neurosciences cognitives computationnelles



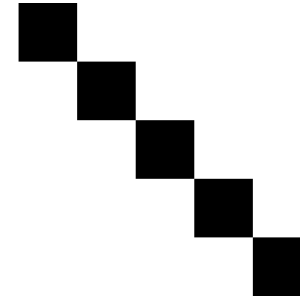
Analogie fonctionnelle: deux formes d'apprentissages

- Apprentissage corrélationnel
 - Apprentissage associatif (non supervisé) de modèles corrélationnels
- Apprentissage pragmatique
 - Apprentissage supervisé de tâches

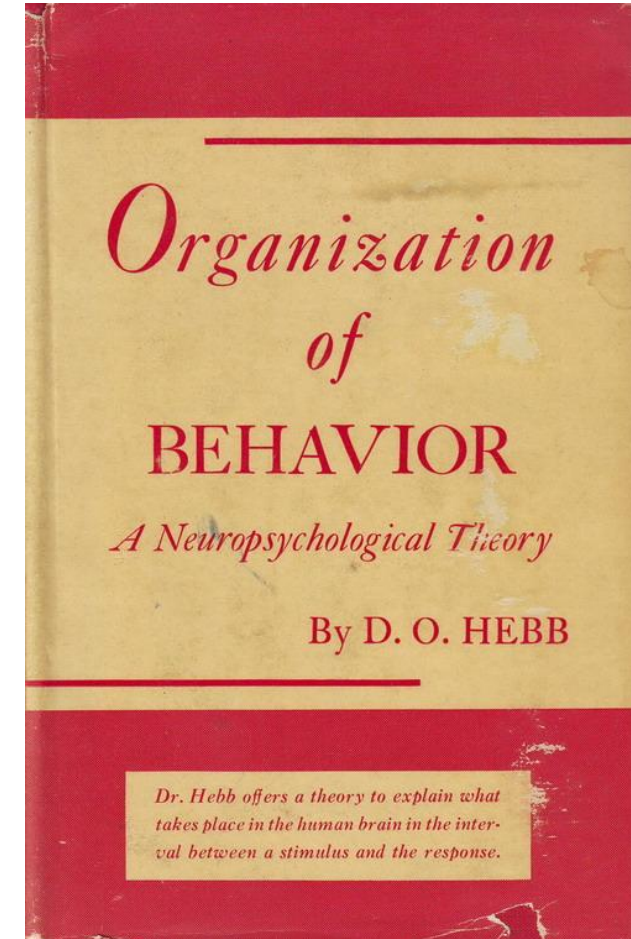


Analogie fonctionnelle: apprentissage corrélational (non supervisé)

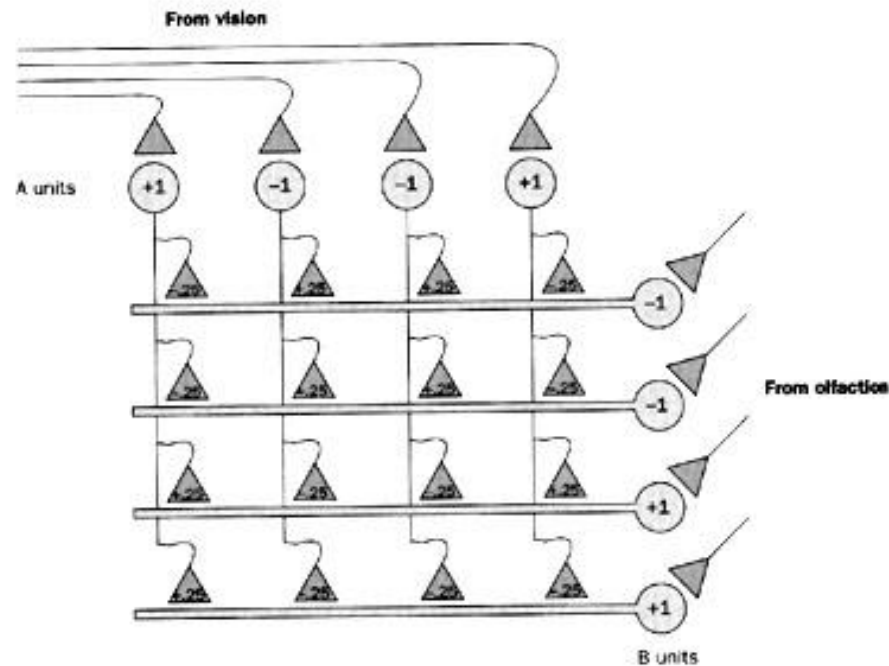
- Apprentissage associatif (hebbien)
 - Apprentissage non supervisé, auto-organisé (*self-organizing*)
 - Apprentissage de modèles corrélacionnels probabilistes
 - Utile pour extraire la structure statistique de l'environnement
 - Descriptif (représentation du monde)
 - Biologiquement réaliste (LTP / LTD)
 - Incapable de s'adapter aux exigences fonctionnelles
 - Quoi faire, comment traiter l'information, quand tel et tel stimuli est présent



Analogie fonctionnelle: apprentissage corrélationnel (non supervisé)

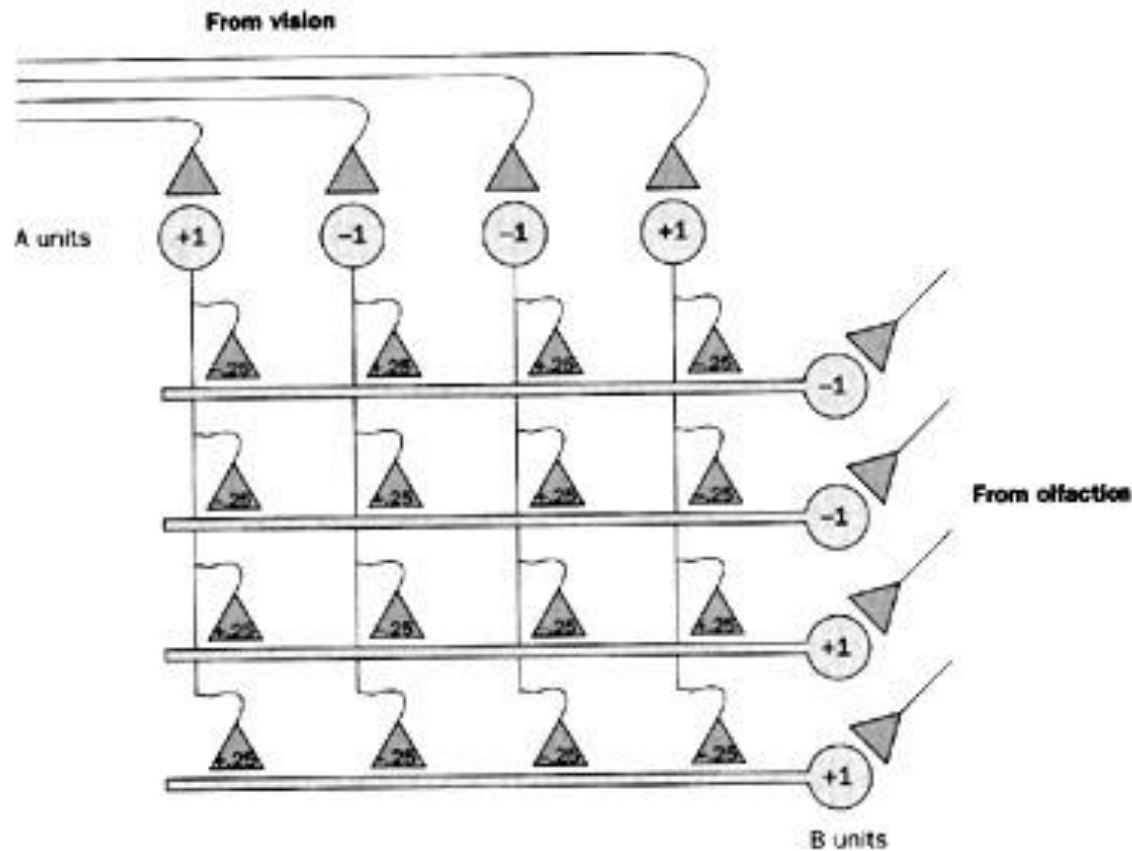


Analogie fonctionnelle: apprentissage corrélationnel (non supervisé)



	O_1	O_2	O_3	O_4
V_1	-0.25	0.25	0.25	-0.25
V_2	-0.25	0.25	0.25	0.25
V_3	0.25	-0.25	-0.25	0.25
V_4	0.25	-0.25	-0.25	-0.25

Analogie fonctionnelle: apprentissage corrélationnel (non supervisé)



$$w_{ji(t+1)} = w_{ji(t)} + \Delta w_{ji(t)}$$

$$\Delta w_{ji(t)} = \eta a_i a_j$$

i.e., le changement synaptique est fonction de la corrélation entre l'activité du neurone présynaptique a_i et du neurone postsynaptique a_j

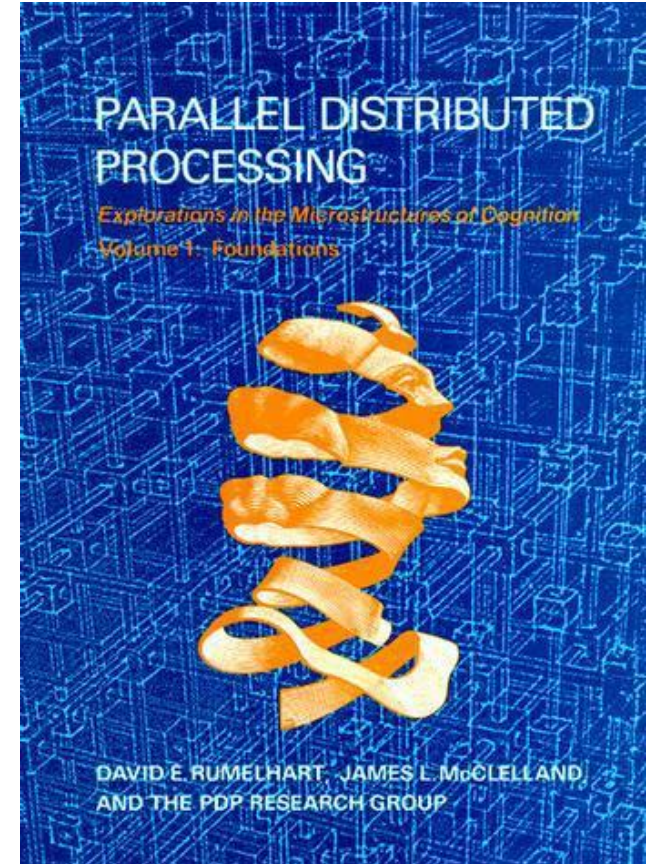
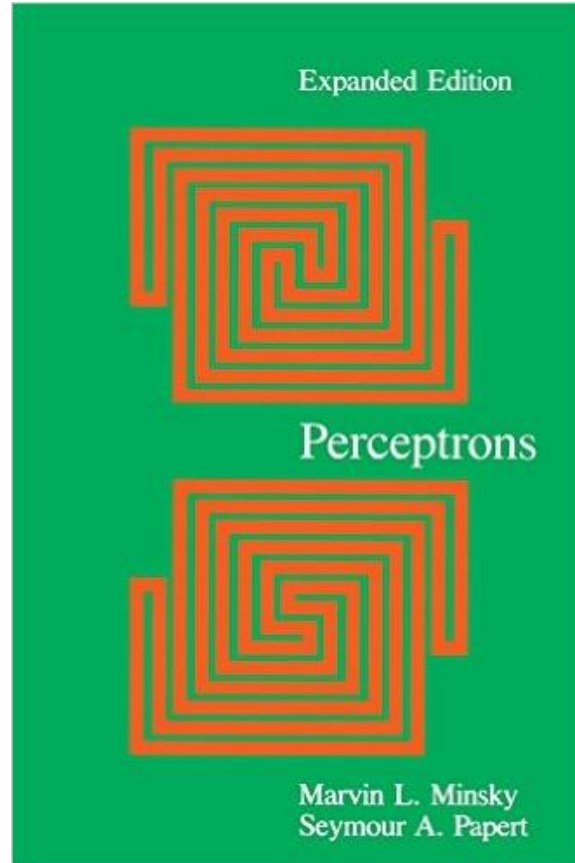
Analogie fonctionnelle: apprentissage pragmatique

- Apprentissage pragmatique dirigé par l'erreur (*error-driven*)
 - Apprentissage supervisé
 - Apprentissage de tâches
 - Utile pour savoir quoi faire
 - Directif (pilote de l'action, pragmatique)
 - Réalisme biologique plus difficile (mais impliquant probablement la dopamine)

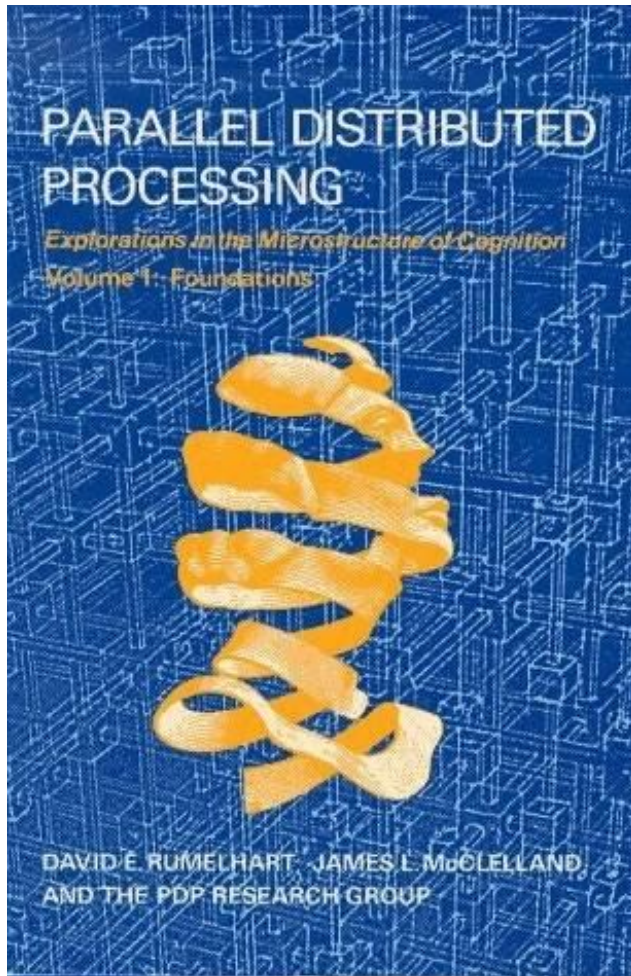
$$\begin{array}{r} 652 \\ +471 \\ \hline 3 \end{array}$$



Analogie fonctionnelle: apprentissage pragmatique dirigé par l'erreur



Analogie fonctionnelle: apprentissage pragmatique dirigé par l'erreur



CHAPTER 8

Learning Internal Representations by Error Propagation

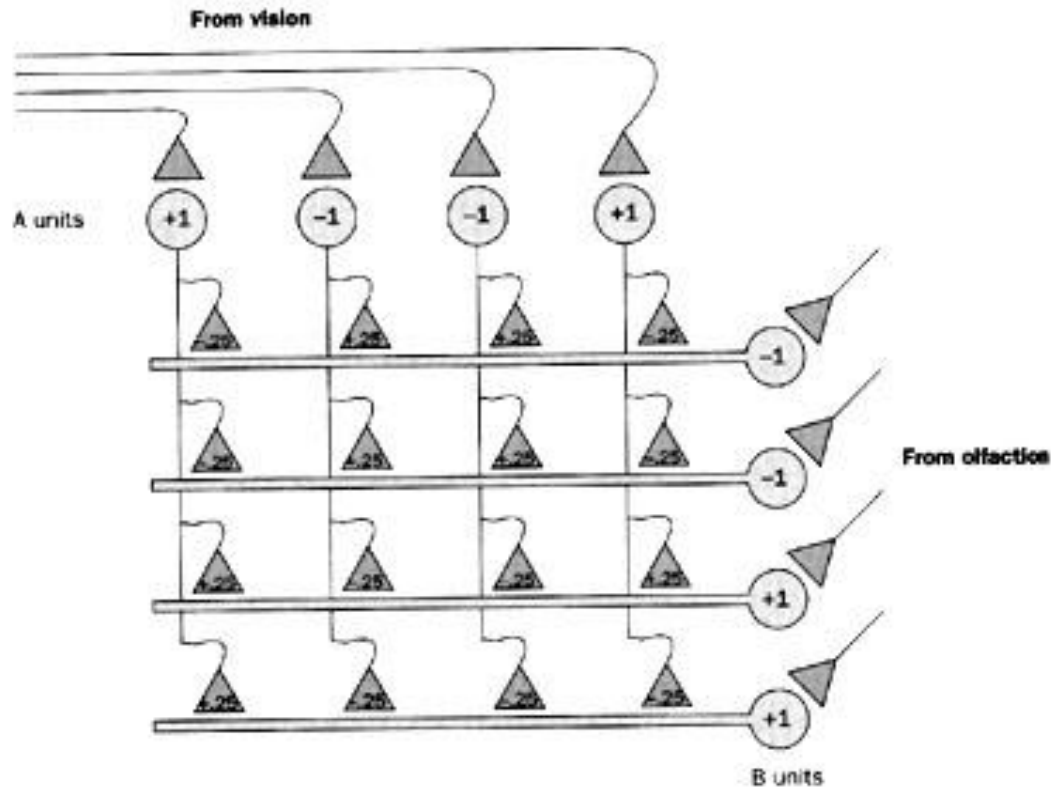
D. E. RUMELHART, G. E. HINTON, and R. J. WILLIAMS

THE PROBLEM

We now have a rather good understanding of simple two-layer associative networks in which a set of input patterns arriving at an input layer are mapped directly to a set of output patterns at an output layer. Such networks have no *hidden* units. They involve only *input* and *output* units. In these cases there is no *internal representation*. The coding provided by the external world must suffice. These networks have proved useful in a wide variety of applications (cf. Chapters 2, 17, and 18). Perhaps the essential character of such networks is that they map similar input patterns to similar output patterns. This is what allows these networks to make reasonable generalizations and perform reasonably on patterns that have never before been presented. The similarity of patterns in a PDP system is determined by their overlap. The overlap in such networks is determined outside the learning system itself—by whatever produces the patterns.

The constraint that similar input patterns lead to similar outputs can lead to an inability of the system to learn certain mappings from input to output. Whenever the representation provided by the outside world is such that the similarity structure of the input and output patterns are very different, a network without internal representations (i.e., a

Analogie fonctionnelle: apprentissage pragmatique dirigé par l'erreur

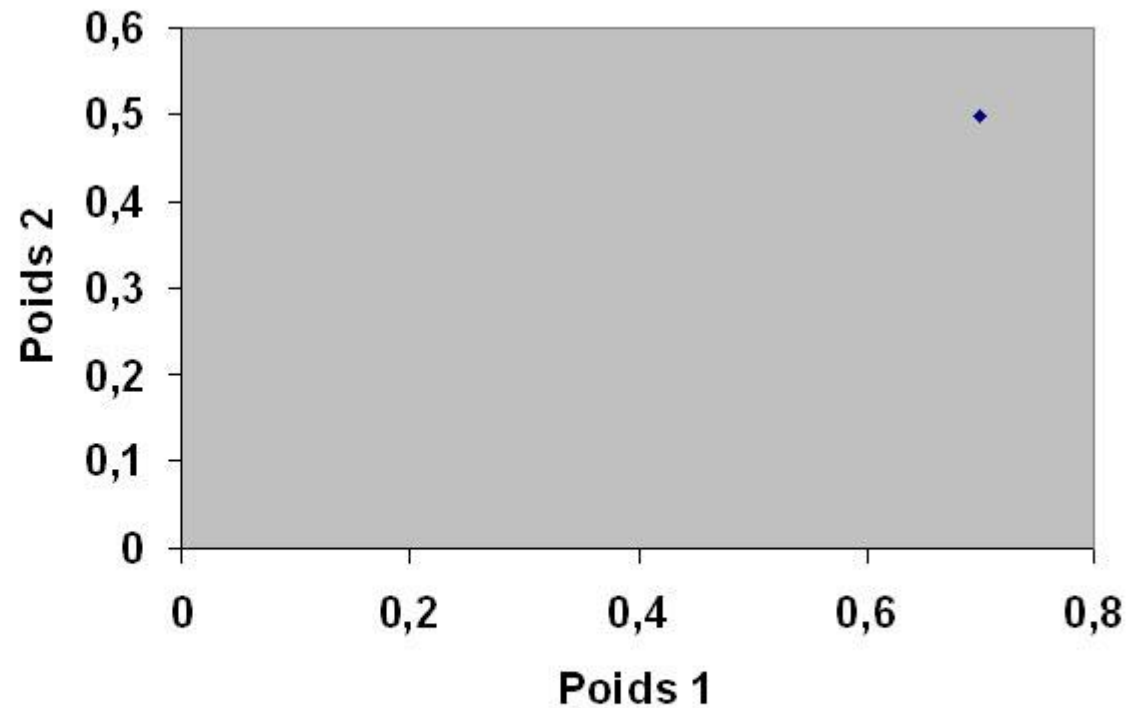
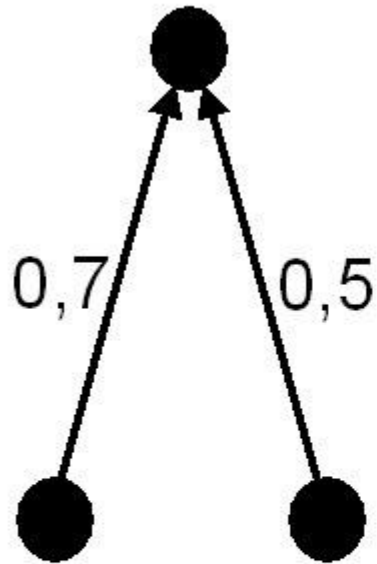


$$w_{ji(t+1)} = w_{ji(t)} + \Delta w_{ji(t)}$$

$$\Delta w_{ji(t)} = \eta \delta_i a_j$$

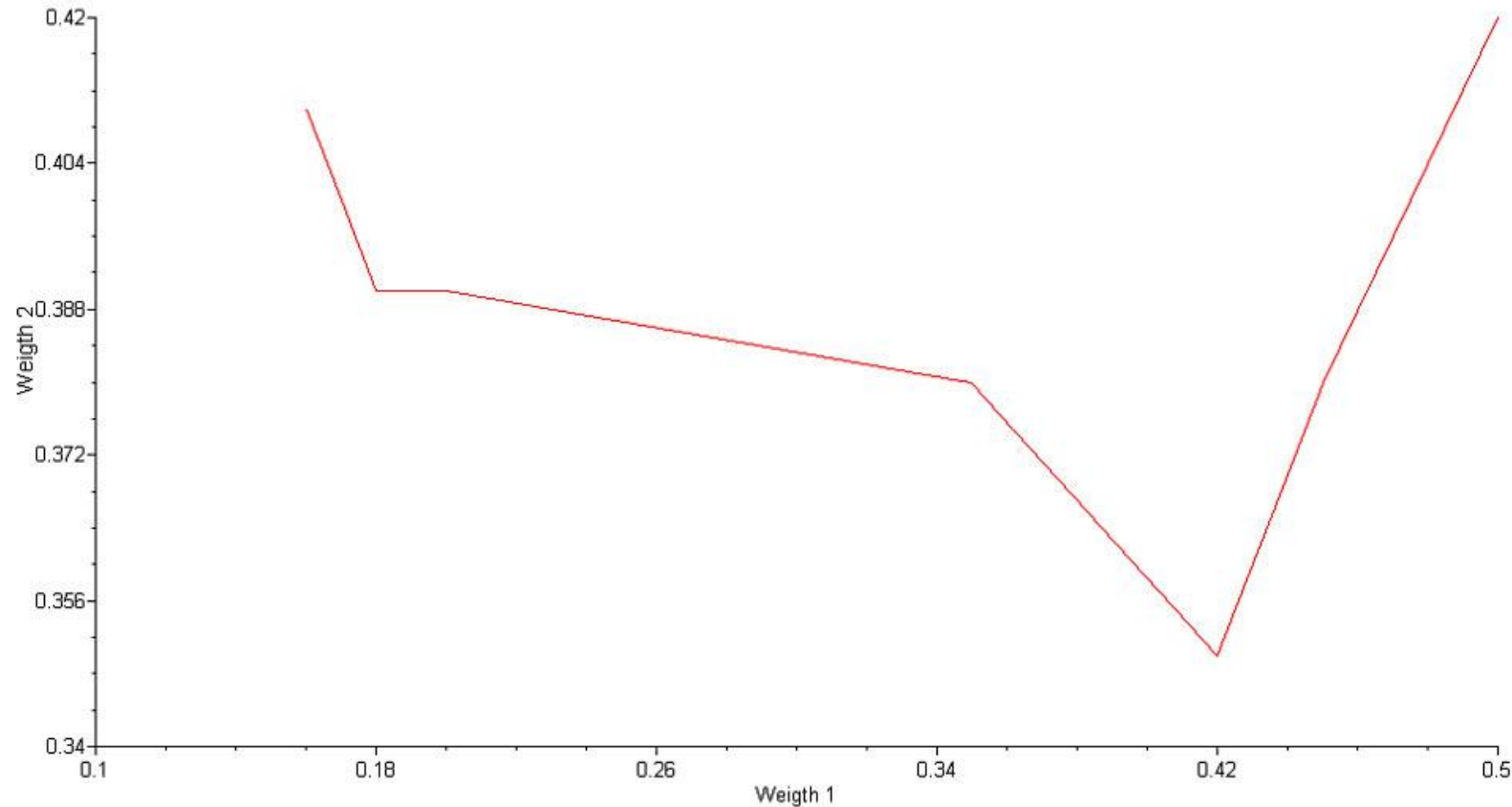
$$\delta_i = t_i - a_i$$

Analogie fonctionnelle: apprentissage pragmatique dirigé par l'erreur



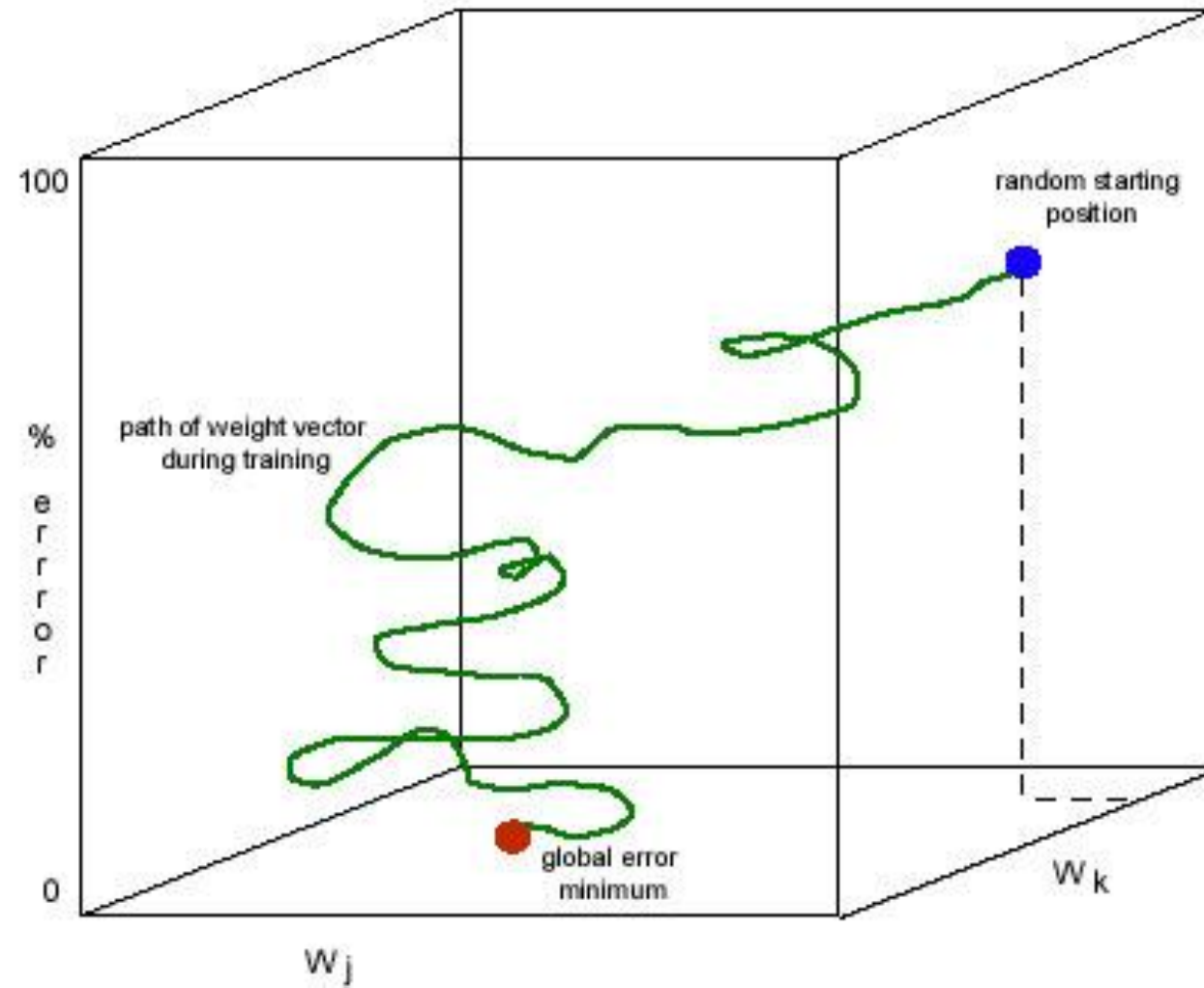
État synaptique d'un réseau simple

Analogie fonctionnelle: apprentissage pragmatique dirigé par l'erreur

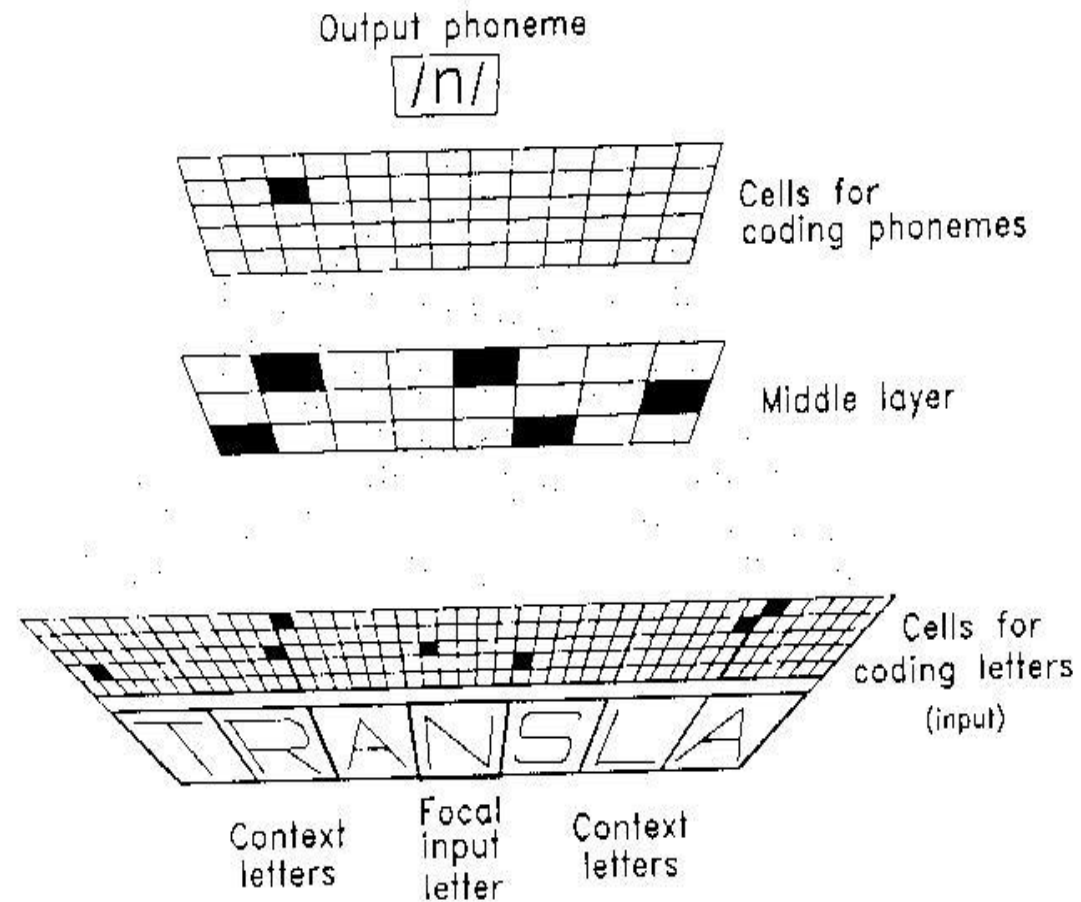


Apprentissage dans un réseau

Analogie fonctionnelle: apprentissage pragmatique dirigé par l'erreur



Analogie fonctionnelle: un modèle de l'apprentissage (apprentissage de tâches, dirigé par l'erreur)



Analogie fonctionnelle: un n (apprentissage de tâches, di

Analyzing the Internal Representations of Trained Neural Networks

John A. Bullinaria

Centre for Speech and Language, Department of Psychology
Birkbeck College, Malet Street, London WC1E 7HX, UK
j.bullinaria@physics.org

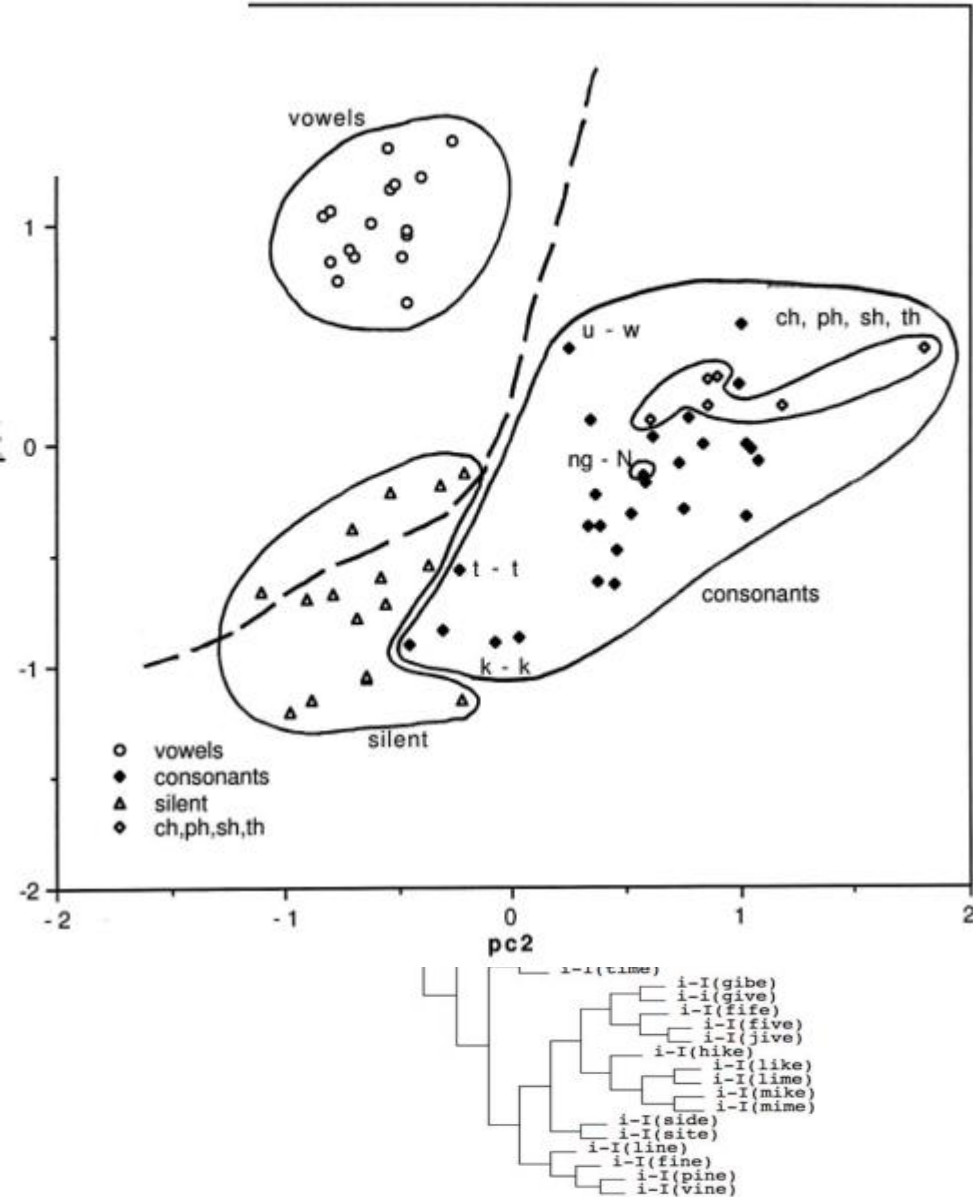


Figure 3: HCA of representative 'i' words in hidden unit space.

Analogie fonctionnelle: un n (apprentissage de tâches, di

Analyzing the Internal Representations of Trained Neural Networks

John A. Bullinaria

Centre for Speech and Language, Department of Psychology
Birkbeck College, Malet Street, London WC1E 7HX, UK
j.bullinaria@physics.org

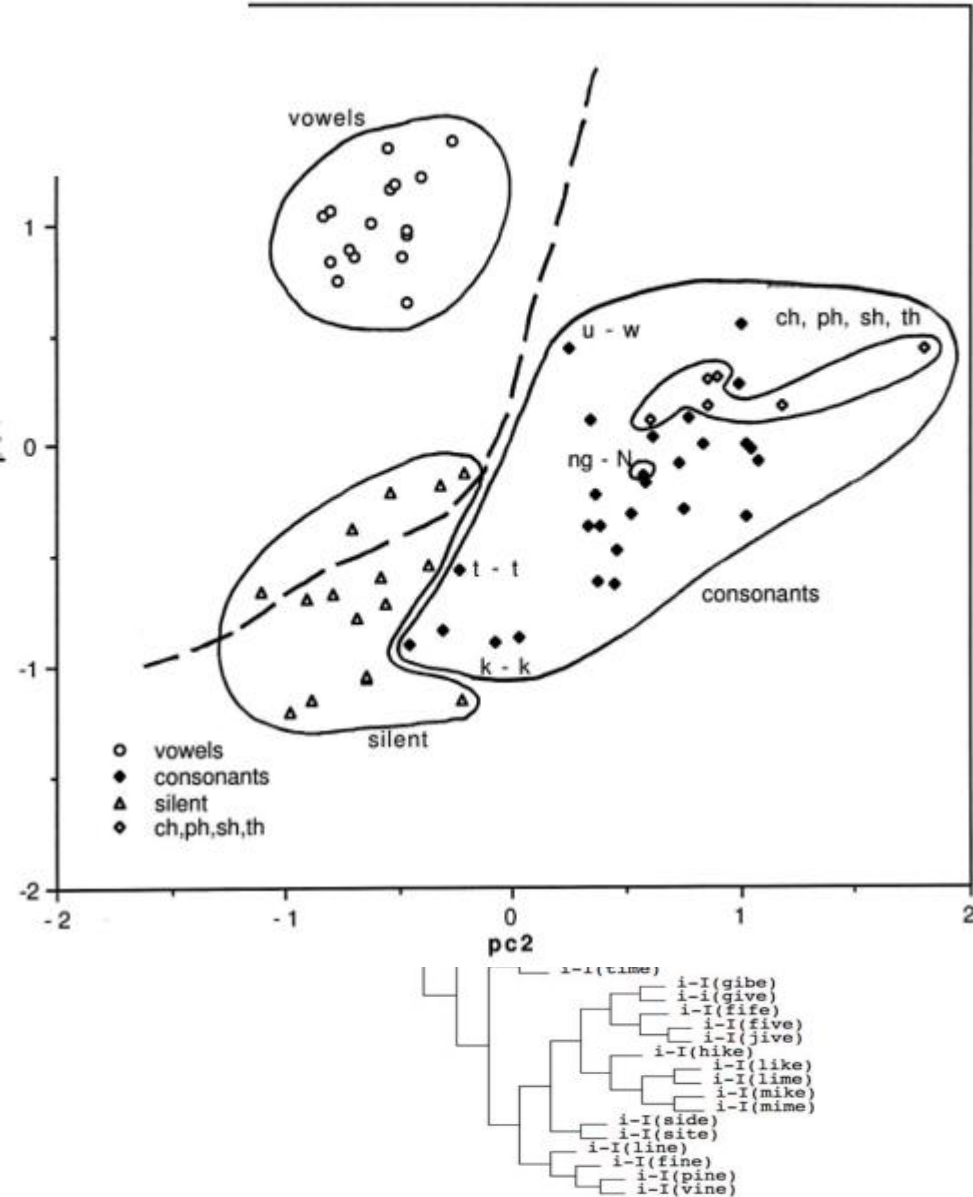
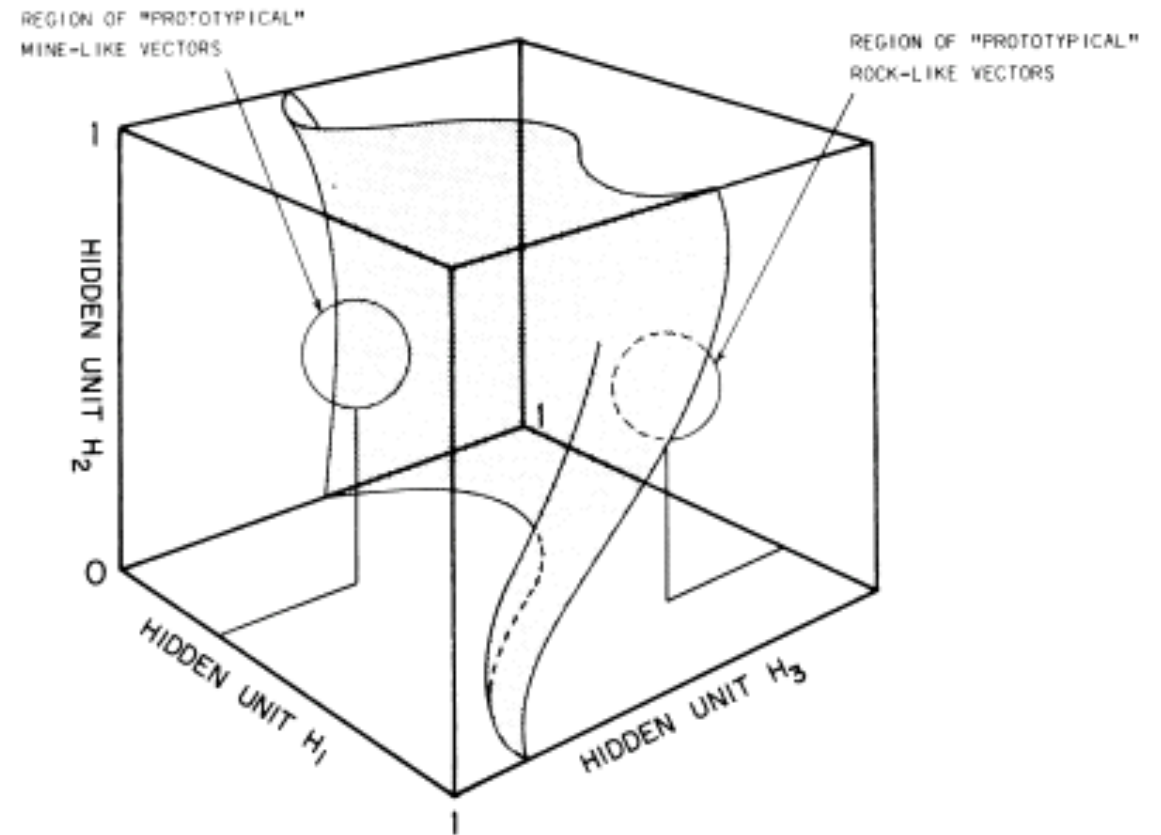
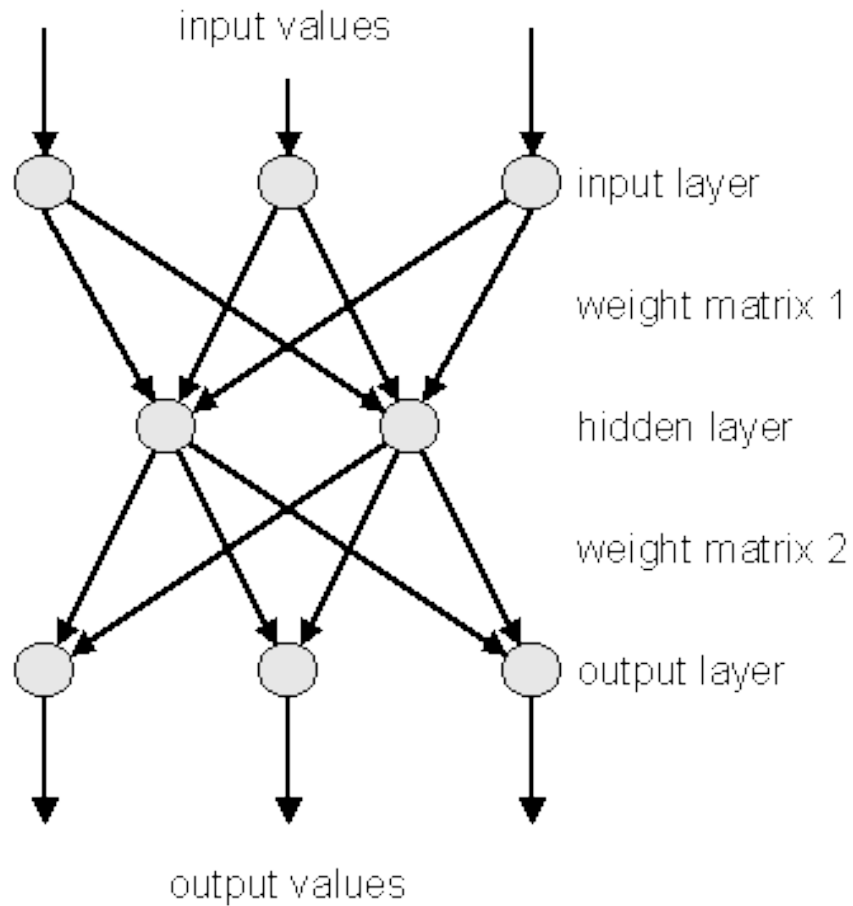
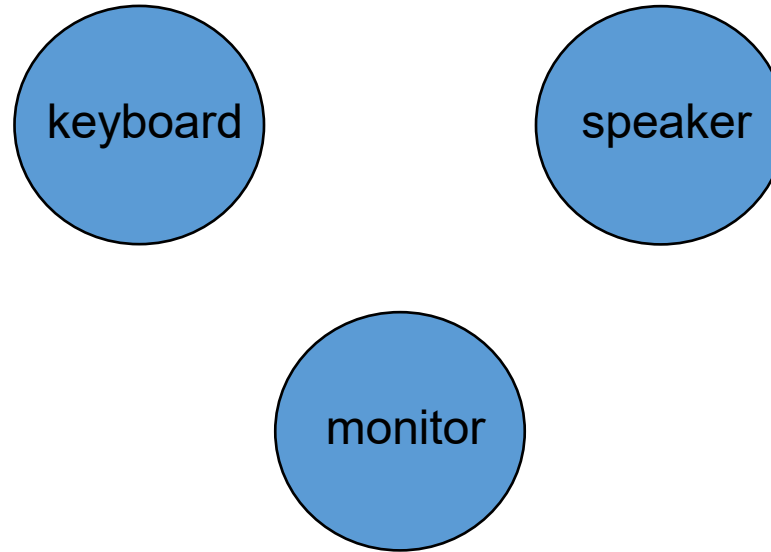


Figure 3: HCA of representative 'i' words in hidden unit space.

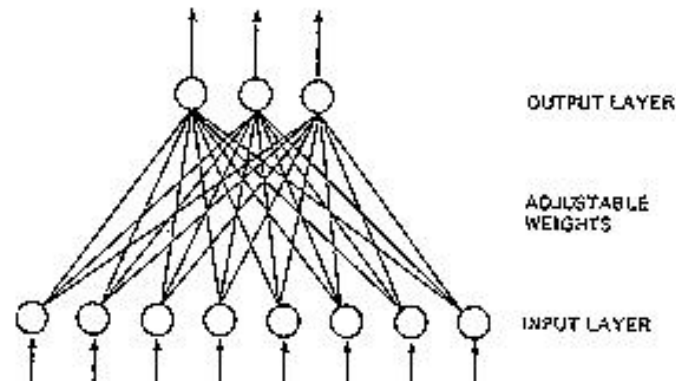
Analogie fonctionnelle: la sémantique représentationnelle des réseaux



Analogie fonctionnelle: la sémantique représentationnelle des réseaux



Analogie fonctionnelle: la sémantique représentationnelle des réseaux



A $\longrightarrow$ $\langle 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 \dots, 0 \rangle$
26

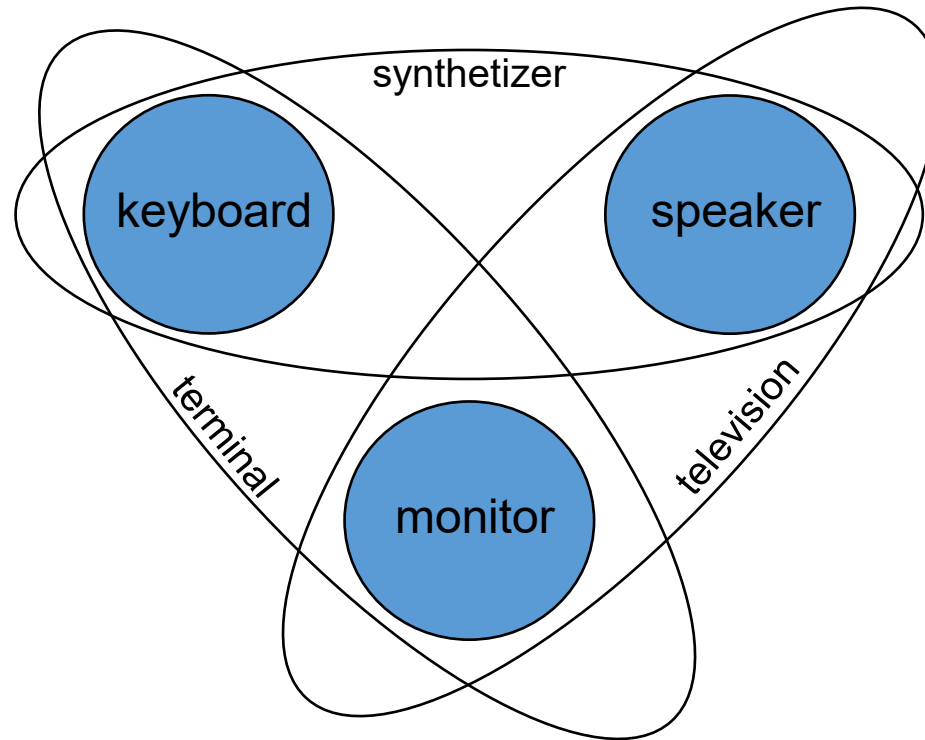
B $\longrightarrow$ $\langle 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 \dots, 0 \rangle$

C $\longrightarrow$ $\langle 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 \dots, 0 \rangle$

...

Z $\longrightarrow$ $\langle 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 \dots, 1 \rangle$

Analogie fonctionnelle: la sémantique représentationnelle des réseaux

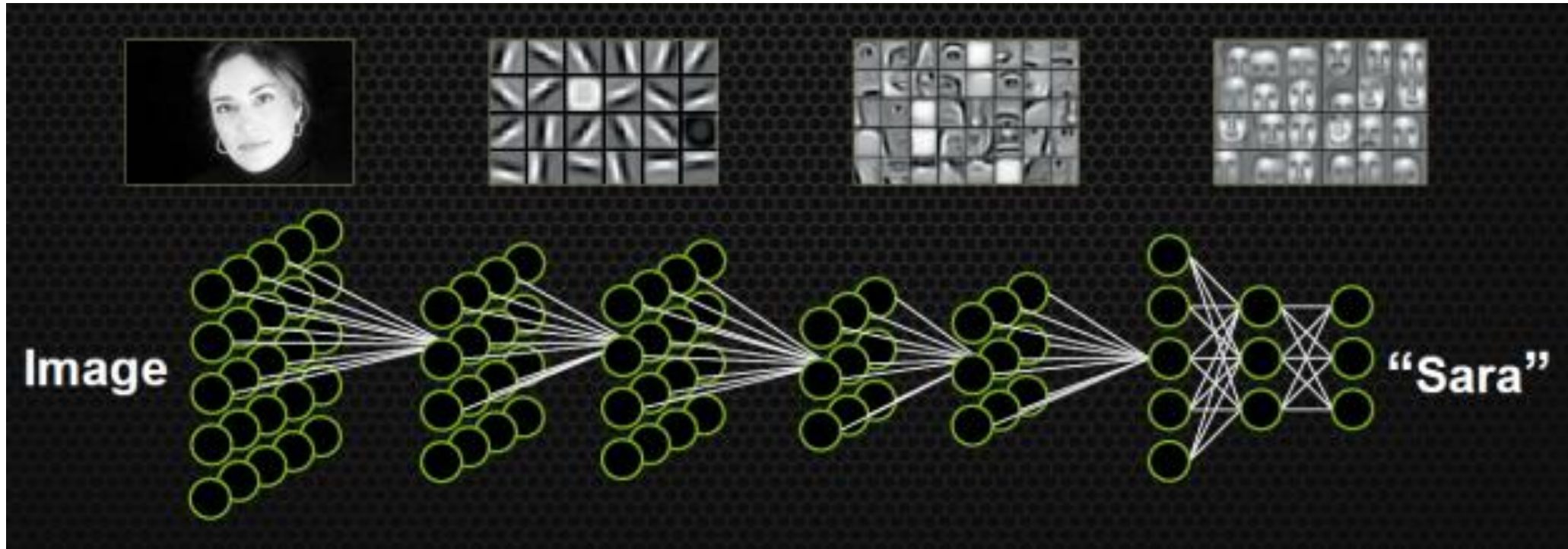


Synthétiseur: $\langle 1, 1, 0 \rangle$

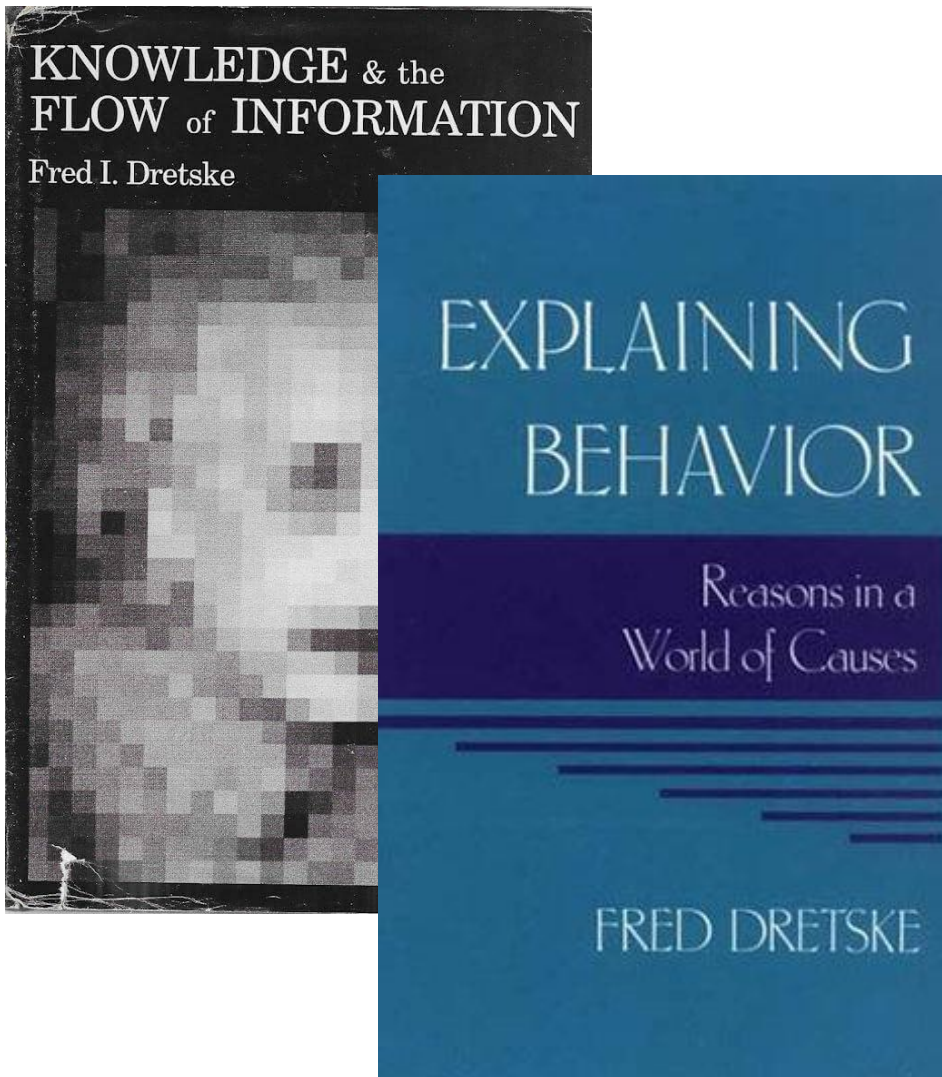
Terminal: $\langle 1, 0, 1 \rangle$

Téléviseur: $\langle 0, 1, 1 \rangle$

Analogie fonctionnelle: la sémantique représentationnelle des réseaux

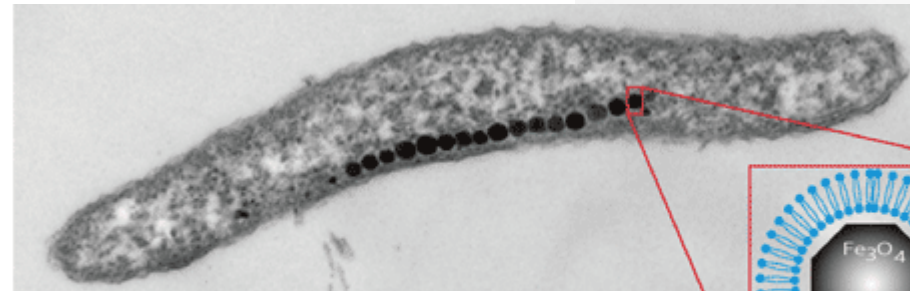
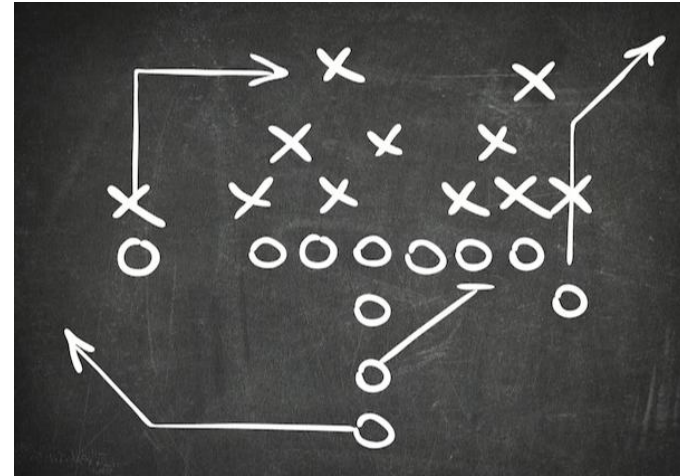
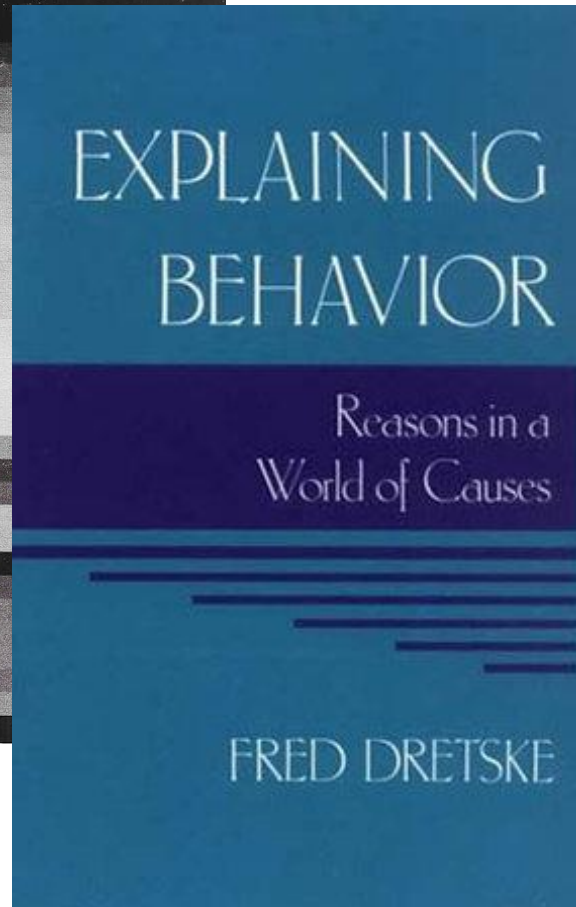
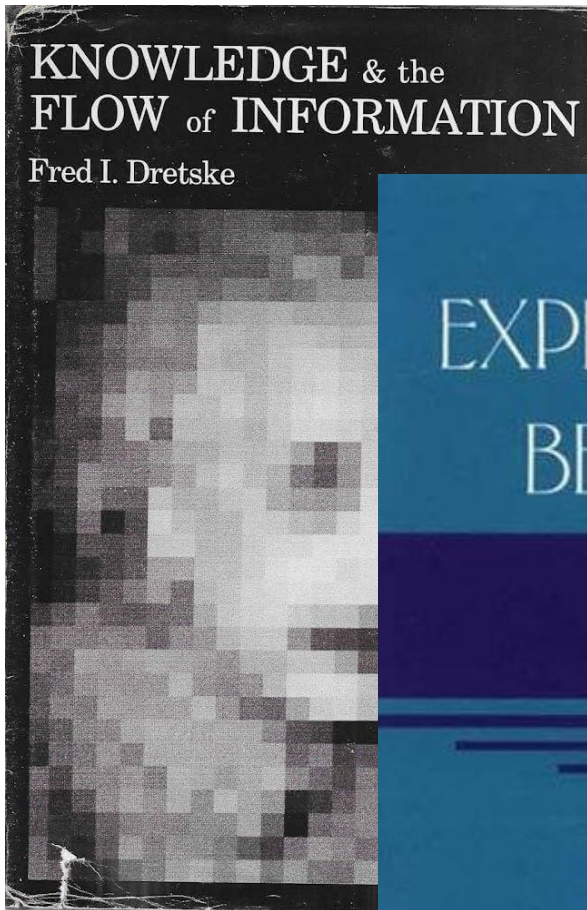


Analogie fonctionnelle: la sémantique représentationnelle des réseaux

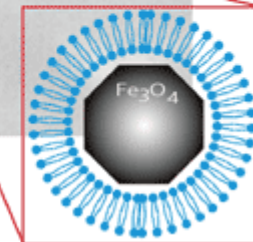


- Un état E d'un système S représente un état M du monde si, et seulement, si 1) il y a une corrélation nomologique entre E et M, et 2) E a la fonction de représenter M pour S. (Dretske, 1988)
- Trois types de représentations
 1. Conventionnelle
 2. Indicielles (indices de Peirce)
 3. Naturelles

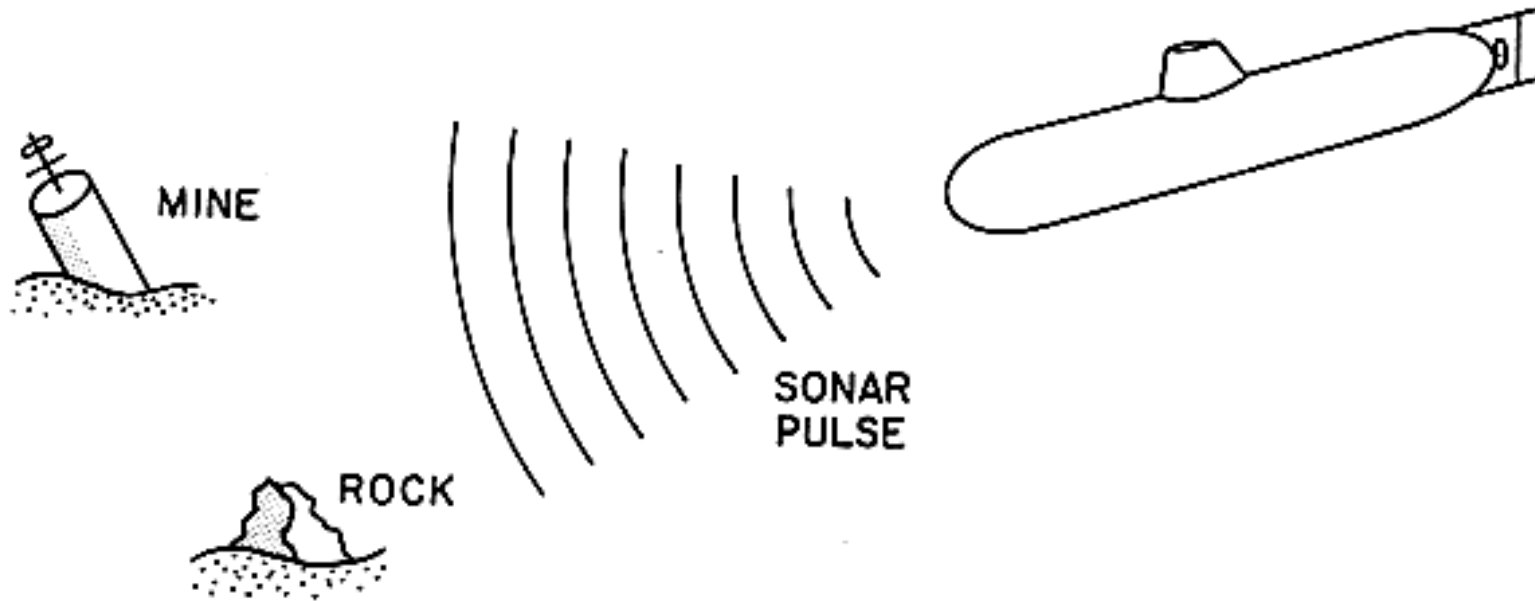
Analogie fonctionnelle: la sémantique représentationnelle des réseaux



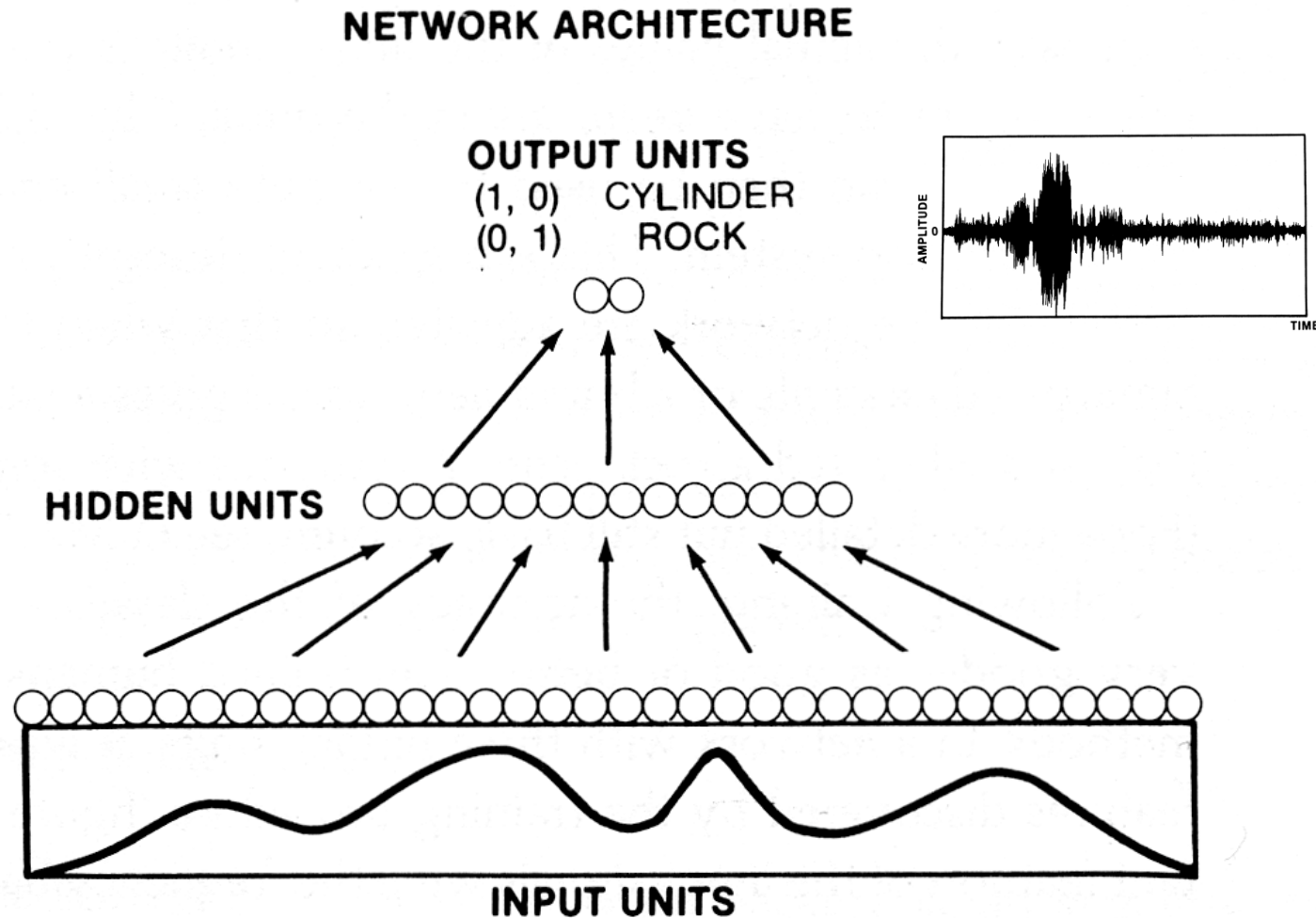
Schüler D, Genetics and cell biology of magnetosome formation in magnetotactic bacteria, *FEMS Microbiology Reviews*, 2008



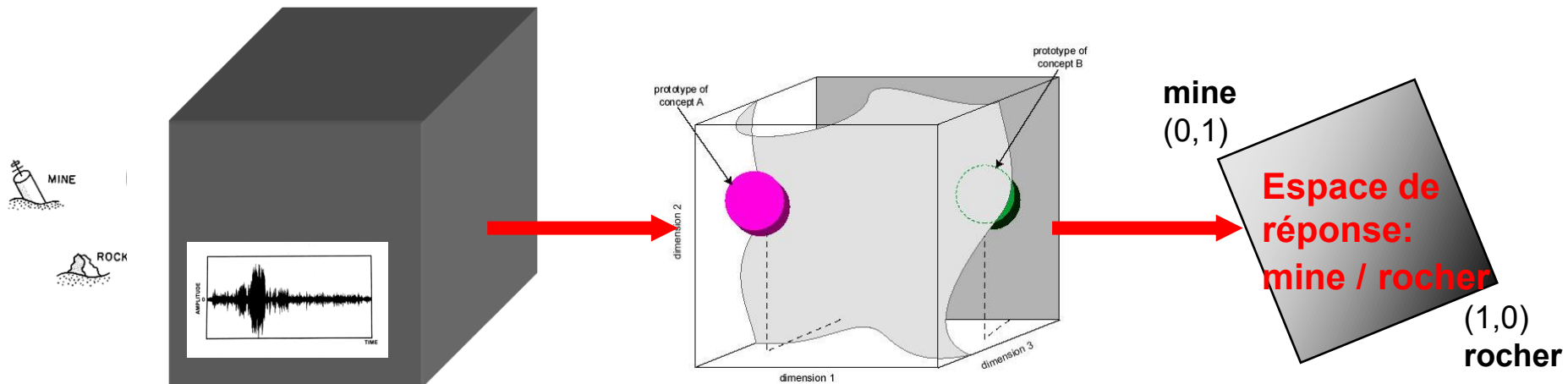
Analogie fonctionnelle: la sémantique représentationnelle des réseaux



Analogie fonctionnelle: la sémantique représentationnelle des réseaux



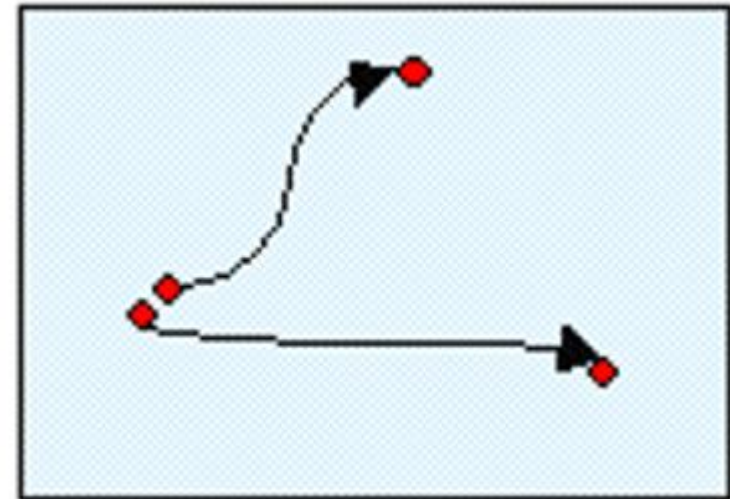
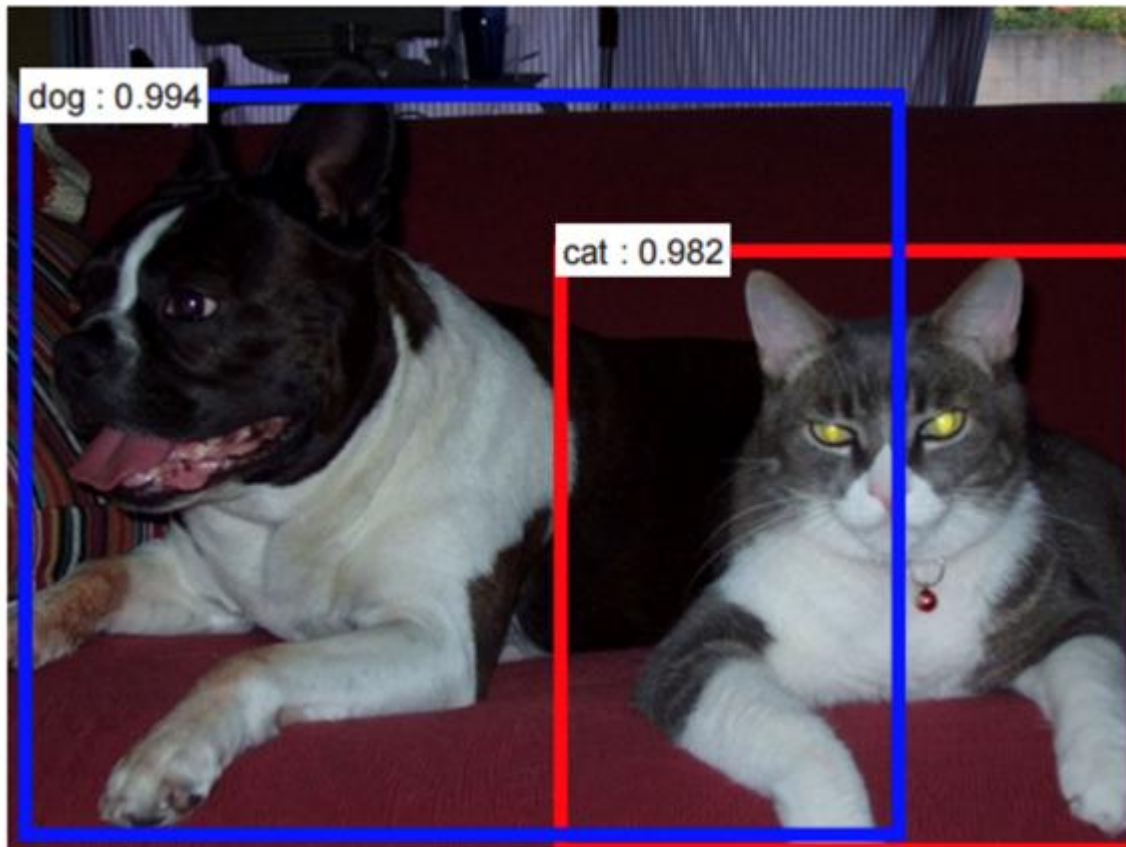
Analogie fonctionnelle: la sémantique représentationnelle des réseaux



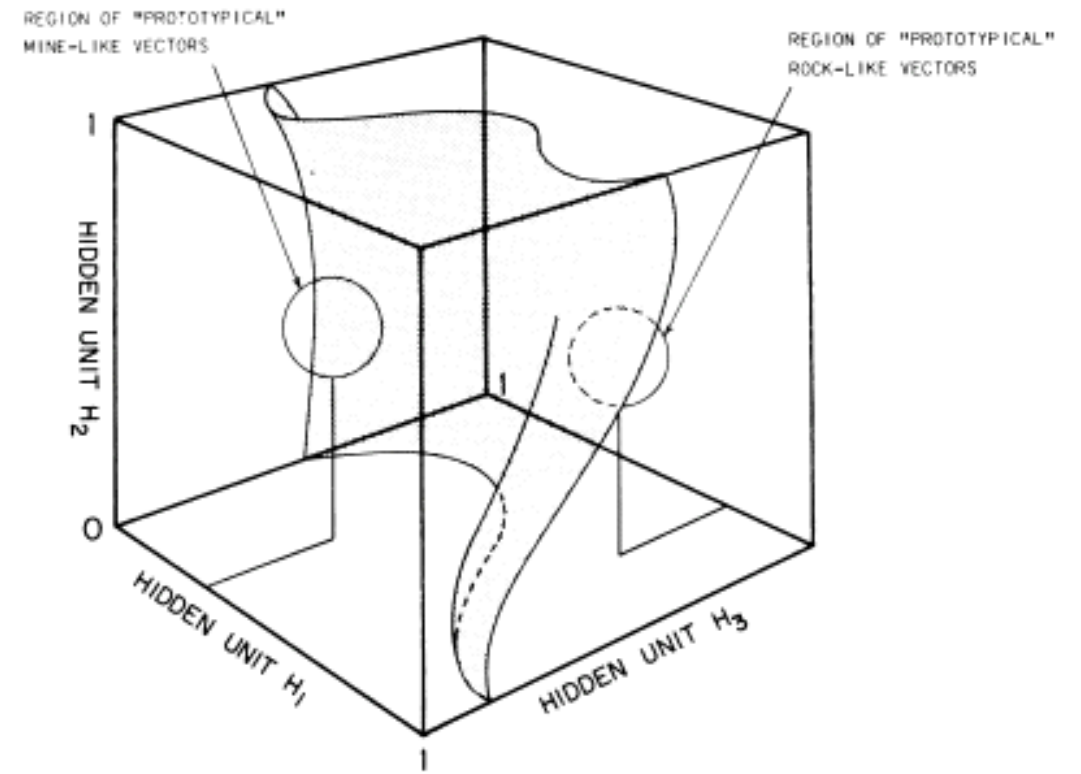
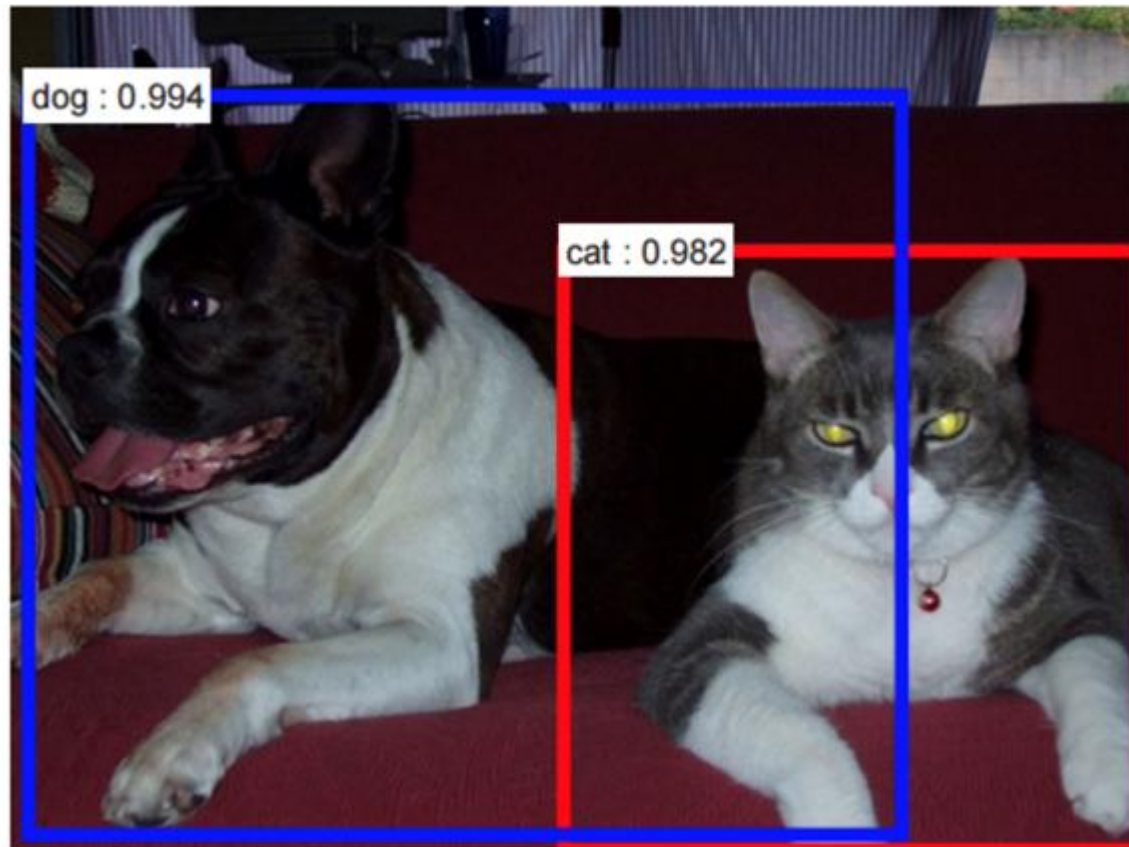
Sémantique informationnelle:

Un état E d'un système S (ex.: un état d'activation de la couche cachée d'un réseau de neurones) représente un état M du monde (ex.: la présence d'une mine) si, et seulement, si 1) il y a une corrélation nomologique entre E et M, et 2) E a la fonction de représenter M pour S. (Dretske, 1988).

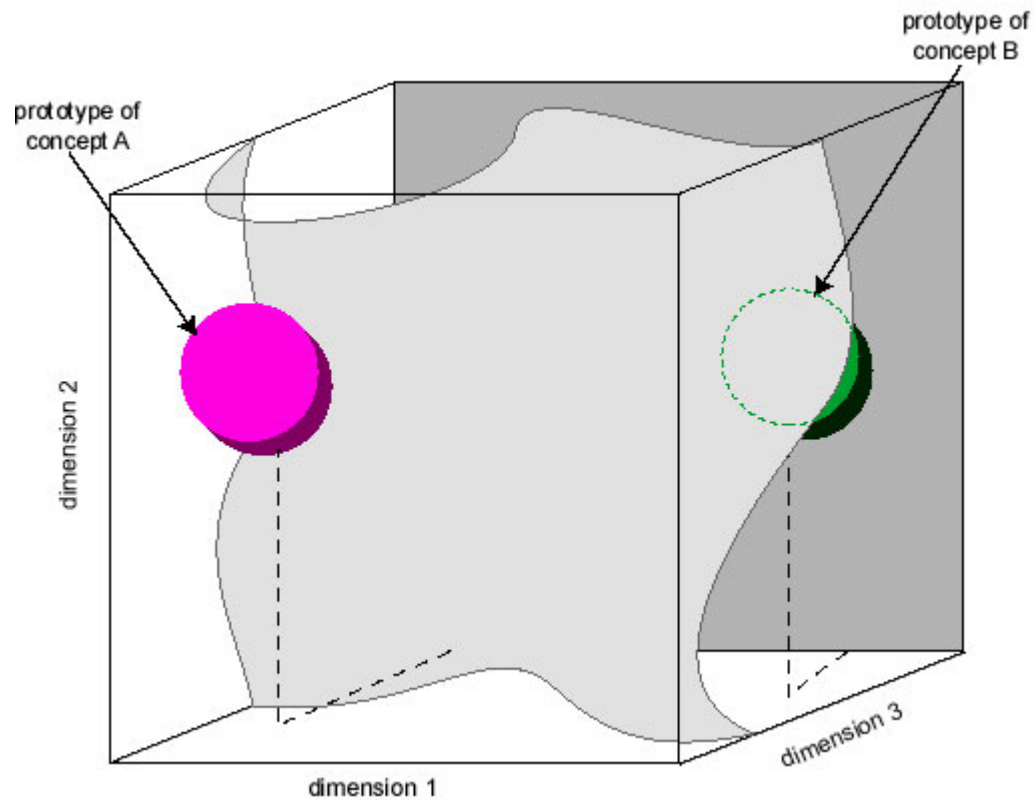
Analogie fonctionnelle: la sémantique représentationnelle des réseaux



Analogie fonctionnelle: la sémantique représentationnelle des réseaux

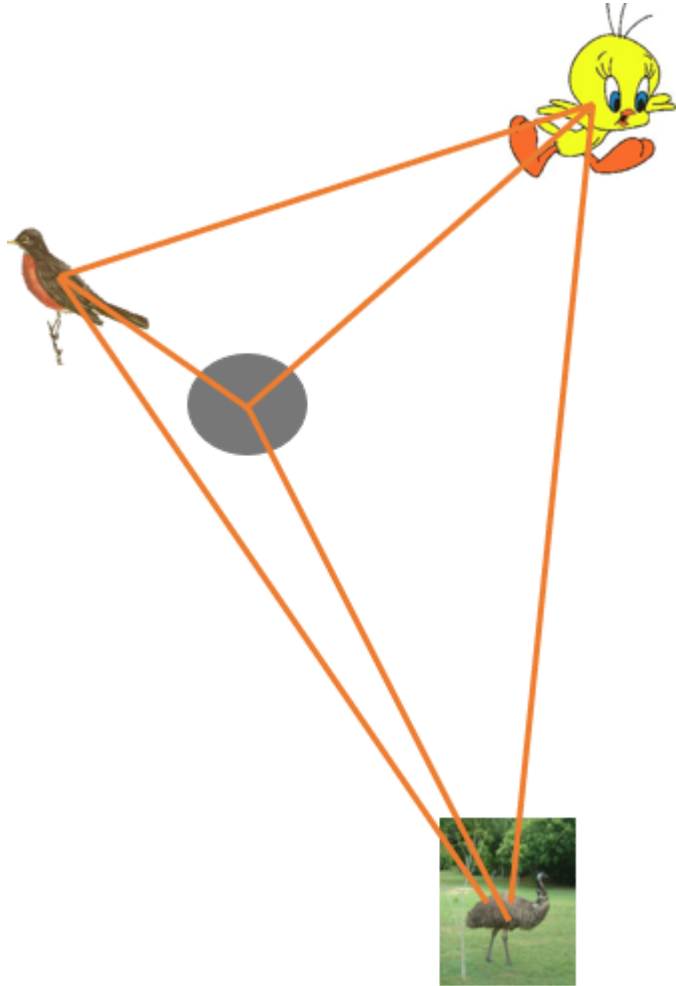


Analogie fonctionnelle: la sémantique représentationnelle des réseaux



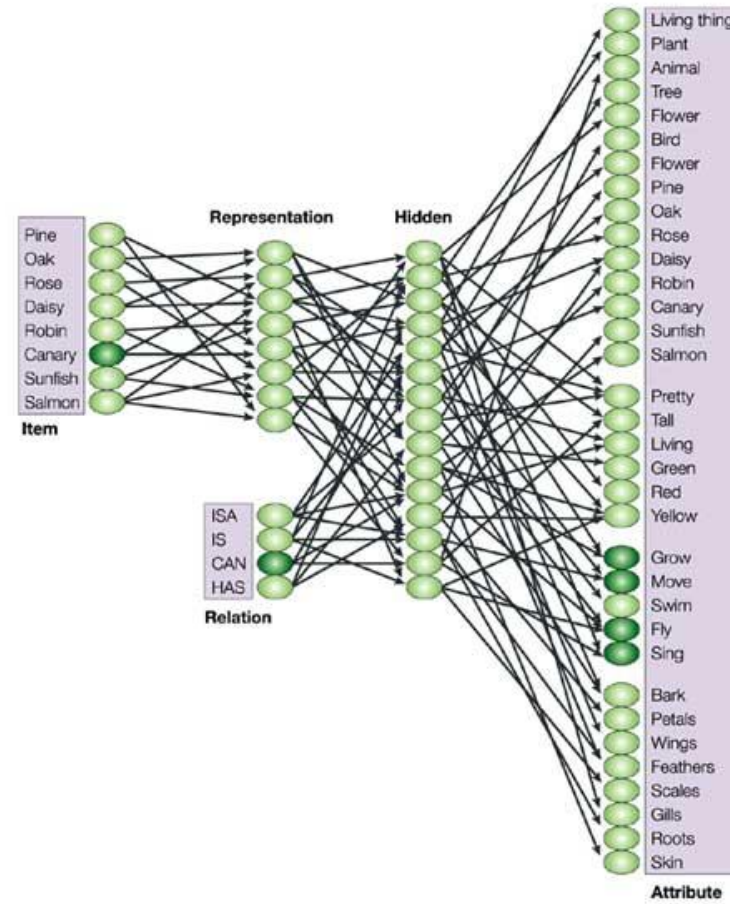
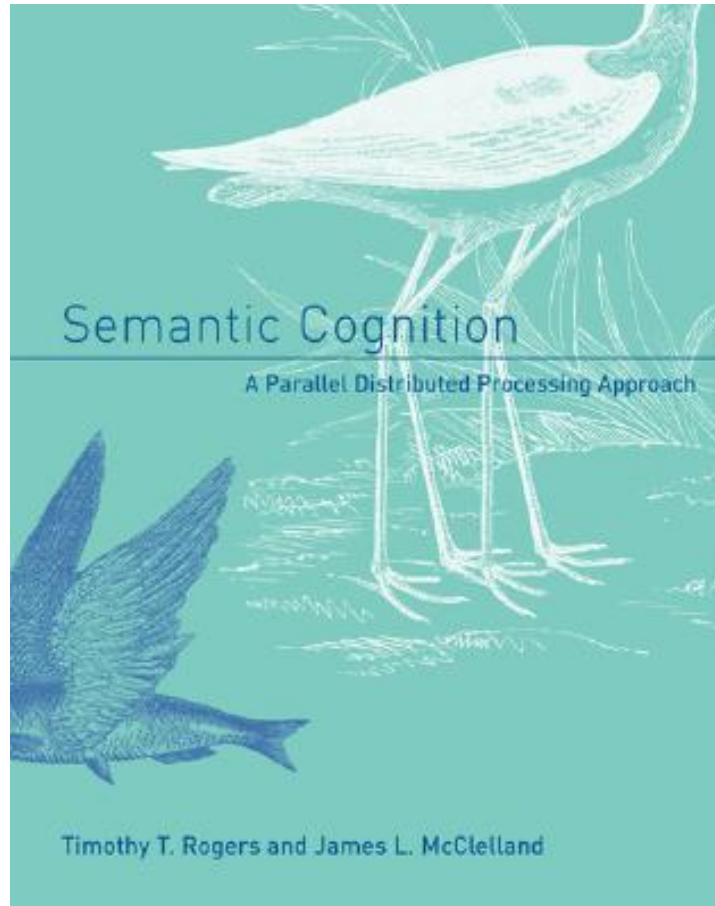
- Propriétés des représentations
 - Métrique sémantique
 - Généralisation automatique
 - Résistance au bruit
 - Dégradation progressive

Analogie fonctionnelle: la sémantique représentationnelle des réseaux



- Propriétés des représentations
 - Métrique sémantique
 - Généralisation automatique
 - Résistance au bruit
 - Dégradation progressive

Analogie fonctionnelle: la sémantique représentationnelle des réseaux



Analogie fonctionnelle: la sémantique représentationnelle des réseaux

